

DATA SCIENCE 2 - DATA & A.I.

3

V MACHINE LEARNING

1 WHAT IS MACHINE LEARNING

PYTHON BASICS

Python for data science



WORKING WITH ARRAYS

Numpy



DATA ENGINEERING

pandas



DATA SCIENCE 2 DATA & A.I. 3



DATA VISUALISATION

Matplotlib



MACHINE LEARNING

Automatically find patterns

V



MACHINE LEARNING

scikit-learn

WHAT IS MACHINE LEARNING

Automatically find patterns

01

INTRODUCING SCIKIT-LEARN

Machine learning with Python

02

HYPERPARAMETERS AND CROSS VALIDATION

Holdout samples
and cross-validation

03

REGRESSION

Best fitting line

04

MACHINE LEARNING

05

DECISION TREES

Best separating lines

06

K-MEANS CLUSTERING

Object grouping

07

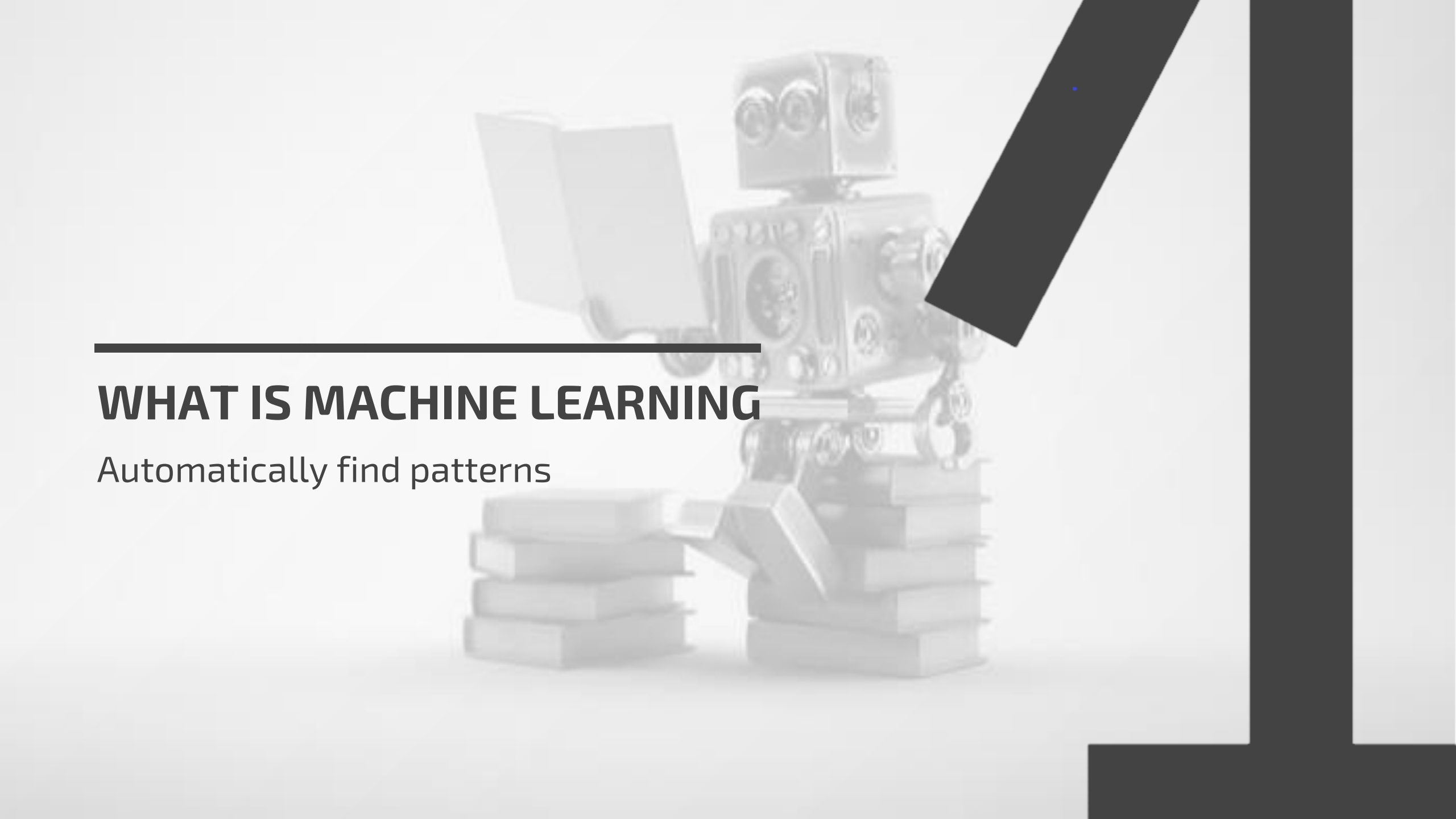
ASSOCIATION RULES

Frequent itemsets

08

ARTIFICIAL NEURAL NETWORK

Imitate the human brain



WHAT IS MACHINE LEARNING

Automatically find patterns

BUSINESS

DATA DRIVEN
DECISION SUPPORT

DESIGN THINKING

BUSINESS
STRATEGY

BUSINESS
ANALYSIS

BUSINESS
INTELLIGENCE

BUSINESS
TRANSLATION

DATA ADMINISTRATION

DATA
GOVERNANCE

DATA
ARCHITECTURE

DATA
INFRASTRUCTURE

DATA
PROCESSING

DATA
INTEGRATION

DATA
PREPARATION

DATA ANALYSIS (OBJECTIVES)

DESCRIPTIVE
ANALYSIS

EXPLORATIVE
ANALYSIS

CONFIRMATORY
ANALYSIS

PREDICTIVE
ANALYSIS

PRESCRIPTIVE
ANALYSIS

OPERATIONS
RESEARCH

DATA ANALYSIS (TECHNIQUES)

SUMMERIZATION

VISUALISATION

STATISTICS

MACHINE
LEARNING

EXPLAINABLE AI

OPTIMIZATION

DEPLOYMENT & INTEGRATION

DASHBOARDING

MODEL
DEPLOYMENT

CLOUD
INTEGRATION

EMBEDDED
SYSTEMS

IOT

SENSORS &
ACTUATORS

WHY

Why are we doing this

What do we want to achieve

What do we want as a result



DATA ANALYSIS (OBJECTIVES)

DESCRIPTIVE
ANALYSIS

EXPLORATIVE
ANALYSIS

CONFIRMATORY
ANALYSIS

PREDICTIVE
ANALYSIS

PRESCRIPTIVE
ANALYSIS

OPERATIONS
RESEARCH

DATA ANALYSIS (TECHNIQUES)

SUMMERISATION

VISUALISATION

STATISTICS

MACHINE
LEARNING

EXPLAINABLE AI

OPTIMISATION



How are we doing this

What steps will we take

What techniques will we use

HOW

DATA ANALYSIS

Formal test of a model (confirmation)
Implicit significance testing
(likelihood pattern is observed by
coincidence)

- Difficult to use when many variables are present and model unknown
 - Assumptions on distribution characteristics of population



VISUAL EXPLORATION

Get insights from the data

- Difficult to visualize more than 3 variables
- Inconsistent interpretation



MACHINE LEARNING

Let the computer derive a pattern
No distribution characteristics
needed
=> Far more methods available to
find patterns

- No implicit significance testing
 - Beware of overfitting



STATISTICAL ANALYSIS

MACHINE LEARNING

“TRADITIONAL” DATA ANALYSIS

- Formulate hypothesis
- Collect data
- Explore data (and refine hypothesis if needed)
- Build model according to hypothesis and exploration
- Test for statistical significance (test for coincidence because of sample)
- Derive conclusions

=> SEARCH FOR PATTERNS YOURSELF

=> STARTS FROM HYPOTHESIS OR EXPLORATION

MACHINE LEARNING

- Collect data
- Train model
- Test model (test for coincidence because of sample)
- Derive conclusions

=> LET THE COMPUTER SEARCH FOR PATTERNS

=> STARTS FROM DATA

WHAT KIND OF PROBLEMS CAN YOU SOLVE WITH MACHINE LEARNING

- Value estimation / regression: predict a continuous variable
e.g. sales prediction; car use
- Classification: predict a categorical variable
e.g. churn prediction; diagnosis
- Segmentation / clustering: split or group cases/observations
e.g. customer segmentation; document topic search
- Co-occurrence / association rule discovery: events happening together
e.g. market basket analysis ; recommendation system

WHAT KIND OF PROBLEMS CAN YOU SOLVE WITH MACHINE LEARNING

**S
U
P
E
R
V
I
S
E
D**

- 
- Value estimation / regression: predict a continuous variable
e.g. sales prediction; car use
 - Classification: predict a categorical variable
e.g. churn prediction; diagnosis

- Segmentation / clustering: split or group cases/observations
e.g. customer segmentation; document topic search
 - Co-occurrence / association rule discovery: events happening together
e.g. market basket analysis ; recommendation system
- 

**U
N
S
U
P
E
R
V
I
S
E
D**

MAJOR DATA ANALYTICS SUBTASKS

CLUSTERING

I want to group/segment my cases, but

- I have no clue what kind of classification to use (how many classes and/or what kind of classes) => exploration
- I have a clue about the classification to use, but I'm unable to get example cases (labeled cases) => prediction

Make groups based on some kind of similarity in features/variables

Use case: Exploration, segmentation

Methods: Hierarchical clustering (Ward, Single linkage Complete linkage, ...);

Non-hierarchical clustering (k-means)

MAJOR DATA ANALYTICS SUBTASKS

ASSOCIATION RULE DISCOVERY

I want to group events/features that happen together

Similar to clustering, but now we do not group cases (rows) but events/features (mostly columns/variables)

Make groups based on co-occurrence of events/features (frequent itemsets)

Use case: Recommender systems, market basket analysis

Methods: Association rule mining

(subtle difference with clustering: clustering will reveal which cases have something in common, association rule discovery will reveal what they have in common. You could get the same kind of information by interpreting your cluster solution, but better to do that by directly using association rule discovery)

MAJOR DATA ANALYTICS SUBTASKS

ESTIMATION

Predict the value of a numerical value based on a set of features (set of numerical or categorical variables)

Regress a model based on examples (labeled cases)

Use case: All kind of numerical predictions

Methods: (linear) regression

MAJOR DATA ANALYTICS SUBTASKS

CLASSIFICATION

Predict the value of a categorical value based on a set of features (set of numerical or categorical variables)

Related to clustering, but now we know the classes and we have example cases (labeled cases)

Make groups based by deriving rules or deriving hyperplanes based on example cases (labeled cases)

Use case: All kind of classifications

Methods: Decision tree; logistic regression; SVM; artificial neural network

(can also be used for estimation if you transform your numerical variable into a categorical variable using ranges)

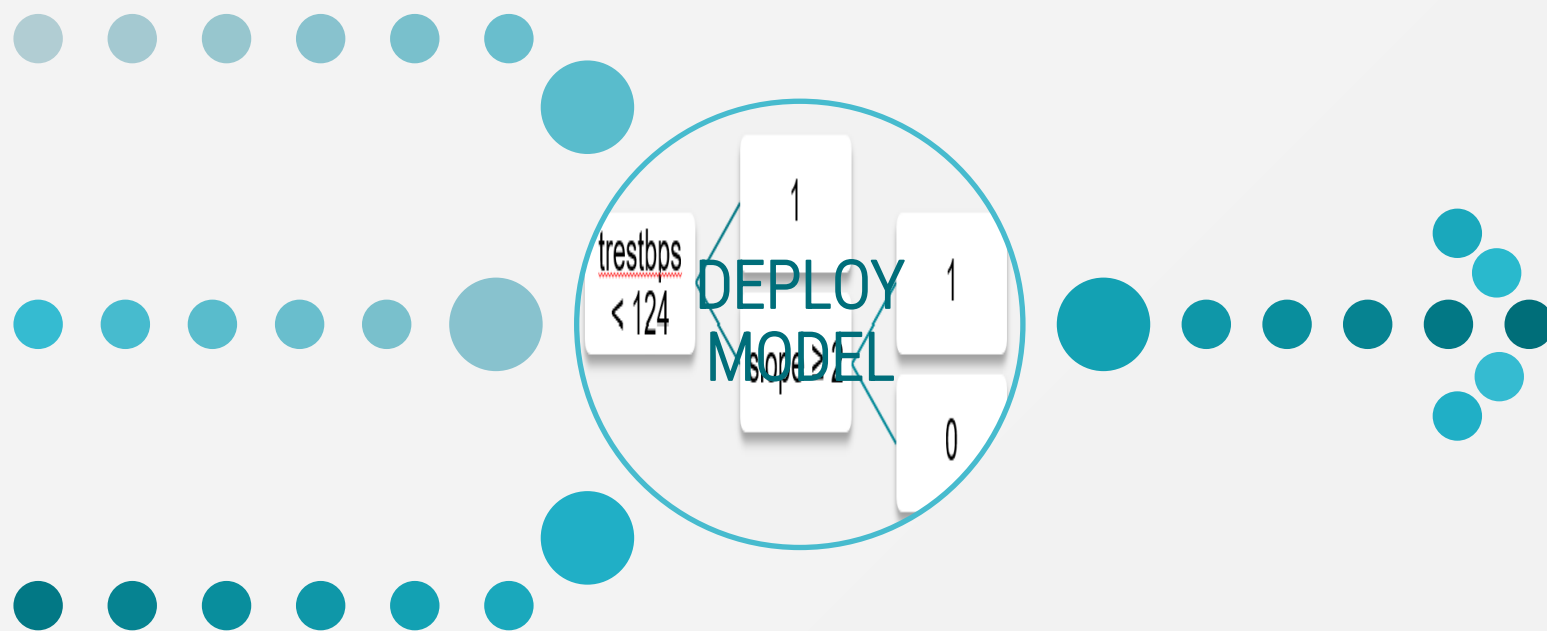
SUPERVISED LEARNING : TRAINING (DERIVE A MODEL FROM LABELED DATA)

Examples (labeled cases)

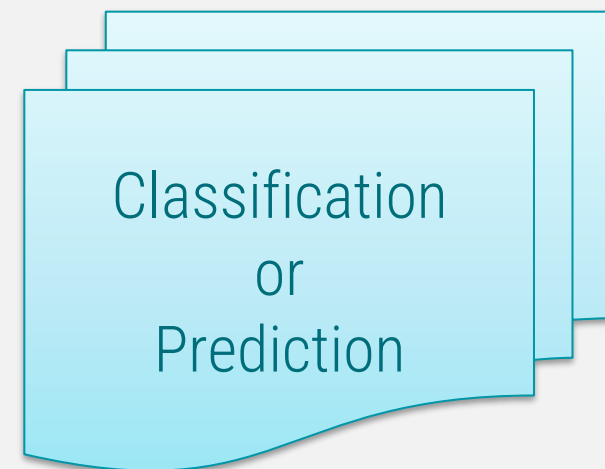


SUPERVISED LEARNING : DEPLOYMENT (APPLY MODEL ON NEW DATA)

New data (new cases)



Results



DATA SCIENCE 2 – DATA & A.I. 3

V MACHINE LEARNING

2 INTRODUCING SCIKIT-LEARN

PYTHON BASICS

Python for data science



WORKING WITH ARRAYS

Numpy



DATA ENGINEERING

pandas



DATA SCIENCE 2 DATA & A.I. 3



DATA VISUALISATION

Matplotlib



MACHINE LEARNING

Automatically find patterns

V



MACHINE LEARNING

scikit-learn

WHAT IS MACHINE LEARNING

Automatically find patterns

01

INTRODUCING SCIKIT-LEARN

Machine learning with Python

02

HYPERPARAMETERS AND CROSS VALIDATION

Holdout samples
and cross-validation

03

REGRESSION

Best fitting line

04

MACHINE LEARNING

05

DECISION TREES

Best separating lines

06

K-MEANS CLUSTERING

Object grouping

07

ASSOCIATION RULES

Frequent itemsets

08

ARTIFICIAL NEURAL NETWORK

Imitate the human brain

INTRODUCING SCIKIT-LEARN

Machine learning with Python

scikit

learn

SCIKIT-LEARN

PYTHON PACKAGE/LIBRARY FOR MACHINE LEARNING

- General purpose machine learning library (for supervised and unsupervised learning)
- Very commonly used

CONSISTENT INTERFACE TO MANY MACHINE LEARNING TECHNIQUES

Common methods to.

- Transform data
- Train model
- Predict data
- Validate model

=> **Uniform access to many machine learning techniques**

DATA PREPARATION

FEATURES

Supervised learning is a kind of dependency model, i.e. a model in which a feature/variable (the dependent variable) is estimated based on a series of other features/variables (independent variables)

In statistics, one talks about dependent and independent variables, in machine learning one talks about target features and predictors or independent features.

- **Predictors** (independent features/variables) : the features that can be used to estimate the target feature
- **Target feature** (dependent feature/variable) : the feature to be estimated

LABELED DATASET

Supervised machine learning is based on training, i.e. deriving a model based on examples, with one example being one set of predictors and the known outcome for that set of predictors. Hence, for supervised machine learning, one needs a labeled dataset, a dataset with example with for every example the known outcome (the known outcome of an example is also called a label)

So a labeled dataset contains a set of example features (dependent features or predictors) and the known outcome or label for each of those examples.

DATA PREPARATION

SCIKIT-LEARN : FEATURE MATRIX AND TARGET ARRAY

- **X** = feature matrix : the feature matrix is commonly referred to as X (capital X because it is a matrix)
- **y** = target array : the target array is commonly referred to as y (small y because it is mostly a vector,) (capital Y if it is a matrix – for multivariate scikit)

Mind that feature matrix **X must be a 2-dimensional structure or matrix in scikit-learn, even if there is only one single feature or predictor. So in case of a single feature in a Pandas Series or a one-dimensional NumPy array, that Pandas series needs to be converted to a **Pandas DataFrame** explicitly, or the one-dimensional NumPy array must be converted to a **two-dimensional NumPy array** explicitly.**

DATA PREPARATION

SCIKIT-LEARN : FEATURE MATRIX AND TARGET ARRAY

So the starting point of supervised machine learning is a labeled dataset. In scikit-learn, this labeled dataset is split into two arrays or matrices:

- **Feature matrix** : the table with the values of the independent features or predictors. It is a table structure because it consists of rows (examples) and columns (independent features or predictors). As it is a table, it can be represented in Python by a [Pandas DataFrame](#), or by a [2-dimensional NumPy array](#) (matrix)
- **Target array** : the vector with the labels, the values of the dependent features or target feature. It is mostly a vector because it contains the labels for a single target feature (one label for every example). As it is a vector, it can be represented in Python by a [Pandas Series](#), or by a [1-dimensional NumPy array](#) (vector). However, some scikit-learn methods can handle multiple target features at once, and in that case the target array also becomes a table that can be represented by a pandas dataframe or a 2-dimensional NumPy array (matrix)

Mind that in many cases the independent features are predictors and the target feature are interchangeable, i.e. any feature can play the role of predictor or target.

DATA PREPARATION

Feature Matrix (X)

n_features $\rightarrow$

```
← n_samples
```

[illegible]

Target Vector (y)

```
← n_samples
```

[illegible]

DATA PREPARATION

SCIKIT-LEARN : NORMALIZATION

It is often a good idea to normalize the [feature matrix](#). Normalization is a statistical method that helps mathematical-based algorithms to interpret features with different magnitudes and distributions equally. The most used normalization techniques is transforming your input data are:

- **StandardScaler** : scaling method so that all features have a mean of 0 and a standard deviation of 1. These are the Z-scores:
$$x_{\text{norm}} = (x - \text{mean}) / \text{std_dev}$$
- **MinMaxScaler** : scaling method so that all features are in a range between 0 and 1 : $x_{\text{norm}} = (x - \text{min}) / (\text{max} - \text{min})$

NORMALIZATION WITH SCIKIT-LEARN

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
scaler = StandardScaler() # or MinMaxScaler()
X_norm = scaler.fit_transform(X)
```

Methods

`scaler.fit(X)` : compute the mean and standard deviation of all features (or minimum and maximum) to be used for later scaling.

`X_norm = scaler.transform(X)`: transforms (scales) all the features, based on the fit. Returns array of shape (n_samples, n_features).

`X_norm = scaler.fit_transform(X)`: fit all features and transform (scale) them. Returns array of shape (n_samples, n_features).

`X = scaler.inverse_transform(X_norm)`: undo the scaling for all features. Returns array of shape (n_samples, n_features).

REMARK

If you want the result in a Pandas DataFrame, the following can be used:

```
X_norm = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
```

MODEL VALIDATION BASICS (SUPERVISED LEARNING)

COMPARE PREDICTED VALUES WITH REAL VALUES (LABELS/GROUND TRUTH/GOLDEN STANDARD)

For supervised learning, model validation is easy. As you start from a labeled dataset, you have a set of examples for which you know the real outcome (labels). This is also called the ground truth or gold standard. So you can compare the predicted values with the real values (labels):

- Derive (train/fit) a model based on labeled data
- Use the derived model to predict the target variable in the labeled data based on the independent features/predictors in the labeled data
- Compare the predicted target feature values from the previous step with the real target feature values (labels) in the labeled dataset

HOW TO COMPARE REAL AND PREDICTED VALUES FOR REGRESSION METHODS (PREDICTION OF NUMERICAL FEATURES)

Start calculating the difference between the real and predicted values. Based on those differences, validation metrics can be calculated:

- MAE : mean absolute error
- MAPE : mean absolute percentage error
- RMSE : root mean squared error
- ...

MODEL VALIDATION BASICS

(SUPERVISED LEARNING)

HOW TO COMPARE REAL AND PREDICTED VALUES FOR CLASSIFICATION METHODS (PREDICTION OF CATEGORICAL FEATURES)

Start compiling the confusion matrix based on the number of correctly and wrongly predicted cases. Based on this confusion matrix, validation metrics can be calculated:

- Accuracy
- Precision
- Recall
- F-value
- AUC
- ROC
- ...

SEE DATA SCIENCE 1 / DATA & A.I. 2 AND SCIKIT-LEAN DOCUMENTATION

- sklearn.metrics : <https://scikit-learn.org/stable/api/sklearn.metrics.html>

DATA ANALYTICS PIPELINE

DATA PREPARATION

- Load (labeled) source data
- Compile feature matrix
- Compile target array (for supervised methods)

MODEL SELECTION AND HYPERPARAMETER SELECTION (MODEL SPECIFIC)

- Decide on the method to use (linear regression, decision tree, K-means clustering, ...)
- Decide on the hyperparameter to use (degree of polynomial, tree depth, number of clusters, ...)
 - Hyperparameters are parameters that the algorithm uses to derive a model
 - Hyperparameters depend on the method (every methods has it's on kind of hyperparameters)

DERIVE MODEL FROM LABELED DATA (TRAIN MODEL/FIT MODEL)

- Apply the method/algorithm on the labeled data to derive the model

DISPLAY MODEL (MODEL SPECIFIC)

- Display the resulting model
 - The resulting model depends on the method used (equation of regression line with intercept and slope, decision tree with nodes and split conditions, groups of observations with centroid, ...)

DATA ANALYTICS PIPELINE

VALIDATE MODEL USING LABELED DATA (SUPERVISED LEARNING)

- Predict target feature for the labeled data
- Calculate the difference between predicted and real values for the labeled data / calculate confusion matrix (classification)
- Calculate validation metrics for the labeled data
 - Regression : MAE, MAPE, RMSE, ...
 - Classification : accuracy, precision, recall, f-score, AUC, ROC

APPLY MODEL ON NEW DATA

- Apply the model on new data
 - In case of supervised machine learning methods, this will predict the target feature for new data
 - In case of unsupervised machine learning methods, this will restructure the feature data (clusters, association rules, new feature matrix with reduced dimensions)

MODEL VALIDATION THE WRONG WAY

DO NOT TRAIN AND VALIDATE THE MODEL ON THE SAME DATA

Training and validating a model on the same data is not a good idea. It is like evaluating students based on an examination with questions that the students got beforehand, including the answers. If a student performs well on that kind of examination, you will never know whether that student really understands the questions and answers, or just learned the answers by heart.

The same problem arises with machine learning; machine learning builds a model by deriving patterns from example data. If you use the same dataset for training and validation, you will never know whether the model really captured the underlying patterns, or just learned the target value by heart for every example (e.g. a decision tree with rules that are too detailed, classifying every single example with a specific rule, instead of a limited set of general rules classifying a large set of similar cases).

OVERFITTING VERSUS GENERALISATION

Mind that the purpose of supervised machine learning is not to predict the labeled training data (you know already the outcome for that data), but to predict new data, data never seen before (and hence not included during training). That means the model should be able to generalise, i.e. be able to capture the overall patterns to be able to predict new data.

So, for validation, we are not that much interested in how well the model can predict the data used for training. We are interested in how well the model can predict new data. A model that scores well on training data but scores bad on new data is an overfitted model. That means it tries too hard to predict the training data, but is not able to grasp the general patterns in the data. It is not able to generalise, so it predicts the training data very well, but is not able to predict new data. Such a model is useless in practice.

What we need is a model that score well on new data.

MODEL VALIDATION THE RIGHT WAY

DO NOT USE THE LABELED TRAINING DATA FOR VALIDATION

Again, we are not interested in how well the model scores on the labeled training data, but on new data. That means that the labeled training data **MUST NOT** be used for validation. Hence, after training, a new labeled dataset needs to be constructed for the validation.

Constructing a new labeled dataset after training is not a practical approach; constructing a labeled dataset might require expert knowledge in the domain at hand. As a data scientist, you cannot have expert knowledge for every potential domain. So labeling a dataset might require external experts. It is not always possible to call those in after every training job.

The solution is to find a way to use the labeled dataset for both training and validation.

USE A HOLDOUT SAMPLE FOR VALIDATION

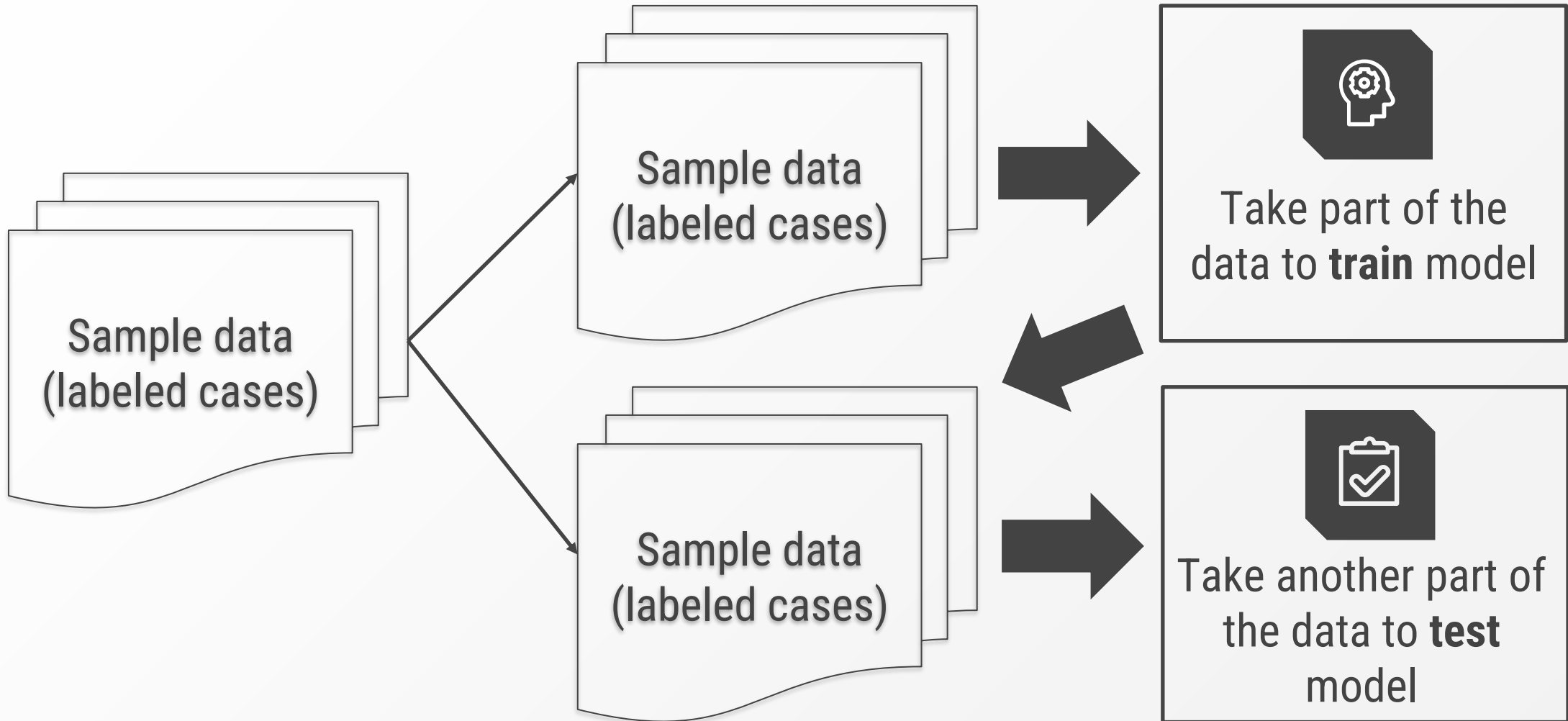
The answer to that problem is to use a holdout sample. That means that some labeled examples are set aside before training and are not used during training. After training, the holdout sample, i.e. the labeled examples that were set aside and not used in training, can now be used for an independent validation as they are new to the model.

SPLIT LABELED DATA INTO TRAINING SET AND TEST SET

In practice, it is as simple as to split the labeled data set in two subsets, one subset to be used for training, and one subset to be used for testing (validation).

The split does not need to be 50/50. The more training examples the better, but of course there need to be enough test examples to get a representative sample for testing. Common ratios are 70%/80% for training and 30%/20% for testing.

TRAIN - TEST SPLIT



TRAIN – TEST SPLIT WITH SCIKIT-LEARN

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8)
```

Parameters

X, y : feature matrix and target vector (more general: can be any number of NumPy or Pandas DataFrames, but they all need to have the same length)

test_size :

- Default: complement of train_size, if train_size is also None: 0.25
- int: number of test samples
- float: proportion of test samples (should be between 0.0 and 1.0)

train_size :

- Default: complement of test_size
- int: number of training samples
- float: proportion of training samples (should be between 0.0 and 1.0)

shuffle (default=True): whether or not to shuffle the data before splitting

stratify (default=None): for classification, if not None, the distribution of the classes in training and test sets are similar as in this array (usually = y)

Returns

X_train, X_test, y_train, y_test: splitted Numpy arrays of Pandas DataFrames (more general: 2 times the number of arrays given as input)

MODEL VALIDATION BASICS (UNSUPERVISED LEARNING)

METHOD SPECIFIC

For unsupervised learning (clustering, dimensionality reduction, association rules), model validation is not easy. There are no labels to compare with, there is no ground truth or golden standard.

The way to validate this is very method specific.

Some methods allow to derive some metrics that can be used in validation:

- Clustering : Inter cluster distance (should be high), intra cluster distance (should be low), silhouette (should be high)
- Association rules : lift, confidence

MIND VALIDATION METRICS AS THESE ARE USED TO DERIVE THE MODEL

Do not blindly rely on validation metrics of unsupervised methods, as these metrics are used under the hood to derive the model:

- Clustering : inter cluster distance and intra cluster distance is used to form the clusters
- Association rules : lift and confidence are used to select relevant rules

Using these metrics for validation of unsupervised models is a bit like using training data to validate supervised models. If unsupervised models score low on those metrics, you can be sure the model is bad, but the opposite is not true, if unsupervised models score high on those metrics, you still can not know for sure the model is good.

MODEL VALIDATION BASICS (UNSUPERVISED LEARNING)

THE PROOF OF THE PUDDING IS IN THE EATING

The only way to check whether an unsupervised model is good is to implement it and check if it does the job that has to be done, or solve the problem it has to solve. That kind of validation can be very hard and time consuming.

DATA SCIENCE 2 – DATA & A.I. 3

V MACHINE LEARNING

3 HYPERPARAMETERS AND MODEL VALIDATION

PYTHON BASICS

Python for data science



WORKING WITH ARRAYS

Numpy



DATA ENGINEERING

pandas



DATA SCIENCE 2 DATA & A.I. 3



DATA VISUALISATION

Matplotlib



MACHINE LEARNING

Automatically find patterns

V



MACHINE LEARNING

scikit-learn

WHAT IS MACHINE LEARNING

Automatically find patterns

01

INTRODUCING SCIKIT-LEARN

Machine learning with Python

02

HYPERPARAMETERS AND CROSS VALIDATION

Holdout samples
and cross-validation

03

REGRESSION

Best fitting line

04

MACHINE LEARNING

05

DECISION TREES

Best separating lines

06

K-MEANS CLUSTERING

Object grouping

07

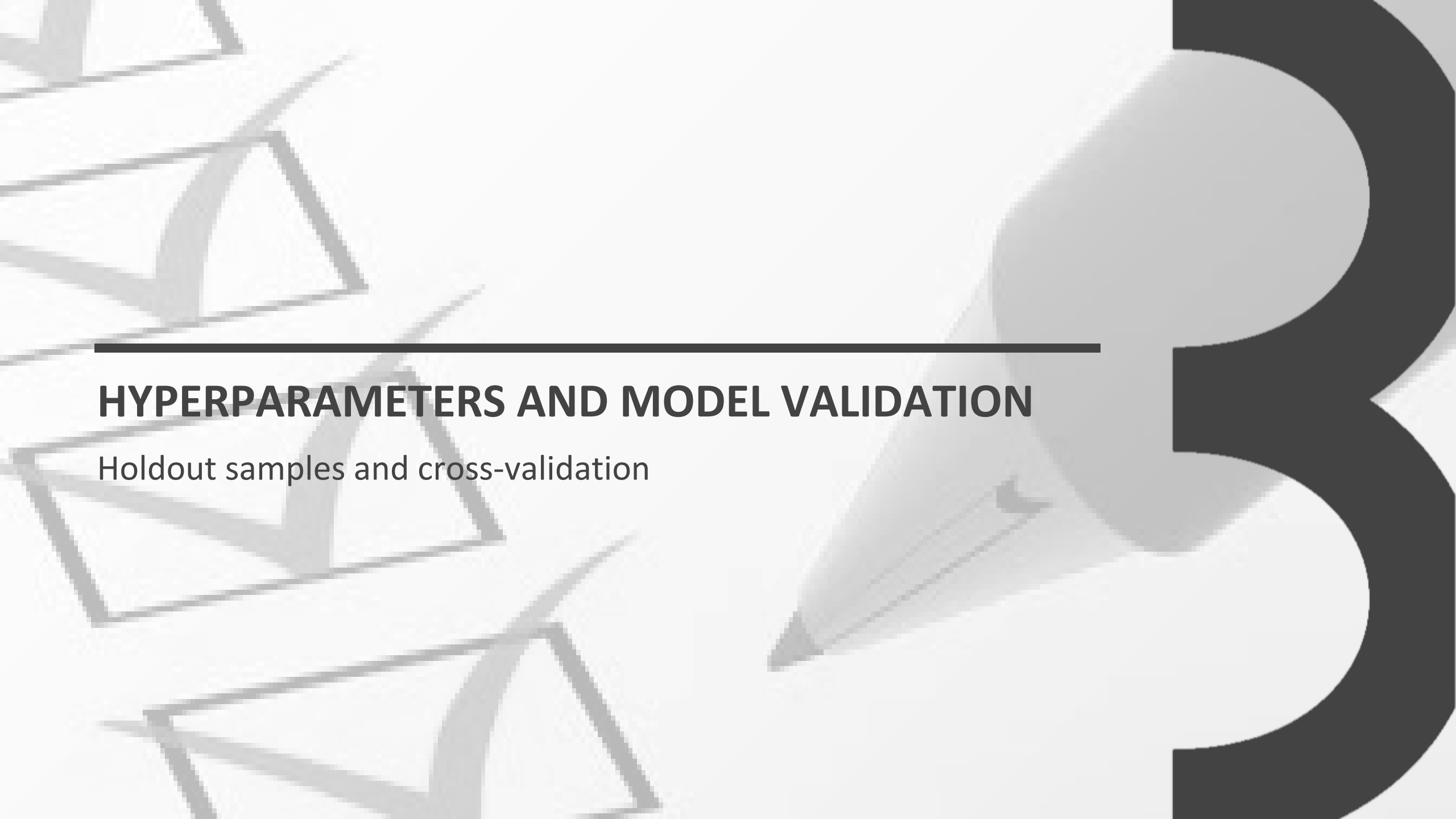
ASSOCIATION RULES

Frequent itemsets

08

ARTIFICIAL NEURAL NETWORK

Imitate the human brain



HYPERPARAMETERS AND MODEL VALIDATION

Holdout samples and cross-validation

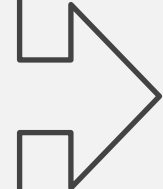
WHAT KIND OF PROBLEMS CAN YOU SOLVE WITH MACHINE LEARNING

S
U
P
E
R
V
I
S
E
D



- Value estimation / regression: predict a continuous variable
e.g. sales prediction; car use
- Classification: predict a categorical variable
e.g. churn prediction; diagnosis

- Segmentation / clustering: split or group cases/observations
e.g. customer segmentation; document topic search
- Co-occurrence / association rule discovery: events happening together
e.g. market basket analysis ; recommendation system



U
N
S
U
P
E
R
V
I
S
E
D

DATA ANALYTICS PIPELINE

DATA PREPARATION

- Load (labeled) source data
- Compile feature matrix
- Compile target array (for supervised methods)

MODEL SELECTION AND HYPERPARAMETER SELECTION (MODEL SPECIFIC)

- Decide on the method to use (linear regression, decision tree, K-means clustering, ...)
- Decide on the hyperparameter to use (degree of polynomial, tree depth, number of clusters, ...)
 - Hyperparameters are parameters that the algorithm uses to derive a model
 - Hyperparameters depend on the method (every methods has it's on kind of hyperparameters)

DERIVE MODEL FROM LABELED DATA (TRAIN MODEL/FIT MODEL)

- Apply the method/algorithm on the labeled data to derive the model

DISPLAY MODEL (MODEL SPECIFIC)

- Display the resulting model
 - The resulting model depends on the method used (equation of regression line with intercept and slope, decision tree with nodes and split conditions, groups of observations with centroid, ...)

DATA ANALYTICS PIPELINE

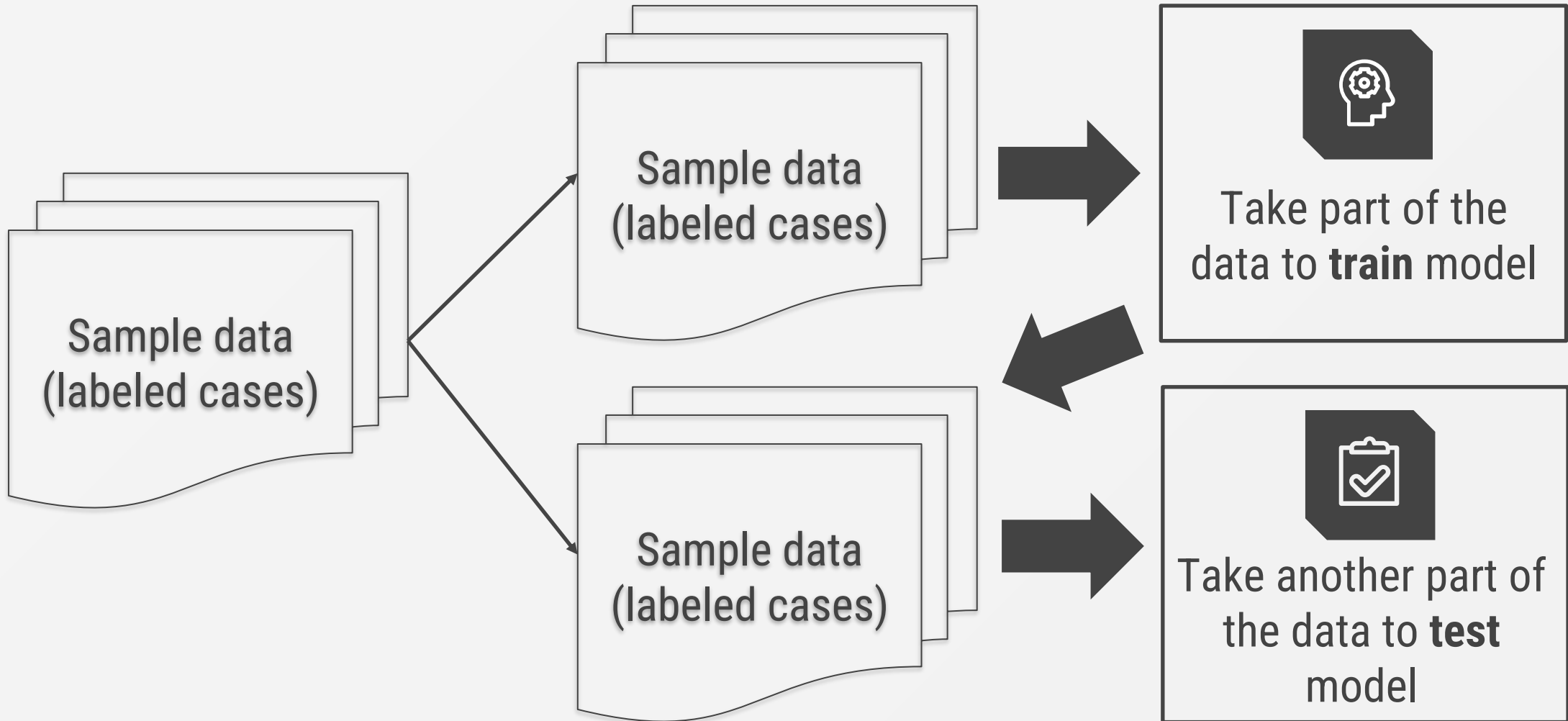
VALIDATE MODEL USING LABELED DATA (SUPERVISED LEARNING)

- Predict target feature for the labeled data
- Calculate the difference between predicted and real values for the labeled data / calculate confusion matrix (classification)
- Calculate validation metrics for the labeled data
 - Regression : MAE, MAPE, RMSE, ...
 - Classification : accuracy, precision, recall, f-score, AUC, ROC

APPLY MODEL ON NEW DATA

- Apply the model on new data
 - In case of supervised machine learning methods, this will predict the target feature for new data
 - In case of unsupervised machine learning methods, this will restructure the feature data (clusters, association rules, new feature matrix with reduced dimensions)

TRAIN - TEST SPLIT



CROSS-VALIDATION

ALTERNATIVE FOR SPLITTING LABELED DATA INTO TRAINING SET AND TEST SET

The problem with [splitting](#) labeled example data into a [training](#) set and [test](#) set is that [less labeled examples are available for training](#) (as they are set aside for validation). In general, the more labeled examples for training, the better the model. But labeling a dataset can be very costly, hence to 'lose' valuable labelled examples to testing might not be preferable, especially for smaller labeled dataset.

The solution might be cross validation.

CROSS-VALIDATION

[Cross-validation](#) validation means that the labeled examples are split into multiple partitions and that [multiple models are trained on different partitions](#). Every time, [one partition of the labelled data is hold out for testing](#), and all other partitions are used for training (and the hold out partition changes every time). So, for N partitions, N different models are trained, with one of the N partitions as hold out sample for testing, and the N-1 other partitions for training. As a result, we have N models, all trained on slightly different data, covering all the examples in the labeled dataset (all examples are used N-1 times for training, and once for testing), resulting in N validation metrics. Those validation metrics can be combined, e.g. by calculating the mean over the N validation metric values.

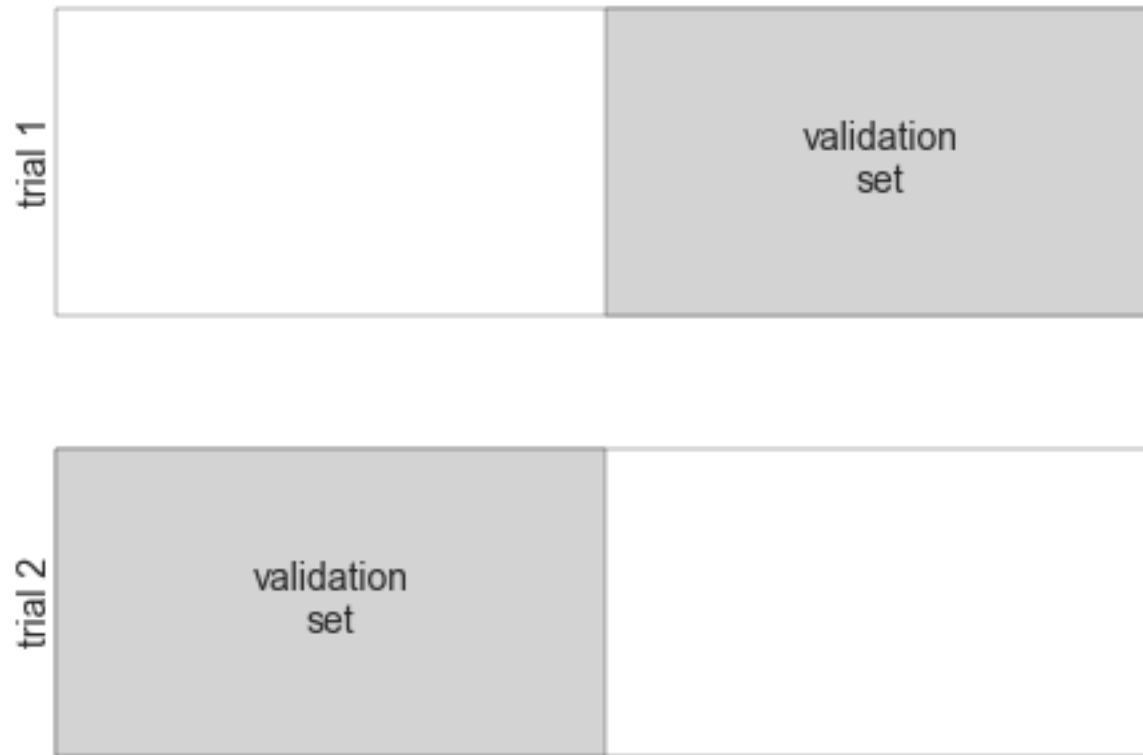
The advantage of this method is that [all data is used both for training and testing](#), and that the combined validation metrics over all parts of the data give a better measure for global performance.

The disadvantage is that we [end up with multiple models](#). That means there is no single model to use for predicting new data. One can [select the best model](#), but that choice might be biased, because the difference in performance is due to difference in training and validation data, so is based on coincidence. There is no guarantee that that model will also perform best on new data (we come back to that later).

Cross-validation is commonly used in model selection and hyperparameter tuning (see later).

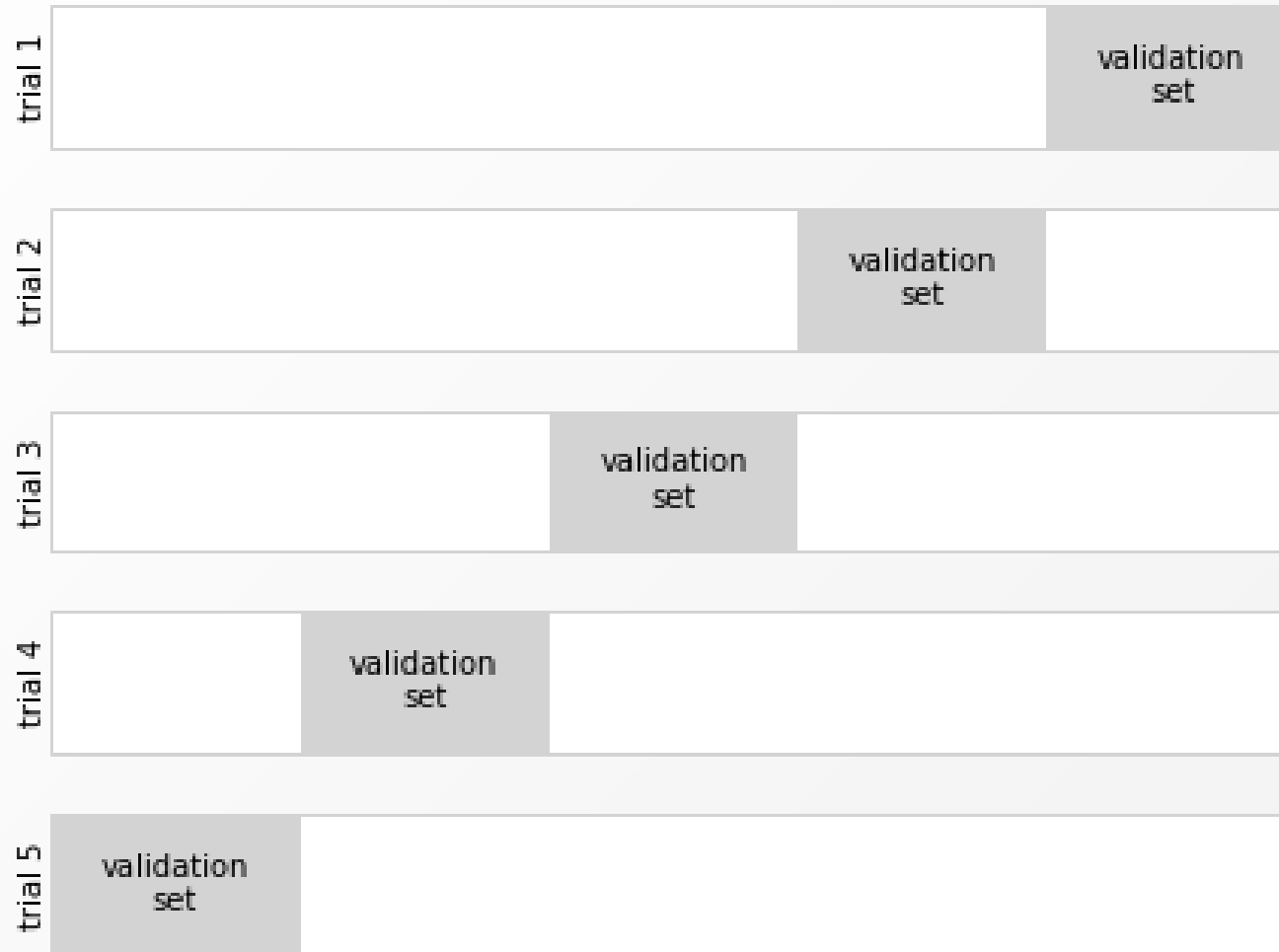
CROSS-VALIDATION

(2-FOLD CROSS VALIDATION)



CROSS-VALIDATION

(K-FOLD CROSS VALIDATION)



CROSS VALIDATION WITH SCIKIT-LEARN

```
from sklearn.model_selection import cross_val_score  
scores = cross_val_score(model, X, y, cv=5)
```

Parameters

model : e.g. [LinearRegression](#), [DecisionTreeClassifier](#), ...

X : feature matrix

y : target vector

cv :

- int: number of cross validation folds (default = 5-fold)
- splitter: e.g. [LeaveOneOut\(\)](#) (from `sklearn.model_selection` import `LeaveOneOut`)

Returns

scores : array with scores (for every cv fold), these are model dependent, e.g. accuracy for decision trees and R2 for linear regression

SIMPLE MODELS VERSUS COMPLEX MODELS

UNDERFITTING VERSUS OVERFITTING

MULTIPLE MODELING TECHNIQUES

When using supervised machine learning, multiple techniques can be used to derive a predictive model

- regression: linear regression, polynomial regression, artificial neural network, ...;
- classification: decision tree, support vector machine, artificial neural network,

The challenge is to select the technique that is most appropriate for your problem or data. That does not necessarily mean the most advanced or complex method. The best choice has to do with underfitting and overfitting.

SIMPLE METHODS

A [simple method](#) is a method that uses a limited set of parameters to create a predictive model. The limited number of parameters reduces the flexibility to capture patterns in data.

The problem with simple models is [UNDERFITTING](#), i.e. the model is not flexible enough to capture the patterns in the data.

A good example of a simple method is linear regression, which simply tries to fit a straight line through the data, and hence only uses an intercept parameter and one slope parameter for every dimension (every independent feature or predictor). That works very well if the data has more or less the shape of a cloud around a linear line. But that will not work if the data does not have a linear shape (e.g. U-shaped data). The only thing linear regression can do is to fit a straight line, nothing more, nothing less.

SIMPLE MODELS VERSUS COMPLEX MODELS

UNDERFITTING VERSUS OVERFITTING

COMPLEX METHODS

A **complex method** is a method that uses an extended set of parameters to create a predictive model. The extended number of parameters increases the flexibility to capture patterns in data.

The problem with complex models is **OVERFITTING**, i.e. the model is too flexible and does not grasp the general pattern but just grasps the characteristics of individual cases.

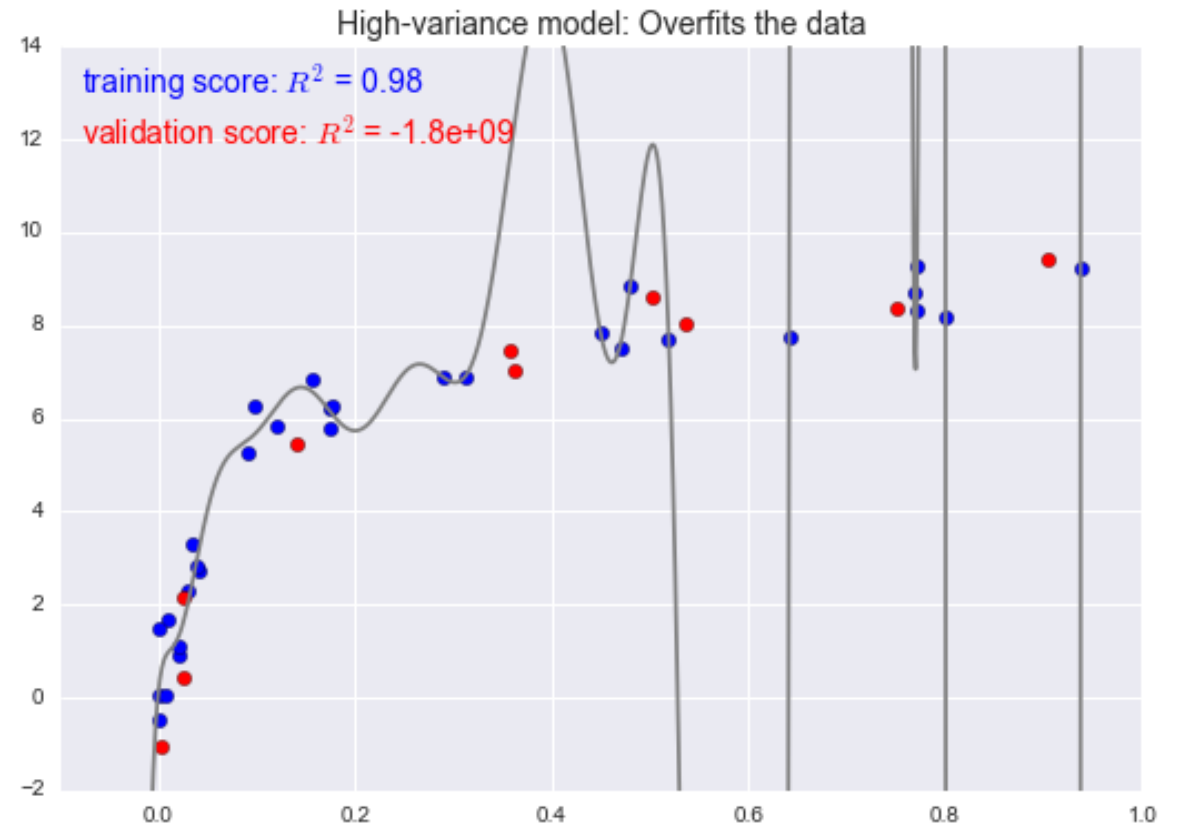
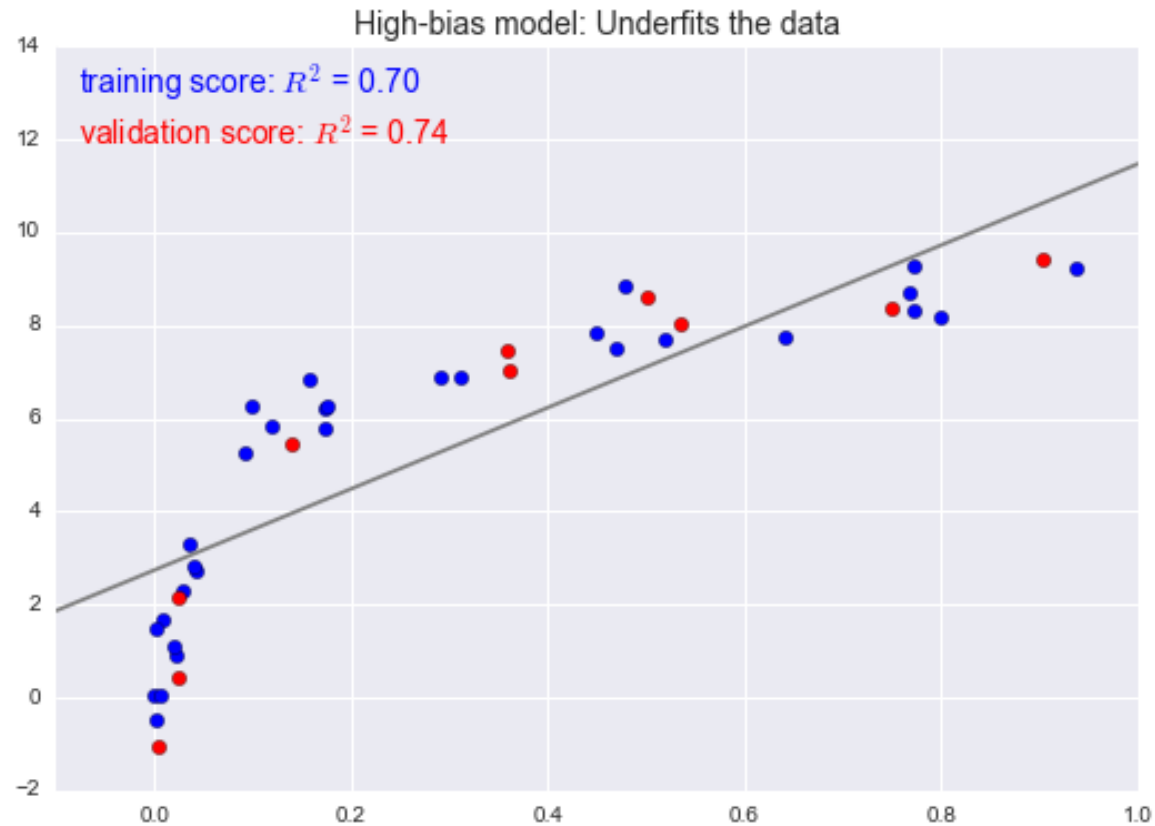
A good example of a complex method is high order polynomial regression, which tries to fit a high order polynomial through the data, and hence requires much more parameters besides intercept and slope ($c_1 x^1, c_2 x^2, c_3 x^3, \dots c_n x^n$). That works very well if the data has more or less the shape of the polynomial, but can go wrong if the data has a different shape, because the polynomial will start predicting individual cases instead of the general pattern.

HOW TO DETECT UNDERFITTING AND OVERFITTING

- **Underfitting** : the model will **score bad on both training set and test set** (the model is not able to capture any relevant pattern)
- **Overfitting** : the model **will score good on the training set but bad on the test set** (the model got tailored to the examples in the training set, but is not able to generalize, not able to predict new data well)

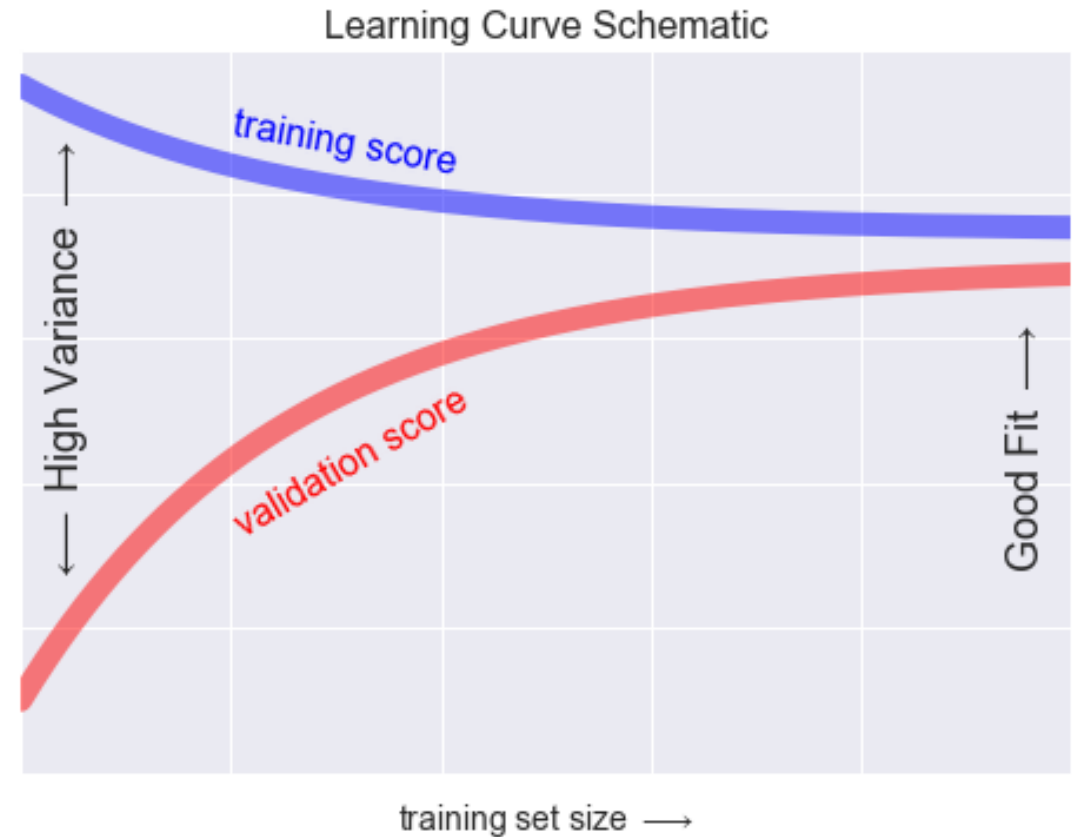
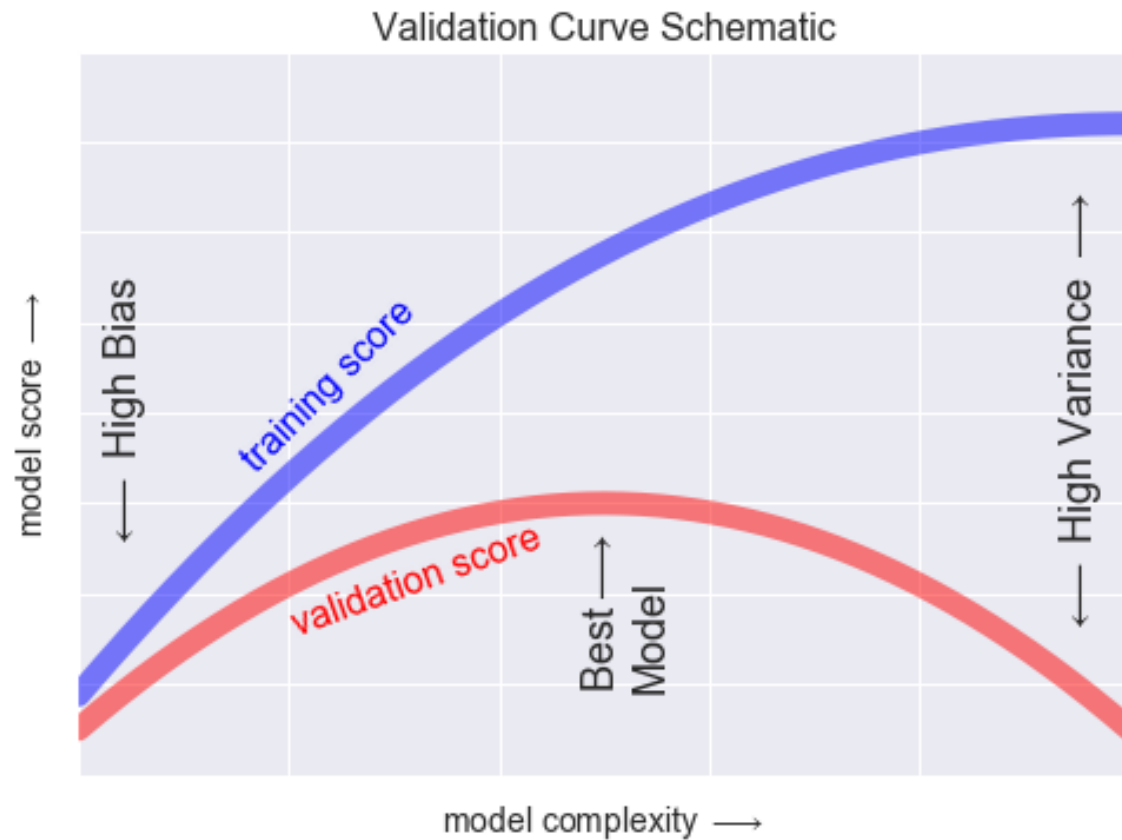
SIMPLE MODELS VERSUS COMPLEX MODELS

UNDERFITTING VERSUS OVERFITTING



SIMPLE MODELS VERSUS COMPLEX MODELS

UNDERFITTING VERSUS OVERFITTING



MODEL SELECTION AND HYPERPARAMETER TUNING

TRAIN – VALIDATE - TEST SPLIT

MODEL SELECTION AND HYPERPARAMETER TUNING

The challenge is to select the modeling technique that is most appropriate for your problem or data. This is called model selection (e.g. decision tree, support vector machine, artificial neural network).

But every method also has hyperparameters that also influence the model and hence can have a big impact on the model quality:

- Decision trees : tree depth, minimum samples split, maximum leaf nodes, . .
- Support vector machine : kernel function, C, gamma, ...
- Artificial neural network : number of layers, number of neurons per layer, activation function, ...

So it is not only about choosing [the right modeling technique](#), but also [the right hyperparameters](#). This is called [hyperparameter tuning](#).

Mostly, it is only possible to find the best modeling technique and set of hyperparameters by trial and error. But doing this right requires a train-validate-test setup (see below), meaning that more labeled examples are lost for training.

TRAIN – VALIDATE - TEST SPLIT

Instead of a train-test split, model selection and hyperparameter tuning requires 3 subsets of labeled examples :

- [Train dataset](#) : dataset used to train multiple models with multiple hyperparameter sets. The result is a set of models with a set of hyperparameters, all trained on the same training set.
- [Validation dataset](#) : dataset to validate all models and hyperparameter sets generated in the previous step. All models and hyperparameter sets are validated using the same validation set. Select the best scoring model and hyperparameter set.
- [Test dataset](#) : dataset for the ultimate, independent test of the one selected best model and hyperparameter set from the previous step to check whether the selected model is general enough to predict new data.

MODEL SELECTION AND HYPERPARAMETER TUNING WITH CROSS-VALIDATION

USING CROSS-VALIDATION FOR MODEL SELECTION AND HYPERPARAMETER TUNING

Cross-validation comes in handy for model selection and hyperparameter tuning because it allows multiple training and testing runs on multiple labeled subsets making use of all information in the labeled dataset and excluding the coincidence factor of data selection.

Using [cross-validation for model selection and hyperparameter tuning](#) only requires the labelled example dataset to be split into [two labeled subsets instead of three](#) labeled subsets (train-validate-test subsets)

PRACTICAL SETUP : TRAIN/VALIDATE – TEST SPLIT

- Split the labeled data into two subsets: [a train/validate subset](#) and [a holdout test subset](#)
- Use the [train/validate subset](#) to [train multiple modeling techniques with multiple hyperparameter sets](#) using cross-validation
 - Every modeling technique/hyperparameter set is trained and validated multiple times, each time validated by one partition and trained using all other partitions. For every modeling technique/hyperparameter set, the average of all validation scores over all runs is calculated
- Select the modeling technique and hyperparameter set with the [highest average score](#) (average of all scores over all runs of the cross-validation) in the previous step
- [Train a new model using the best modeling technique and hyperparameter set](#) as selected in the previous step. [Use all data in the train/validate labeled subset](#) for the training
 - So the train/validate labeled subset is used twice, once to find the best modeling technique/hyperparameter set using cross-validation, once to train a new model based on the best modeling technique/hyperparameter set
- [Test](#) the model trained in the previous step [using the holdout test set](#) for an independent test (test generalization on new data)

HYPERPARAMETER TUNING WITH SCIKIT-LEARN

One parameter

```
from sklearn.model_selection import validation_curve
train_scores, val_scores = validation_curve(model, X=X_train, y=y_train, cv=5,
                                           param_name=name, param_range=range)
```

Parameters

model : e.g. `LinearRegression`, `DecisionTreeClassifier`, ...

X : feature matrix

y : target vector

cv : int: number of cross validation folds (default = 5-fold) or splitter (e.g. `LeaveOneOut()`)

param_name :

- name of model parameter, e.g. `fit_intercept` for `LinearRegression` or `max_depth` for `DecisionTreeClassifier`
- `model.get_params()` can be used to list the parameters

param_range :

- values of the parameters to be validated, e.g. `[True, False]` or `np.arange(1,6)`

Returns

train_scores, val_scores : 2D-arrays with scores (e.g. accuracy for classification or R2 for regression), shape is (nbr. in param range, nbr. of folds)

HYPERPARAMETER TUNING WITH GRID SEARCH

Multiple parameters

GRID SEARCH

Grid search is a way to test a method for [multiple hyperparameter combinations](#). First a set of hyperparameters to be checked is compiled. Next, for every selected hyperparameter, a set of parameter values to be checked is compiled. The combination of all parameter values of interest forms a grid (e.g. a set of 2 hyperparameters, one with 4 parameter values of interest, one with 3 parameter values of interest, form a grid of 4 by 3, resulting in 12 hyperparameter value combinations). Next every cell of the grid is searched, i.e. a model with the combination of hyperparameter values present in the grid cell is trained and validated. In the end, the best scoring model can be used to select the best combination of hyperparameter values.

FULL GRID SEARCH AND RANDOM GRID SEARCH

The total [number of hyperparameter](#) value combination can become [very high](#) (e.g. 5 hyperparameters with an average of 4 hyperparameter values results in 1024 combinations; for neural networks, far more combinations are possible as the number of layers and the amount of neurons per layer can be high).

A [full grid search](#) with train and test a model for every cell in the grid, i.e. every combination of hyperparameter values.

For large datasets, and complex methods, training can become very long, making it practically impossible to do a full grid search. In that case, a [random grid search](#) can be used, where grid cells are randomly chosen and only those combination are trained and tested.

More advanced optimisation methods exist apart from pure random selection.

HYPERPARAMETER TUNING WITH GRID SEARCH

Multiple parameter

```
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(model, cv=5, param_grid=param)
```

Parameters

model : e.g. `LinearRegression`, `DecisionTreeClassifier`, ...

cv : int: number of cross validation folds (default = 5-fold) or splitter (e.g. `LeaveOneOut()`)

param_grid :

- dictionary with parameters names and parameter values for the full grid search, e.g. `{'criterion': ['gini', 'entropy'], 'max_depth': np.arange(1,6)}`

Methods

```
grid.fit(X_train, y_train)
```

Attributes

```
grid.best_params_
grid.best_score_
model = grid.best_estimator_
```

CROSS-VALIDATION AND GRID SEARCH PIPELINE WITH SCIKIT-LEARN (CLASSIFICATION)

MODEL SELECTION AND HYPERPARAMETER TUNING (REPEAT THIS STEP FOR MULTIPLE MODELING TECHNIQUES)

```
from sklearn.model_selection import GridSearchCV
```

```
# Define parameter grid (model specific)
```

```
grid_param = {'criterion' : ['gini', 'entropy', 'log_loss'],  
              'max_depth' : list(range(2,10)),  
              'min_samples_split' : list(range(2,5))}
```

```
# Setup grid search with N-fold cross validation (e.g. 5-fold)
```

```
grid_search = GridSearchCV(model, grid_param, cv=5)
```

```
# Execute full grid search
```

```
grid_search.fit(X_train, y_train)
```

```
# Display best hyperparameter values and matching validation score
```

```
print(f'Best parameters : {grid_search.best_params_}')
```

```
print(f'Best score      : {grid_search.best_score_:.3f}')
```

DATA SCIENCE 2 - DATA & A.I.

3

V MACHINE LEARNING

4-7 ML TECHNIQUES

PYTHON BASICS

Python for data science



WORKING WITH ARRAYS

Numpy



DATA ENGINEERING

pandas



DATA SCIENCE 2 DATA & A.I. 3



DATA VISUALISATION

Matplotlib



MACHINE LEARNING

Automatically find patterns

V



MACHINE LEARNING

scikit-learn

WHAT IS MACHINE LEARNING

Automatically find patterns

01

INTRODUCING SCIKIT-LEARN

Machine learning with Python

02

HYPERPARAMETERS AND CROSS VALIDATION

Holdout samples
and cross-validation

03

REGRESSION

Best fitting line

04

MACHINE LEARNING

05

DECISION TREES

Best separating lines

06

K-MEANS CLUSTERING

Object grouping

07

ASSOCIATION RULES

Frequent itemsets

08

ARTIFICIAL NEURAL NETWORK

Imitate the human brain

SUPERVISED LEARNING: REGRESSION

Linear and Polynomial Regression



REGRESSION (SUPERVISED LEARNING)

What?

prediction of numerical outcomes

How?

Fit a mathematical function to the data to model the relationship between features and the target.

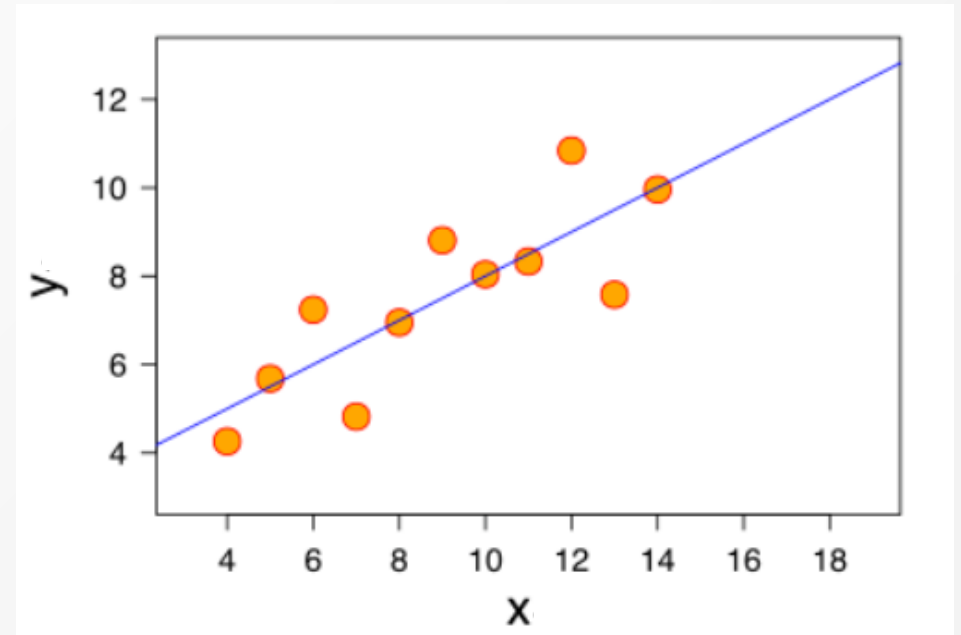
Linear Regression: Fits a [linear function](#) to predict a numerical outcome.

Polynomial Regression: Fits a [polynomial function](#) (with varying degrees) to capture non-linear relationships in the data.

LINEAR REGRESSION

Prediction of numerical features

- One predictor (independent feature):
Try to fit a line to the data in the best possible way: $y = ax + b$
a: slope
b: intercept
- Multiple predictors: $y = b + a_1x_1 + a_2x_2 + \dots + a_nx_n$
n: number of independent features
 $a_1 \dots a_n$: slopes
b: intercept



LINEAR REGRESSION WITH SCIKIT-LEARN

```
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)
```

Hyperparameters:

fit_intercept

- **Type:** boolean, default = True
- **Description:** Determines whether or not the model should calculate the intercept (also called the bias term). If True, the model calculates the intercept. If False, the model assumes the data is already centered around the origin (i.e., the intercept is 0).
- **Usage:** fit_intercept=False may be useful if you know the data is already centered or you want a model without an intercept.

copy_X

- **Type:** boolean, default = True
- **Description:** If True, X will be copied before fitting the model. If False, changes to the original data (such as scaling) will affect the original dataset. It's usually safer to leave this as True.

n_jobs

- **Type:** int, default = None (means 1)
- **Description:** This specifies the number of CPU cores to use when fitting the model. If n_jobs=-1, all available cores will be used. This can help speed up the model fitting when working with large datasets.

LINEAR REGRESSION WITH SCIKIT-LEARN

One predictor

DATA PREPARATION

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
iris = sns.load_dataset('iris')
```

```
# Predictor: Pandas DataFrame, not Series!
```

```
X = iris[['petal_width']]
```

```
# Target feature to predict: Pandas Series
```

```
y = iris['petal_length']
```

SPLIT in TRAIN and TEST dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8)
```

LINEAR REGRESSION WITH SCIKIT-LEARN

```
# MODEL SELECTION AND HYPERPARAMETER SELECTION (MODEL SPECIFIC)
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
# List all selected hyperparameters
```

```
print(model.get_params(deep=True))
```

```
# DERIVE MODEL FROM LABELED DATA (TRAIN MODEL/FIT MODEL)
```

```
model.fit(X_train, y_train)
```

```
# DISPLAY COEFFICIENTS (intercept and slope)
```

```
print(f"Intercept: {model.intercept_}")
```

```
print(f"Coefficient: {model.coef_[0]}") # only 1 slope here
```

LINEAR REGRESSION WITH SCIKIT-LEARN

VALIDATE MODEL USING LABELED TEST DATA

```
# Score model, used metric is model dependent, for regression: R^2
```

```
r2 = model.score(X_test, y_test)
```

```
print(f'R^2 : {r2:.3f}')
```

```
# Other metrics
```

```
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error,  
root_mean_squared_error, r2_score
```

```
# Predict target feature for the labeled test data
```

```
y_test_pred = model.predict(X_test)
```

```
mae = mean_absolute_error(y_true=y_test, y_pred=y_test_pred)
```

```
mape = mean_absolute_percentage_error(y_true=y_test, y_pred=y_test_pred)
```

```
rmse = root_mean_squared_error(y_test, y_test_pred)
```

```
r2 = r2_score(y_test, y_test_pred)
```

```
print(f'MAE : {mae:.3f} - MAPE : {mape:.3f} - RMSE : {rmse:.3f} - R^2 : {r2:.3f}')
```

LINEAR REGRESSION WITH SCIKIT-LEARN

```
# APPLY MODEL ON NEW DATA
```

```
X_pred = ... (new feature data to predict the target feature for)
```

```
y_pred = model.predict(X_pred)
```

LINEAR REGRESSION WITH SCIKIT-LEARN

```
# SHOW REGRESSION LINE IN SCATTERPLOT
```

```
plt.figure(figsize=(8,6))
```

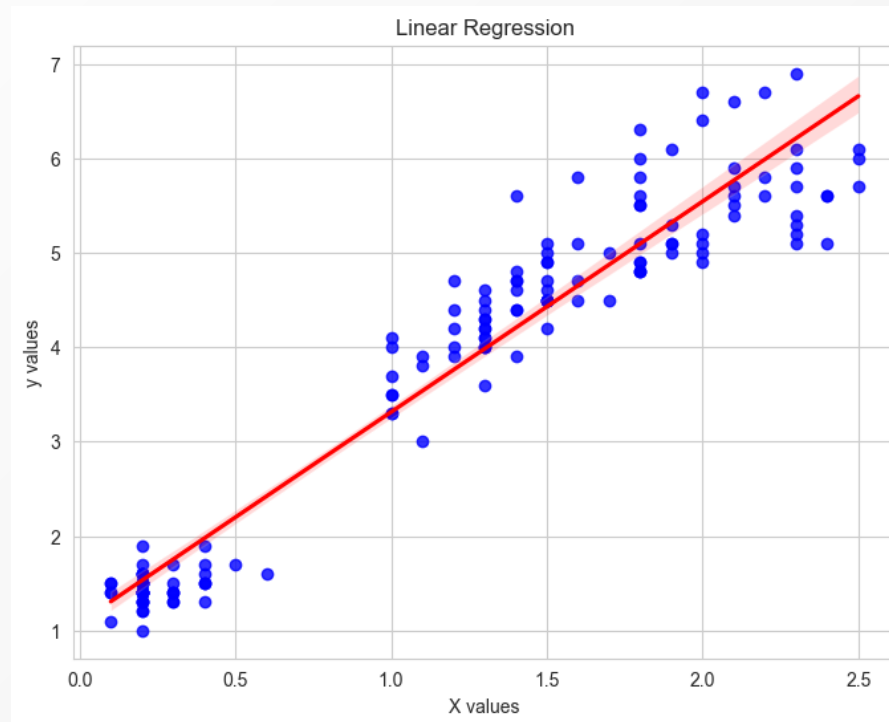
```
sns.regplot(x=X_train, y=y_train, scatter_kws={"color":"blue"}, line_kws={"color":"red"})
```

```
plt.title("Linear Regression")
```

```
plt.xlabel("X values")
```

```
plt.ylabel("y values")
```

```
plt.show()
```



LINEAR REGRESSION WITH SCIKIT-LEARN

Multiple predictors

DATA PREPARATION

```
X = iris[['sepal_width', 'sepal_length', 'petal_width']] # Multiple predictors
y = iris['petal_length'] # Target feature to predict
```

MODEL SELECTION AND HYPERPARAMETER SELECTION (MODEL SPECIFIC)

```
model = LinearRegression()
```

SPLIT in TRAIN and TEST dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8)
```

DERIVE MODEL FROM LABELED DATA (TRAIN MODEL/FIT MODEL)

```
model.fit(X_train, y_train)
```

DISPLAY COEFFICIENTS (intercept and slope)

```
print(f"Intercept: {model.intercept_}")
```

```
print(f"Coefficient: {model.coef_}")
```

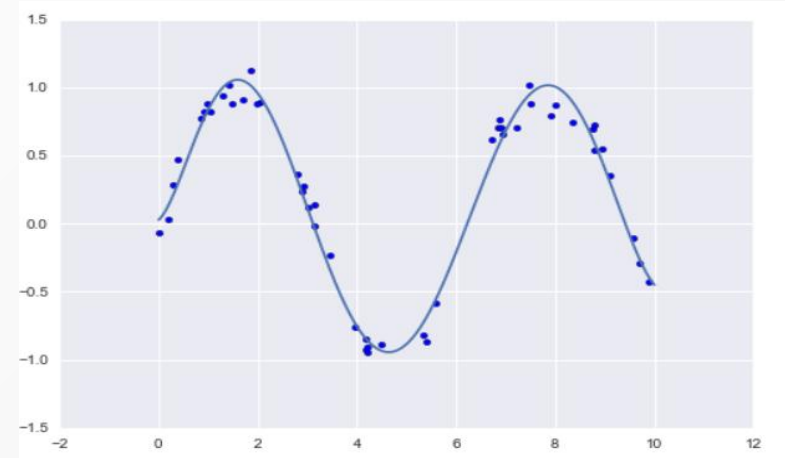

POLYNOMIAL REGRESSION

prediction of numerical features

Try to fit a polynomial function to the data in the best possible way

$$y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

n: degree (order) of the polynomial function



Quadratic:

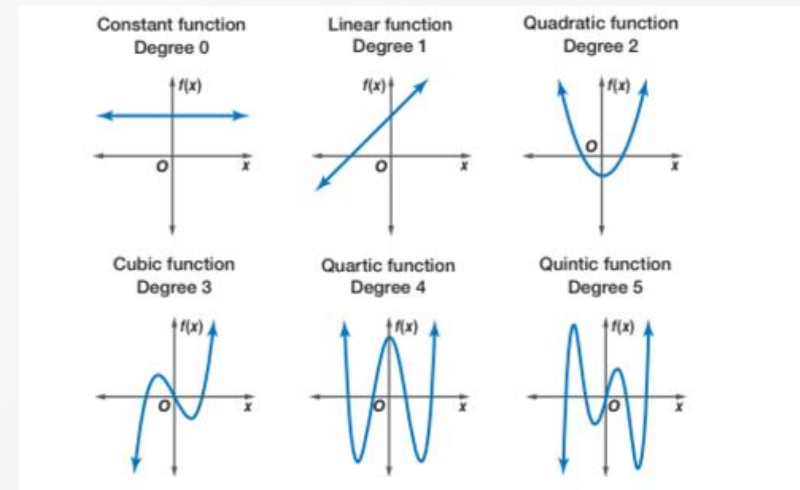
$$n = 2$$

$$y = a_0 + a_1x + a_2x^2$$

Cubic:

$$n = 3$$

$$y = a_0 + a_1x + a_2x^2 + a_3x^3$$



POLYNOMIAL REGRESSION WITH SCIKIT-LEARN

```
# MODEL SELECTION AND HYPERPARAMETER SELECTION (MODEL SPECIFIC)
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.pipeline import make_pipeline
```

```
model = make_pipeline(PolynomialFeatures(degree=2, include_bias=False), LinearRegression())
```

```
# DERIVE MODEL FROM LABELED DATA (TRAIN MODEL/FIT MODEL)
```

```
model.fit(X_train, y_train)
```

```
# VALIDATE MODEL USING LABELED TEST DATA
```

```
r2 = model.score(X_test, y_test) # same for other scores
```

```
# APPLY MODEL ON NEW DATA
```

```
X_pred = ... (new feature data to predict the target feature for)
```

```
y_pred = model.predict(X_pred)
```

DATA SCIENCE 2 - DATA & A.I.

3

V MACHINE LEARNING

4-7 ML TECHNIQUES

PYTHON BASICS

Python for data science



WORKING WITH ARRAYS

Numpy



DATA ENGINEERING

pandas



DATA SCIENCE 2 DATA & A.I. 3



DATA VISUALISATION

Matplotlib



MACHINE LEARNING

Automatically find patterns

V



MACHINE LEARNING

scikit-learn

WHAT IS MACHINE LEARNING

Automatically find patterns

01

INTRODUCING SCIKIT-LEARN

Machine learning with Python

02

HYPERPARAMETERS AND CROSS VALIDATION

Holdout samples
and cross-validation

03

REGRESSION

Best fitting line

04

MACHINE LEARNING

05

DECISION TREES

Best separating lines

06

K-MEANS CLUSTERING

Object grouping

07

ASSOCIATION RULES

Frequent itemsets

08

ARTIFICIAL NEURAL NETWORK

Imitate the human brain

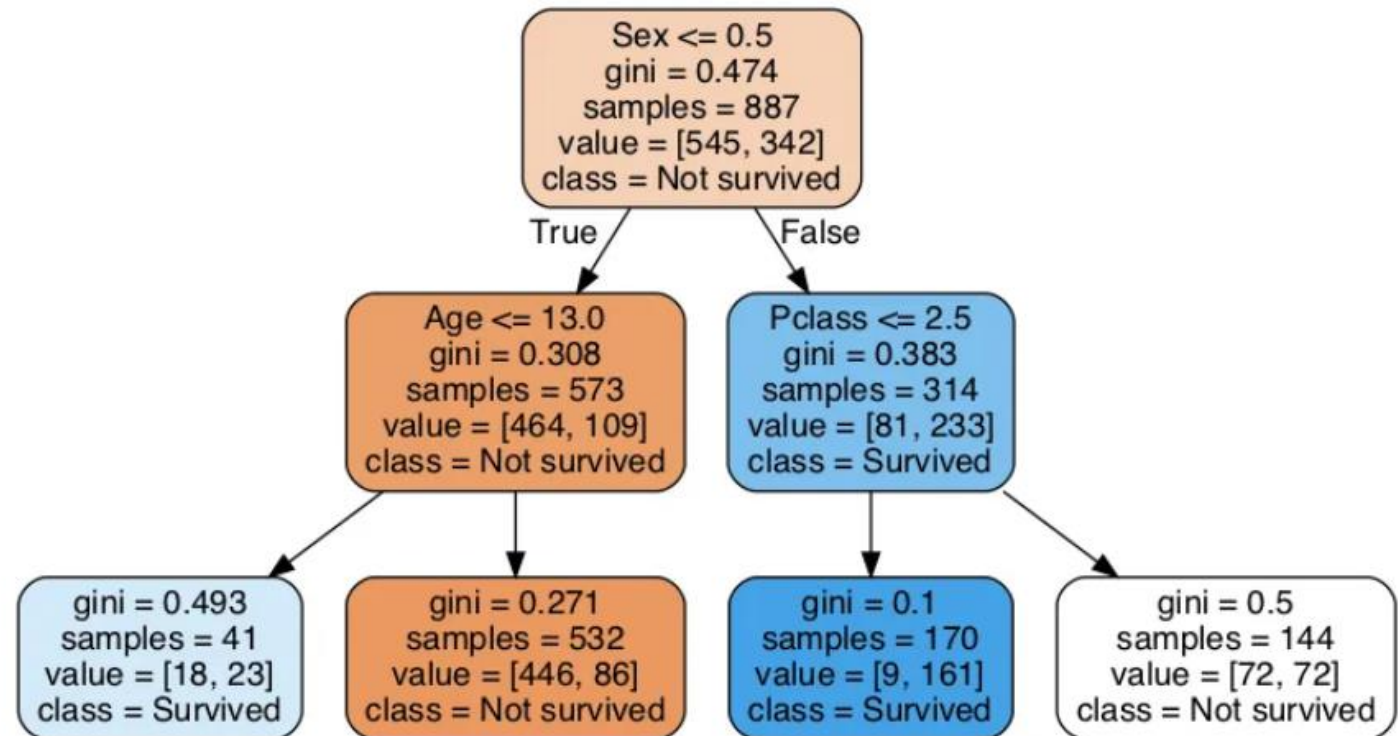
SUPERVISED LEARNING: DECISION TREES

Decision Trees and Random Forests

DECISION TREES

Decision trees:

A supervised learning algorithm used for **classification** and **regression** tasks creating a tree-like model of decisions based on input features



DECISION TREES WITH SCIKIT-LEARN

```
from sklearn.tree import DecisionTreeClassifier  
model = DecisionTreeClassifier(criterion='gini')
```

Hyperparameters:

criterion

- **Type:** string, default='gini'
- **Description:** This function measures the quality of a split. Supported criteria are 'gini' for the Gini impurity and 'entropy' for the information gain.
- **Usage:** Choose 'gini' for the Gini impurity (default) or 'entropy' for information gain when building the decision tree.

max_depth

- **Type:** int, default=None
- **Description:** The maximum depth of the tree. If None, nodes are expanded until all leaves are pure or contain fewer samples than the `min_samples_split`.
- **Usage:** Set this to limit the depth of the tree and avoid overfitting.

min_samples_split

- **Type:** int or float, default=2
- **Description:** The minimum number of samples required to split an internal node. If an integer, it's the minimum number. If a float, it's the fraction of the total number of samples.
- **Usage:** Increase this value to reduce overfitting by making the tree more generalized.

DECISION TREES WITH SCIKIT-LEARN

```
from sklearn.tree import DecisionTreeClassifier  
model = DecisionTreeClassifier(criterion='gini')
```

Hyperparameters:

min_samples_leaf

- **Type:** int or float, default=1
- **Description:** The minimum number of samples required to be at a leaf node. A smaller value allows smaller leaves and may lead to overfitting.
- **Usage:** A higher value can make the model more conservative and less prone to overfitting.

max_features

- **Type:** int, float, string or None, default=None
- **Description:** The number of features to consider when looking for the best split. If None, all features are considered.
- **Usage:** Restricting the number of features can lead to a more generalized model.

DECISION TREES WITH SCIKIT-LEARN

DATA PREPARATION

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
iris = sns.load_dataset('iris')
```

```
# Target feature to predict: Pandas Series
```

```
y = iris['species']
```

```
# Predictors: Pandas DataFrame
```

```
X = iris[['sepal_width', 'sepal_length', 'petal_width', 'petal_width']]
```

SPLIT in TRAIN and TEST dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8, stratify=y)
```

DECISION TREES WITH SCIKIT-LEARN

```
# MODEL SELECTION AND HYPERPARAMETER SELECTION (MODEL SPECIFIC)
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier(max_depth=1)
```

```
# List all selected hyperparameters
```

```
print(model.get_params(deep=True))
```

```
# DERIVE MODEL FROM LABELED DATA (TRAIN MODEL/FIT MODEL)
```

```
model.fit(X_train, y_train)
```

```
# DISPLAY MODEL (MODEL SPECIFIC)
```

```
print(f"Model classes: {model.classes_}")
```

```
from sklearn.tree import plot_tree
```

```
plot_tree(model)
```

DECISION TREES WITH SCIKIT-LEARN

VALIDATE MODEL USING LABELED TEST DATA

```
# Score model, used metric is model dependent, for decision tress: Accuracy
r2 = model.score(X_test, y_test)
print(f'ACC: {r2:.3f}')
```



```
# Other metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
                             classification_report
```



```
# Predict target feature for the labeled test data
y_test_pred = model.predict(X_test)
acc = accuracy_score(y_true=y_test, y_pred=y_test_pred)
prec = precision_score(y_true=y_test, y_pred=y_test_pred, average='weighted')
rec = recall_score(y_true=y_test, y_pred=y_test_pred, average='weighted')
f1 = f1_score(y_true=y_test, y_pred=y_test_pred, average='weighted')
# Multiclass classification -> precision, recall and F1 are calculated by class and averaged.
print(f'ACC : {acc:.3f} - PREC : {prec:.3f} - REC : {rec:.3f} - F1 : {f1:.3f}')
```



```
classification_report(y_true=y_test, y_pred=y_test_pred)
```

DECISION TREES WITH SCIKIT-LEARN

```
# Confusion matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

conf_matrix = confusion_matrix(y_true=y_test, y_pred=y_test_pred, labels=model.classes_)
print(conf_matrix)

# Plot confusion matrix (nicer output in Jupyter)
ConfusionMatrixDisplay.from_predictions(y_true=y_test, y_pred=y_test_pred,
                                       display_labels=model.classes_)
```

DECISION TREES WITH SCIKIT-LEARN

```
# APPLY MODEL ON NEW DATA
```

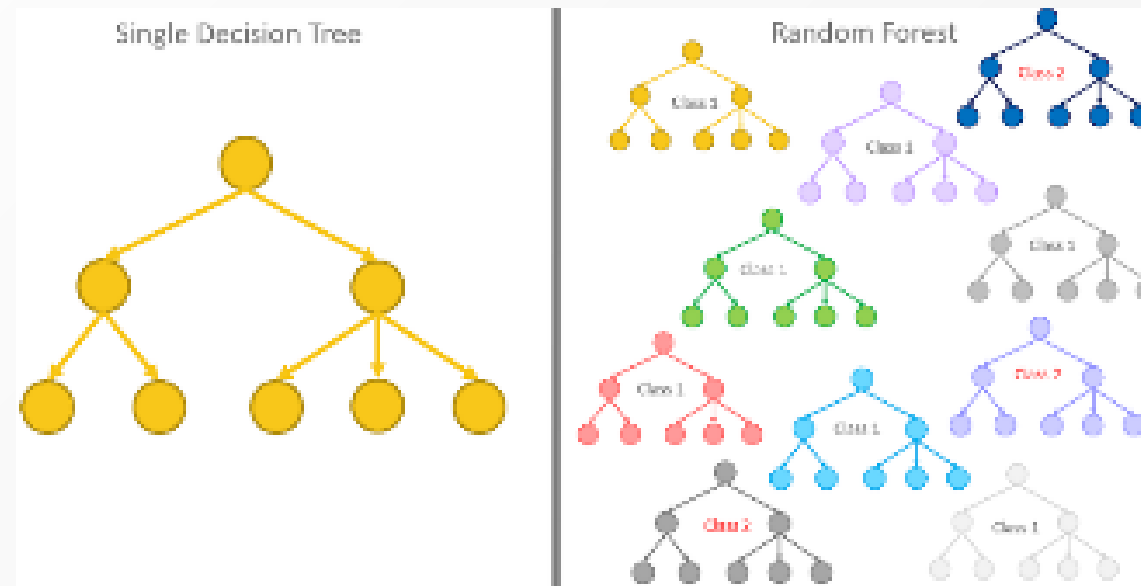
```
X_pred = ... (new feature data to predict the target feature for)
```

```
y_pred = model.predict(X_pred)
```

RANDOM FORESTS

Random forests:

An ensemble method that builds multiple decision trees and aggregates their results for improved accuracy in classification and regression tasks.



RANDOM FORESTS WITH SCIKIT-LEARN

```
from sklearn.ensemble import RandomForestClassifier  
model = RandomForestClassifier(n_estimators=100)
```

Hyperparameters:

n_estimators

- **Type:** int, default=100
- **Description:** The number of trees in the forest. More trees usually lead to better performance but increase computation time.
- **Usage:** Use higher values (e.g., 500, 1000) for larger datasets to improve performance.

max_depth

- **Type:** int, default=None
- **Description:** The maximum depth of each tree in the forest. If None, nodes are expanded until all leaves are pure.
- **Usage:** Restrict this to prevent overfitting, especially when building deep trees.

max_features

- **Type:** int, float, string or None, default='auto'
- **Description:** The number of features to consider when looking for the best split. 'auto' uses the square root of the number of features.
- **Usage:** Try different values such as 'sqrt' (square root) or 'log2' to tune the model for better performance.

min_samples_split

- **Type:** int or float, default=2
- **Description:** The minimum number of samples required to split an internal node in each tree.
- **Usage:** Adjust this to balance between underfitting and overfitting.

RANDOM FORESTS WITH SCIKIT-LEARN

Data preparation

```
iris = sns.load_dataset('iris')
```

```
# Target feature to predict: Pandas Series
```

```
y = iris['species']
```

```
# Predictors: Pandas DataFrame
```

```
X = iris[['sepal_width', 'sepal_length', 'petal_width', 'petal_width']]
```

Split the data into training and test sets

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

Initialize and train the Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
```

```
model.fit(X_train, y_train)
```

RANDOM FORESTS WITH SCIKIT-LEARN

Make predictions

```
y_test_pred = model.predict(X_test)
```

Evaluate the model

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
accuracy = accuracy_score(y_test, y_test_pred)
conf_matrix = confusion_matrix(y_test, y_test_pred)
class_report = classification_report(y_test, y_test_pred)

print(f"Accuracy: {accuracy:.2f}")
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(class_report)
```

DATA SCIENCE 2 - DATA & A.I.

3

V MACHINE LEARNING

4-7 ML TECHNIQUES

PYTHON BASICS

Python for data science



WORKING WITH ARRAYS

Numpy



DATA ENGINEERING

pandas



DATA SCIENCE 2 DATA & A.I. 3



DATA VISUALISATION

Matplotlib



MACHINE LEARNING

Automatically find patterns

V



MACHINE LEARNING

scikit-learn

WHAT IS MACHINE LEARNING

Automatically find patterns

01

INTRODUCING SCIKIT-LEARN

Machine learning with Python

02

HYPERPARAMETERS AND CROSS VALIDATION

Holdout samples
and cross-validation

03

REGRESSION

Best fitting line

04

MACHINE LEARNING

05

DECISION TREES

Best separating lines

06

K-MEANS CLUSTERING

Object grouping

07

ASSOCIATION RULES

Frequent itemsets

08

ARTIFICIAL NEURAL NETWORK

Imitate the human brain

UNSUPERVISED LEARNING: CLUSTERING

K-Means

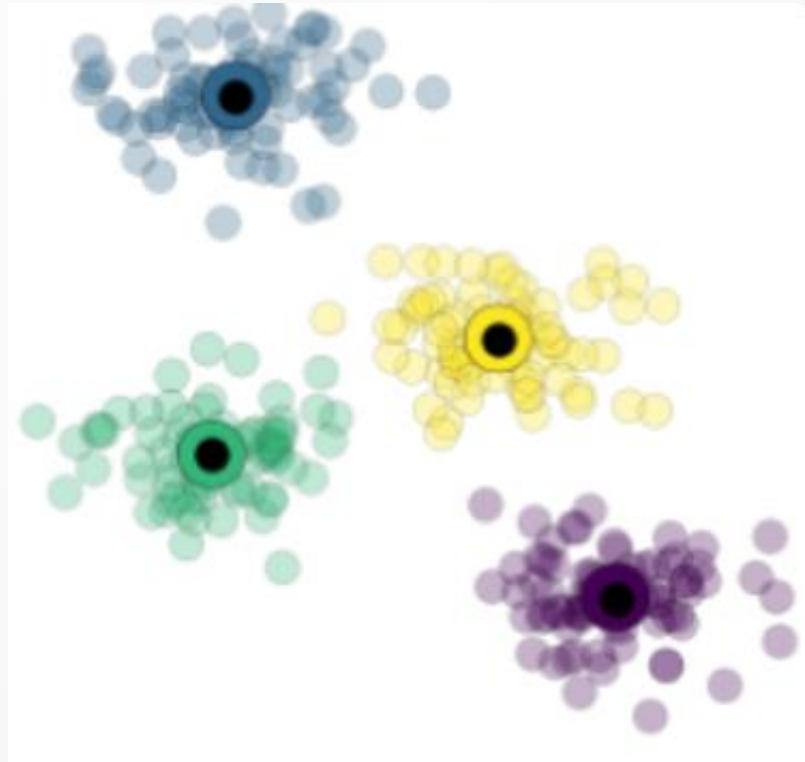


K-MEANS CLUSTERING

K-Means Clustering:

An unsupervised machine learning algorithm used to group similar data points into a predefined number of clusters based on their feature similarities.

See Data & A.I. 2



K-MEANS CLUSTERING WITH SCIKit-LEARN

```
from sklearn.cluster import KMeans  
model = KMeans(n_clusters=4)
```

Hyperparameters:

n_clusters

- **Type:** int, default=8
- **Description:** The number of clusters to form, and the number of centroids to generate.
- **Usage:** Set this to specify how many groups you want the data divided into.

init

- **Type:** {'k-means++', 'random'}, default='k-means++'
- **Description:** Method for initializing the cluster centroids. k-means++ chooses centroids to speed up convergence, while random selects random points.
- **Usage:** k-means++ is typically preferred, but random can be used for experimentation.

max_iter

- **Type:** int, default=300
- **Description:** The maximum number of iterations allowed for the algorithm to run until it converges.
- **Usage:** Increase this if your model is not converging, but usually the default works well.

K-MEANS CLUSTERING WITH SCIKIT-LEARN

Hyperparameters:

tol

- **Type:** float, default=1e-4
- **Description:** The tolerance for the convergence criterion. The algorithm stops when the difference in the cluster centers falls below this threshold.
- **Usage:** Decrease this value to make the algorithm more precise at the cost of computation time.

n_init

- **Type:** int, default=10
- **Description:** The number of times the algorithm will be run with different centroid seeds. The final result will be the best output of these runs.
- **Usage:** Increase this to get a more stable result by running the algorithm multiple times with different initializations.

algorithm

- **Type:** {'auto', 'full', 'elkan'}, default='lkan'
- **Description:** The algorithm to use for clustering. elkan is faster with sparse data, while full uses the standard EM-style algorithm.
- **Usage:** elkan is recommended for efficiency, especially when handling large datasets.

random_state

- **Type:** int, default=None
- **Description:** The seed used by the random number generator. Setting this ensures the same results each time the algorithm is run.
- **Usage:** Useful when you want reproducible results.

K-MEANS CLUSTERING WITH SCIKIT-LEARN

Import necessary libraries

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
import seaborn as sns
import matplotlib.pyplot as plt
```

Create or load some example data

```
# For illustration, we generate a synthetic dataset
from sklearn.datasets import make_blobs
```

```
# Generate synthetic data with 4 clusters
```

```
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=42)
```

Initialize and fit the KMeans model

```
kmeans = KMeans(n_clusters=4, init='k-means++', max_iter=300, random_state=42)
kmeans.fit(X)
```

Get the cluster centers and labels

```
centroids = kmeans.cluster_centers_
labels = kmeans.labels_
```

K-MEANS CLUSTERING WITH SCIKIT-LEARN

```
# Get the cluster centers and labels
```

```
centroids = kmeans.cluster_centers_  
labels = kmeans.labels_
```

```
# Visualize the clusters
```

```
plt.figure(figsize=(8, 6))
```

```
# Plot the points and color by cluster
```

```
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=labels, palette="viridis", s=100, alpha=0.6,  
edgecolor="k")
```

```
# Plot the centroids
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='red', label='Centroids')
```

```
plt.title('KMeans Clustering')
```

```
plt.legend()
```

```
plt.show()
```

```
# Predicting new points
```

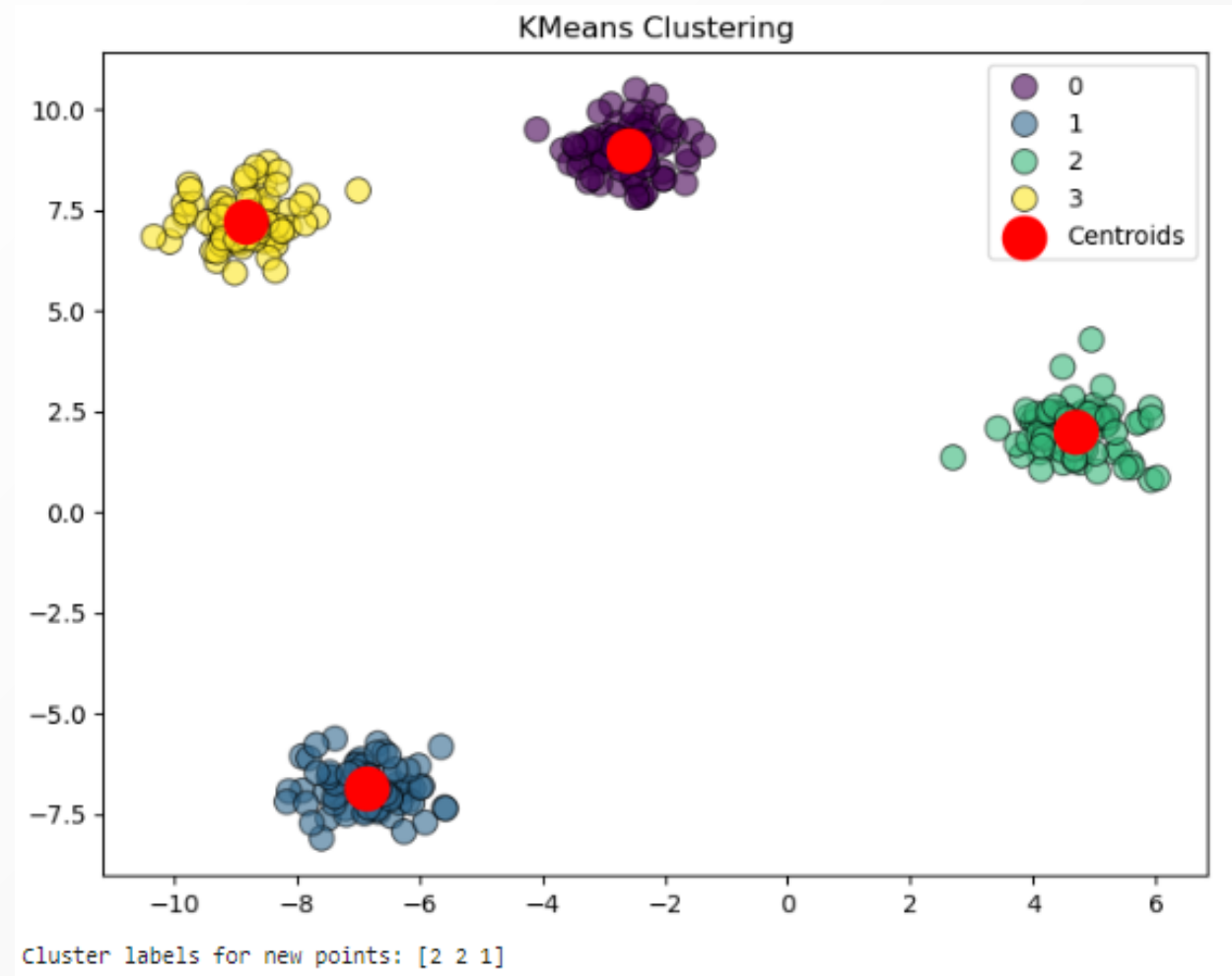
```
new_points = np.array([[0, 0], [5, 5], [-5, -5]])
```

```
predictions = kmeans.predict(new_points)
```

```
print(f"Cluster labels for new points: {predictions}")
```

K-MEANS CLUSTERING WITH SCIKIT-LEARN

EXAMPLE



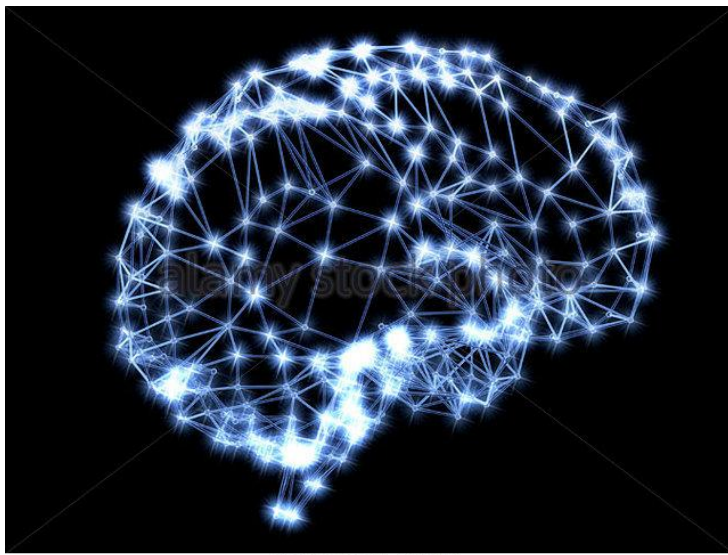
Neural Networks

Data Science 2 / Data & AI 3

Agenda



1. What are Neural Networks?
2. Training a Neural Network
3. Neural Network - Classification
4. Neural Network - Regression

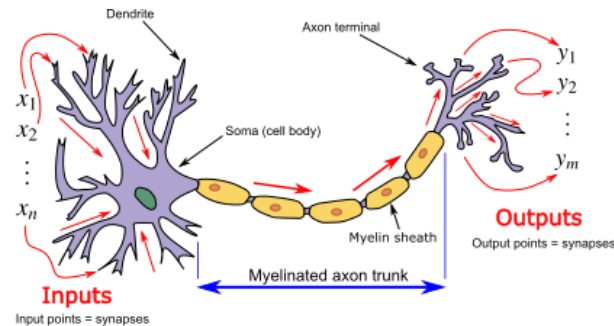


www.alamy.com - C5HK37

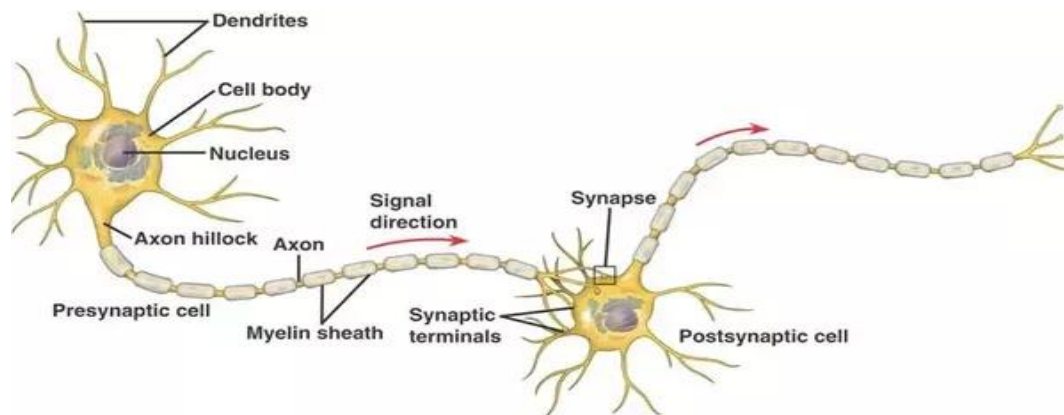
What are Neural Networks?

A biological neural network

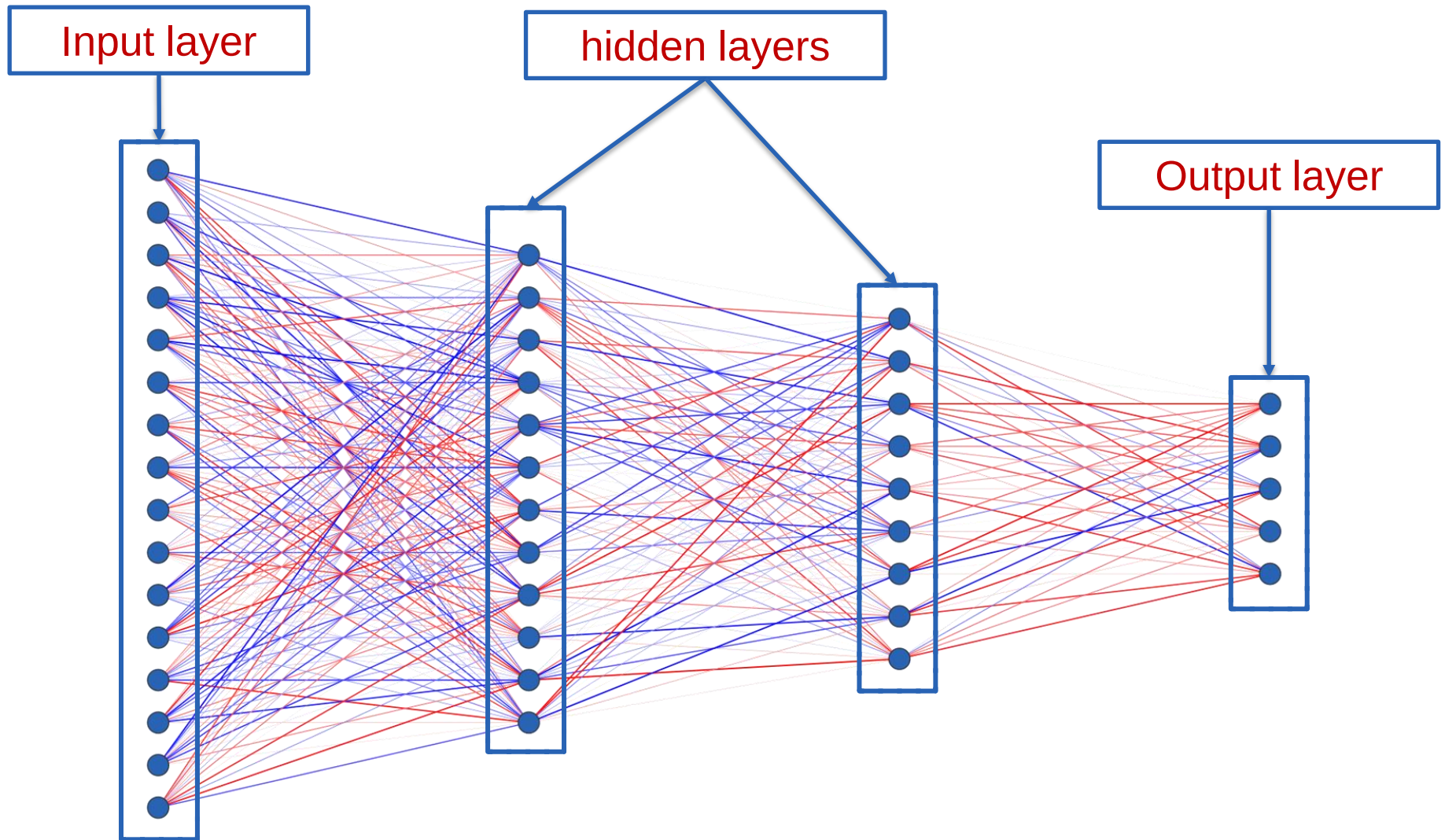
The human brain has about 86,100,000,000 neurons. Input signals are combined and strengthened or weakened (non-linear).



Neurons are connected to each other to form a neural network with 100,000,000,000,000 connections.

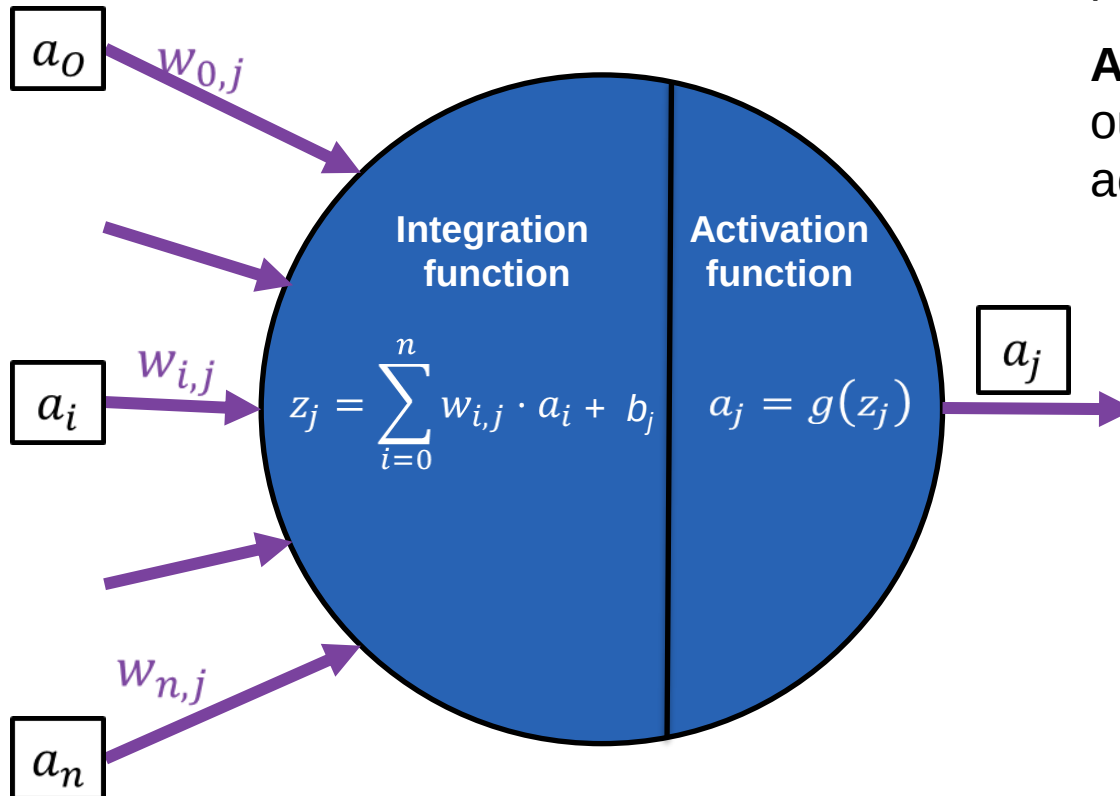


Artificial neural network (ANN or NN)



When the network has more than 1 hidden layer, we talk about **Deep Learning**

Artificial Neuron

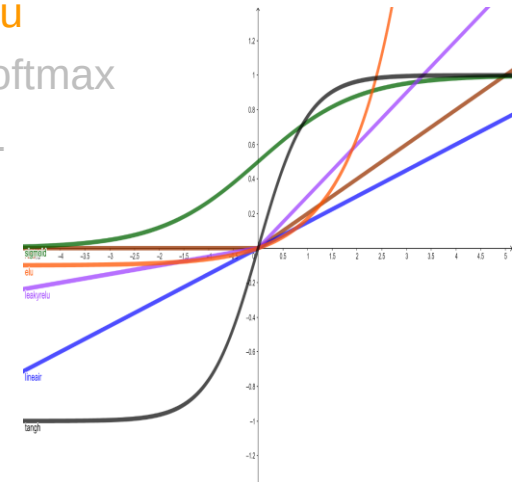


Integration function:

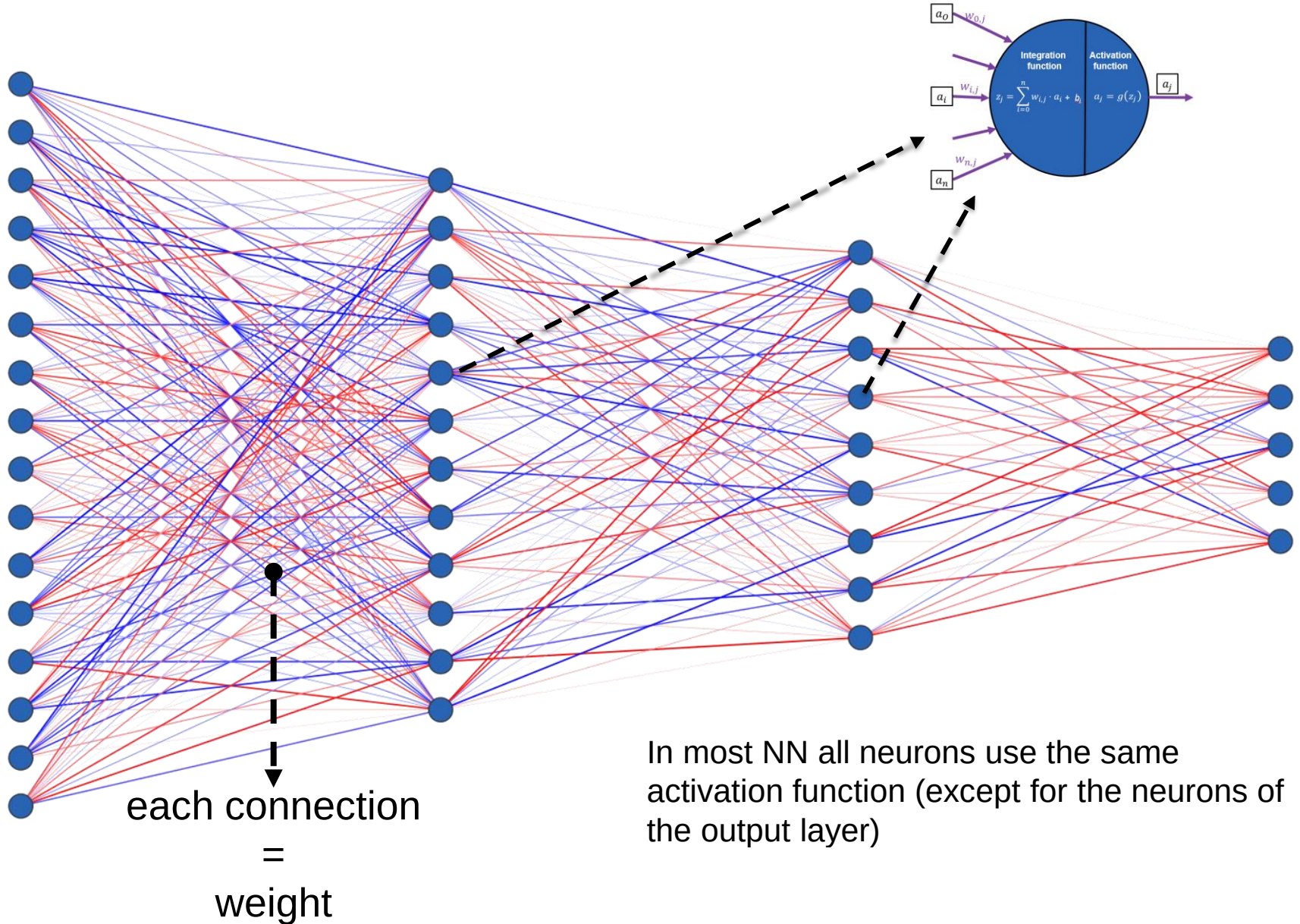
weighted average (**weights** $w_{i,j}$) plus **bias** b_j

Activation function: produces output of the neuron. Several activation functions exist:

- linear
- sigmoid
- relu
- leaky relu
- tanh
- elu
- softmax
- ...

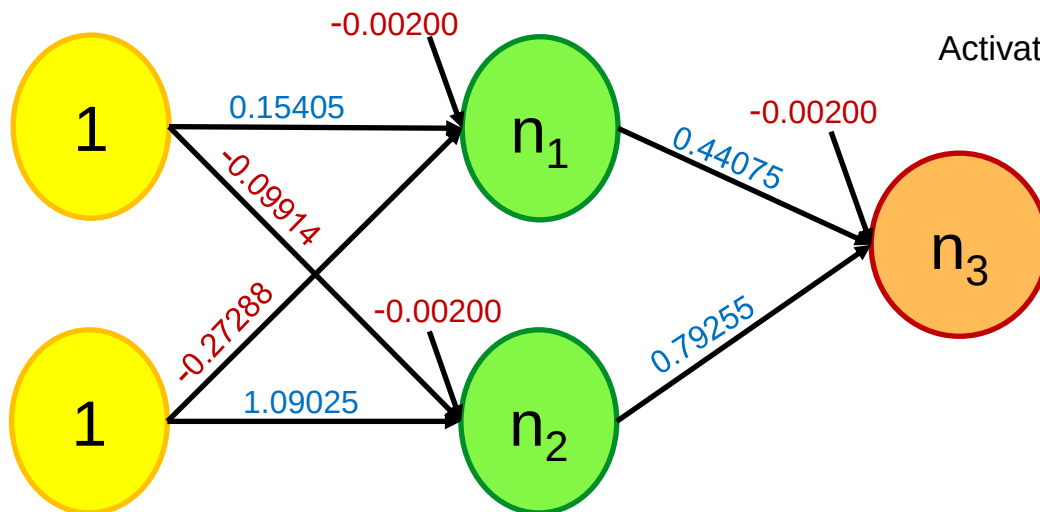


Neural Network



Example

- Input layer: 2 nodes
- One hidden layer: 2 nodes
- Output layer: 1 node
- Activation function: sigmoid
 $g(z_j) = \frac{1}{1+e^{-z_j}}$ for all nodes
- Calculate output with weights and biases as in figure and both inputs equal to 1



n_1 :

$$\text{Integration function: } z_j = 0.15405 \times 1 - 0.27288 \times 1 - 0.00200 = -0.12082$$

$$\text{Activation function: } a_j = \frac{1}{1+e^{-z_j}} = \frac{1}{1+e^{-0.12082}} = 0.46983$$

n_2 :

$$\text{Integration function: } z_j = -0.09914 \times 1 + 1.09025 \times 1 - 0.00200 = 0.98912$$

$$\text{Activation function: } a_j = \frac{1}{1+e^{-z_j}} = \frac{1}{1+e^{-0.98912}} = 0.72891$$

n_3 :

$$\text{Integration function: } z_j = 0.44075 \times 0.46983 + 0.79255 \times 0.72891 - 0.00200 = 0.782781$$

$$\text{Activation function: } a_j = \frac{1}{1+e^{-z_j}} = \frac{1}{1+e^{-0.782781}} = 0.68628$$

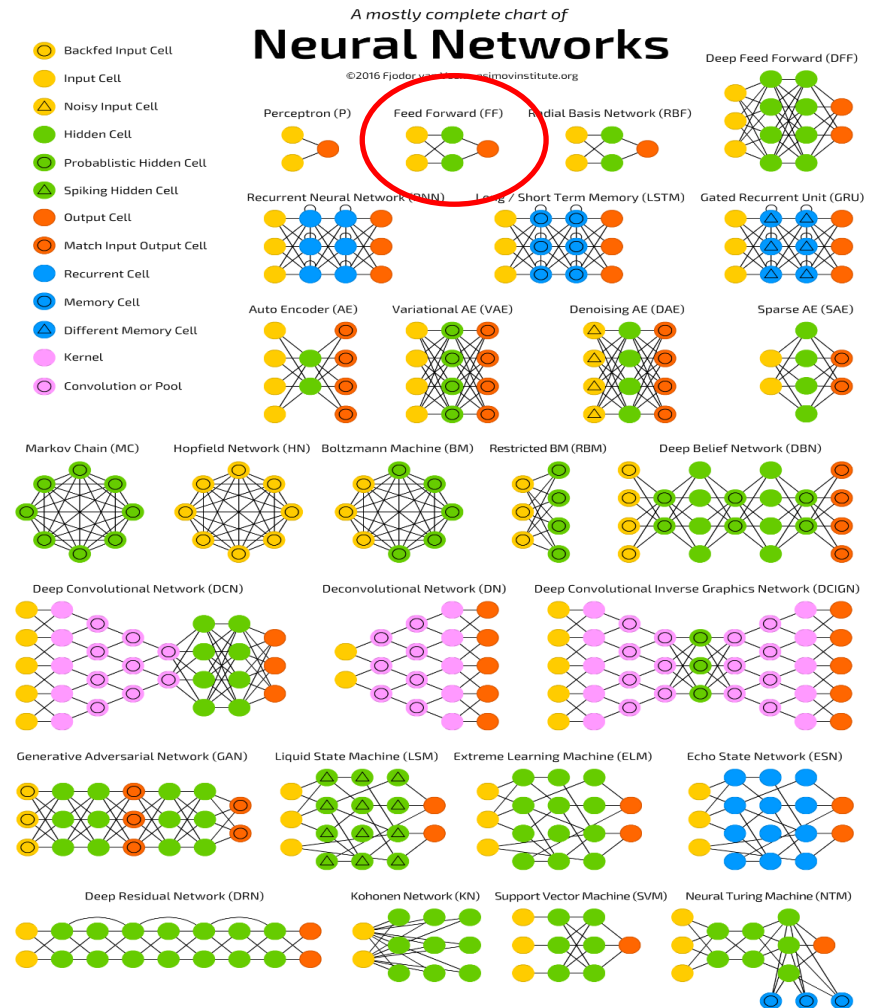
ANN

A lot of different types of layers:

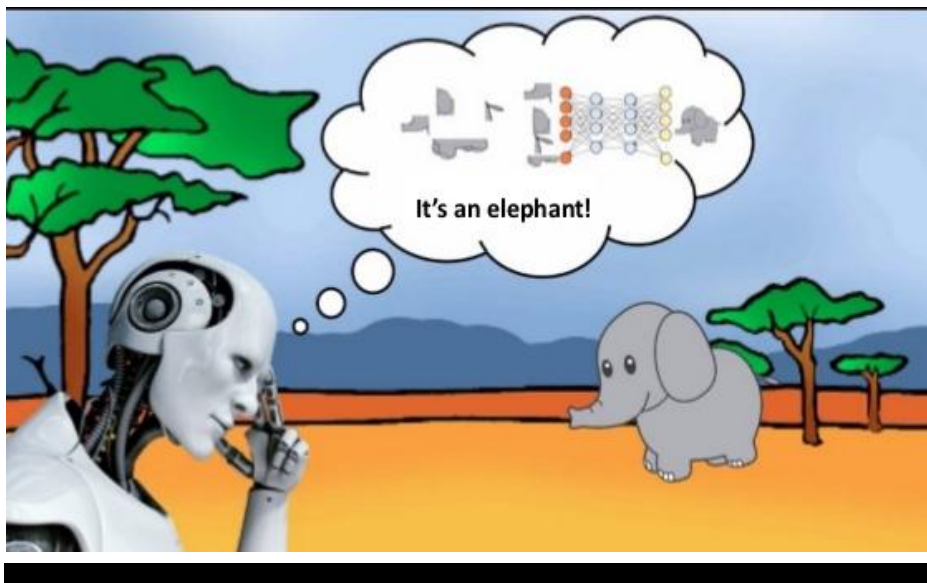
- Fully connected (dense)
- Convolutional
- LSTM
- GRU
- Attention
- ...

A lot of different topologies:

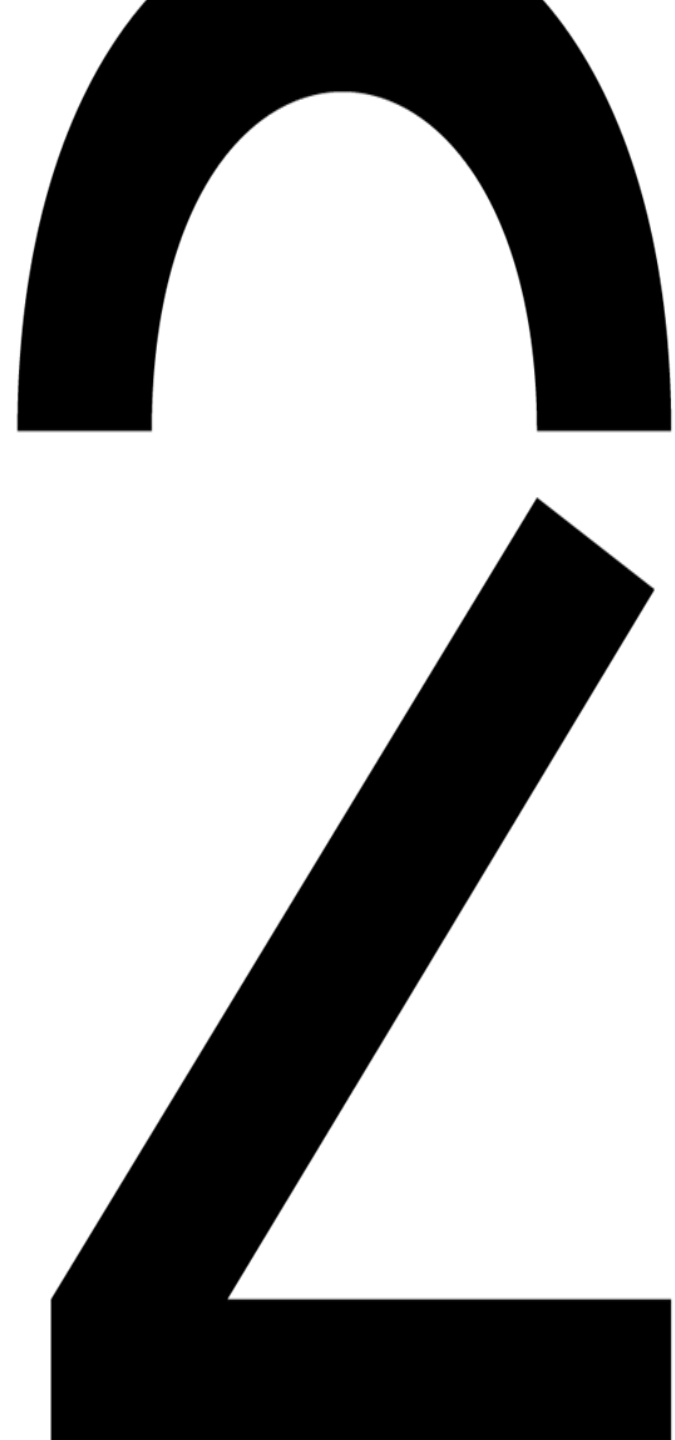
- Feed Forward NN
- Recurrent NN
- Autoencoders
- GAN
- Graph NN
- ...



This course: **Feed Forward NN with fully connected (dense) layers**



Training a Neural Network



Dataset

Split dataset in training, validation and test data

1. Training data

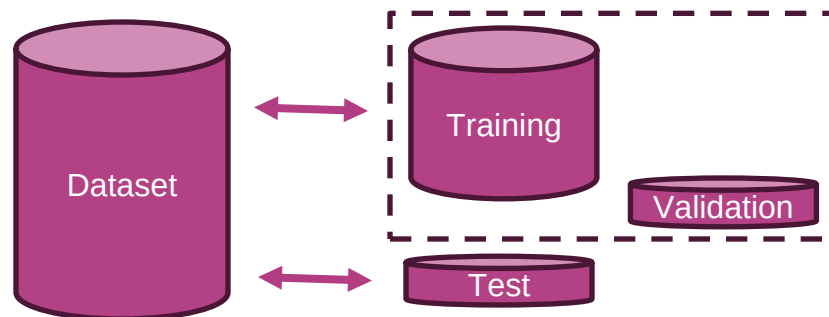
- Used to train the model
- Good representation of your data, e.g. all classes are equally represented (in case of a classification model)
- Size: typical 80-90% of your dataset

2. Validation data

- Used during training to follow up how well the training is doing
- To check whether there is no overfitting
- Size: typical 5-10% of your dataset

3. Test data

- Used after training to evaluate how good to model is doing
- Size: typical 5-10% of your dataset



Training a NN

Very simple example:

- Model with 7 inputs and 3 outputs
- Training set size = 6
- Batch size = 2

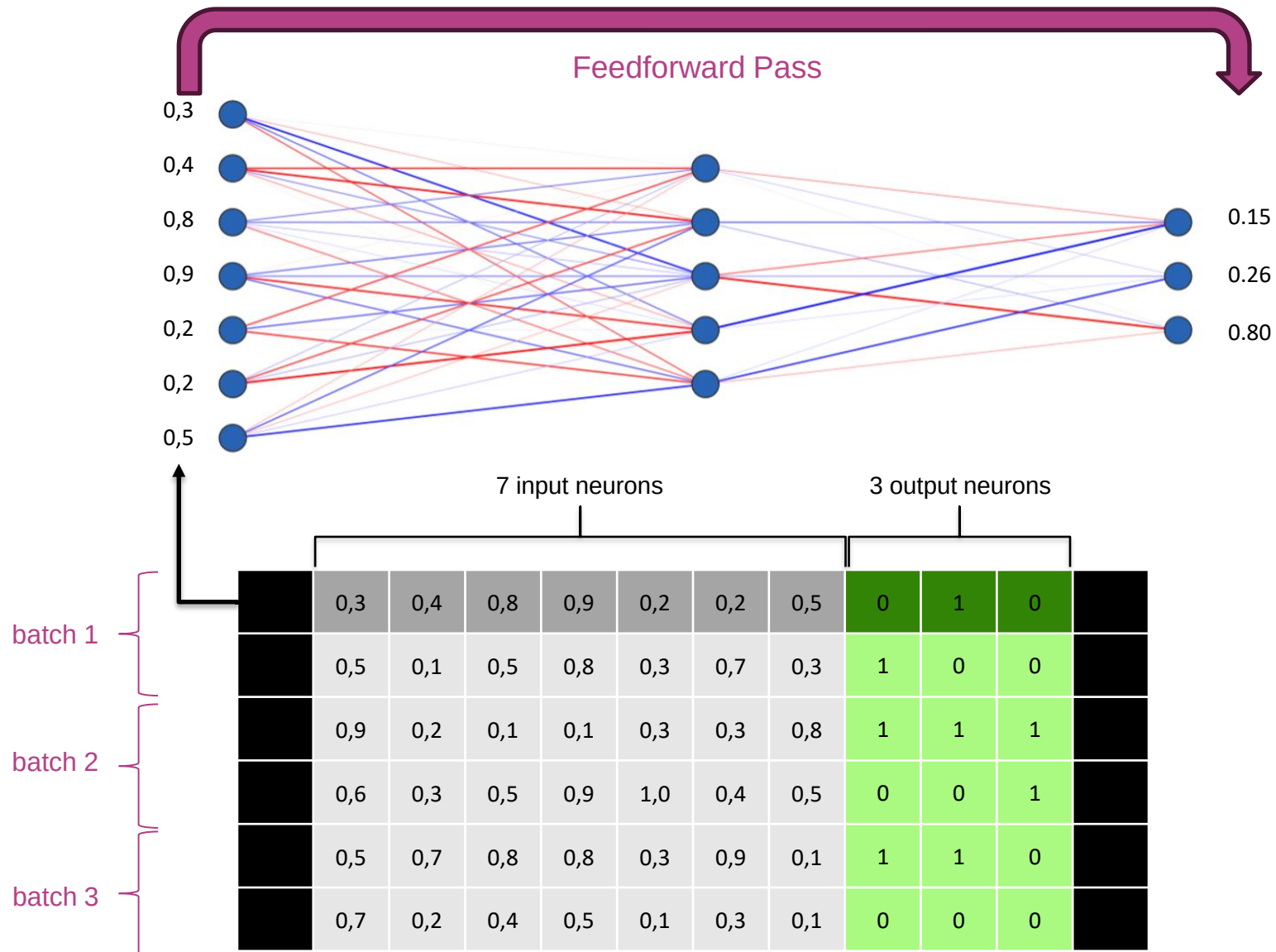
Supervised learning:

- Tell the model what the output should be for a certain input

		7 input neurons							3 output neurons				
batch 1		0,3	0,4	0,8	0,9	0,2	0,2	0,5	0	1	0		
		0,5	0,1	0,5	0,8	0,3	0,7	0,3	1	0	0		
batch 2		0,9	0,2	0,1	0,1	0,3	0,3	0,8	1	1	1		
		0,6	0,3	0,5	0,9	1,0	0,4	0,5	0	0	1		
batch 3		0,5	0,7	0,8	0,8	0,3	0,9	0,1	1	1	0		
		0,7	0,2	0,4	0,5	0,1	0,3	0,1	0	0	0		

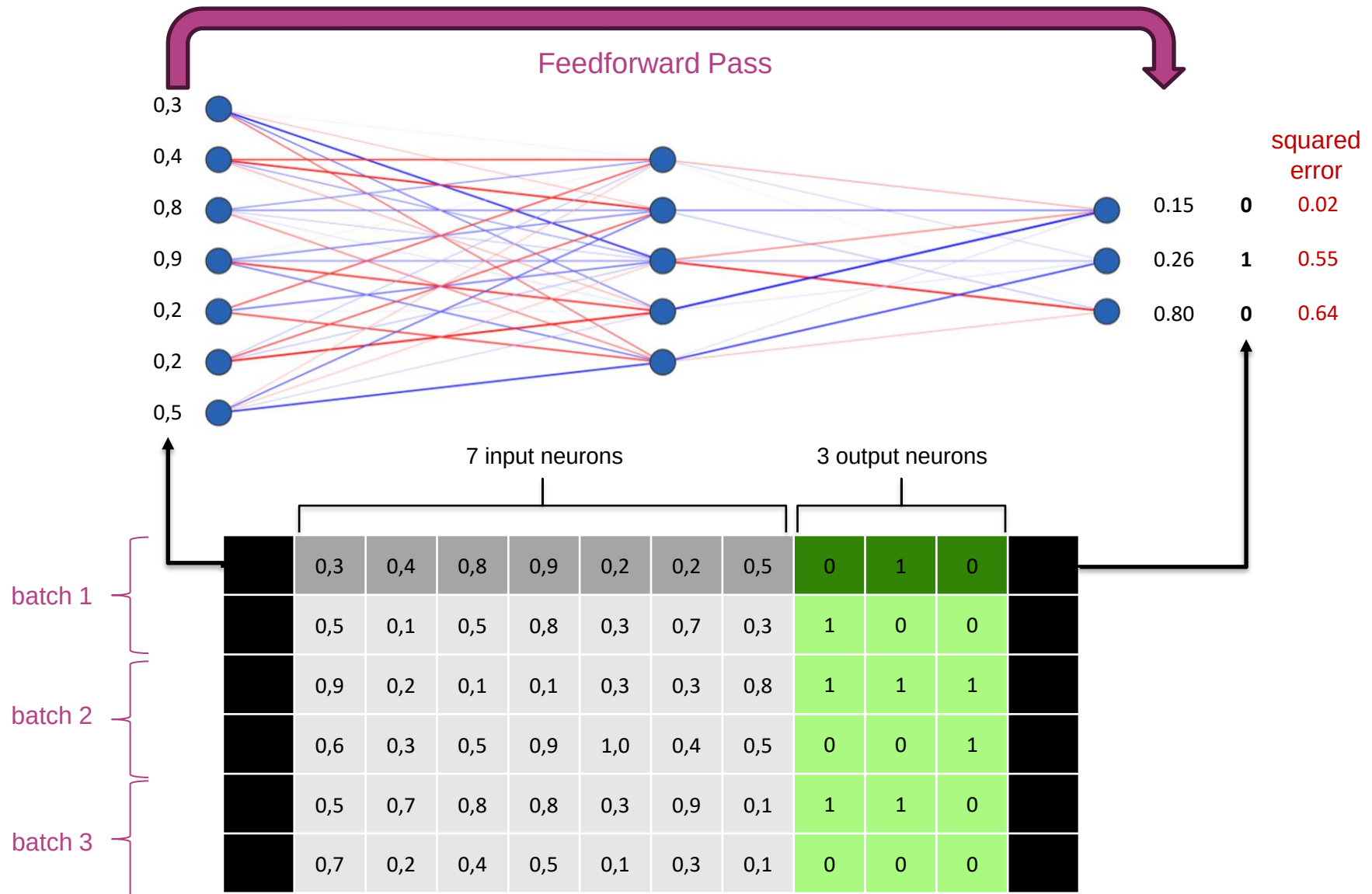
Training a NN – feedforward

Start with random weights and biases and calculated output



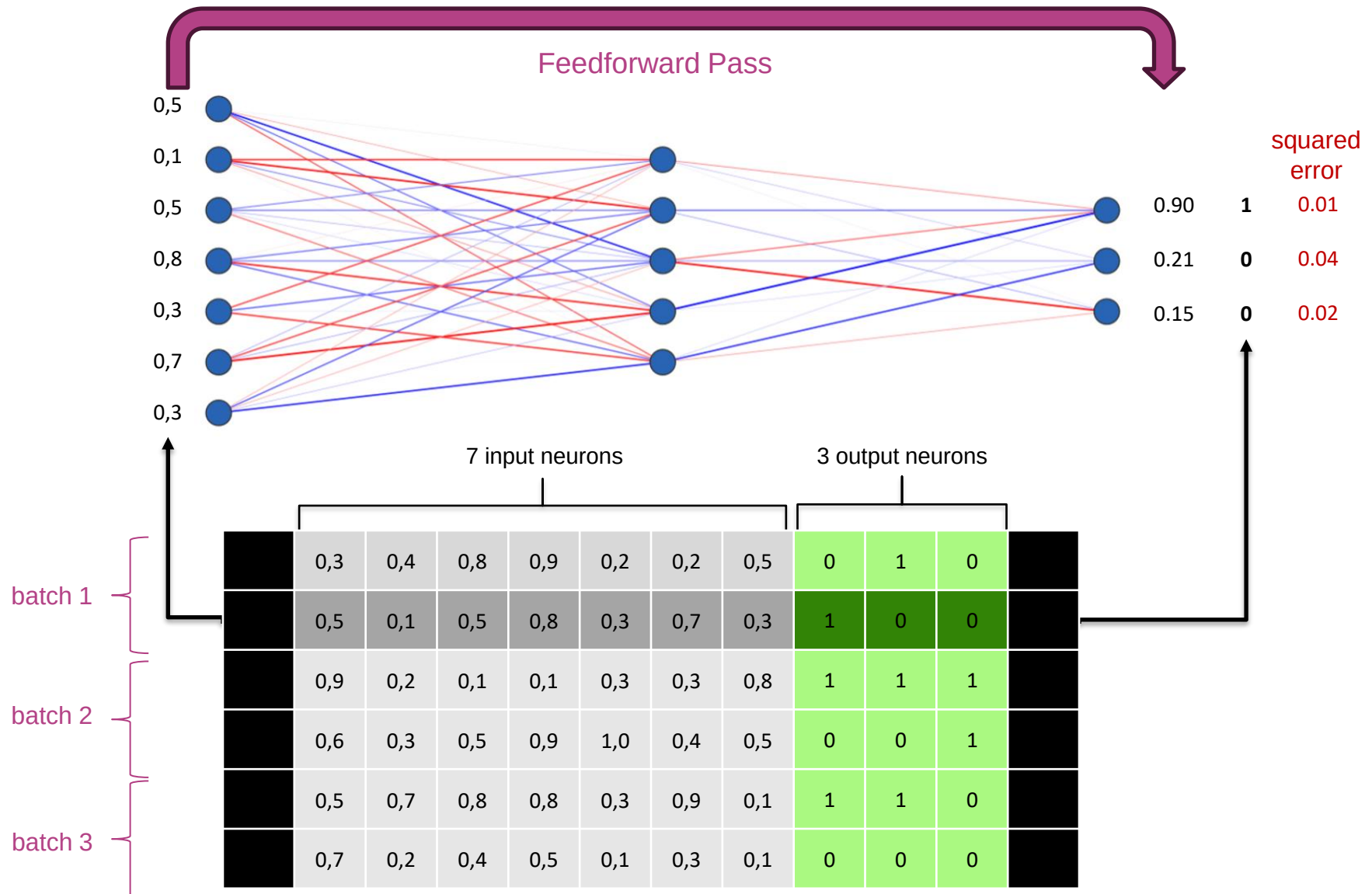
Training a NN – feedforward

Compare with what the output should be



Training a NN – feedforward

Repeat for other samples in the batch

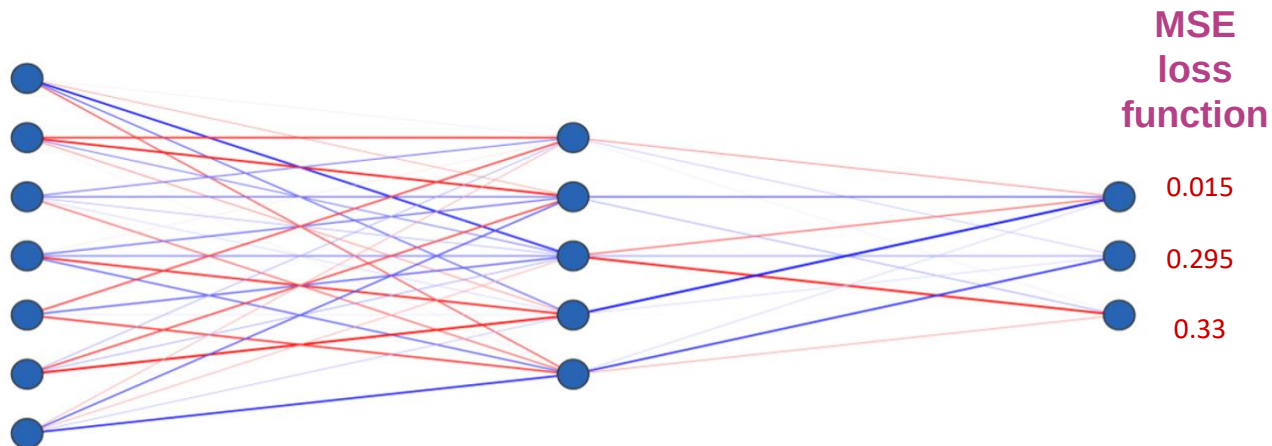


Training a NN – loss function

The **loss function** is calculating the mean error over the batch. Several loss functions exist:

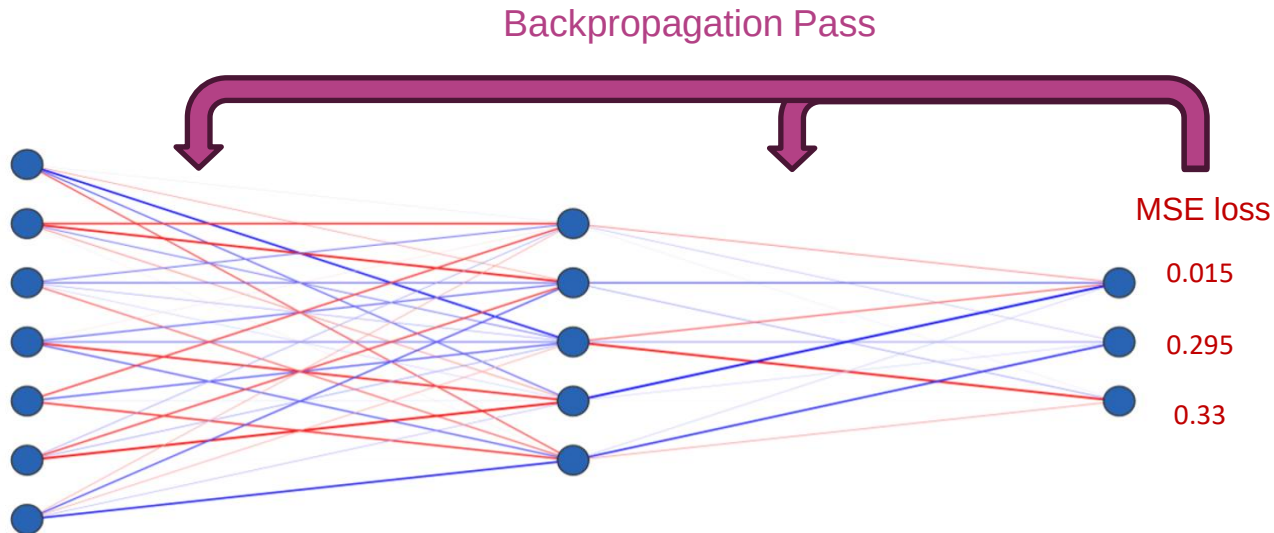
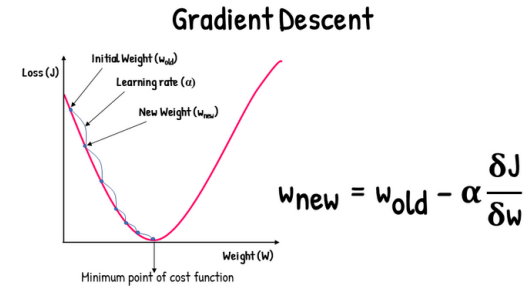
- Mean squared error (MSE) loss (used in this example)
- Mean absolute error (MAE) loss
- Binary cross entropy loss
- Categorical cross entropy loss
- ...

The loss function measures how good the model is performing (the lower, the better). The goal of the training is to minimize the loss function.



Training a NN – backpropagation

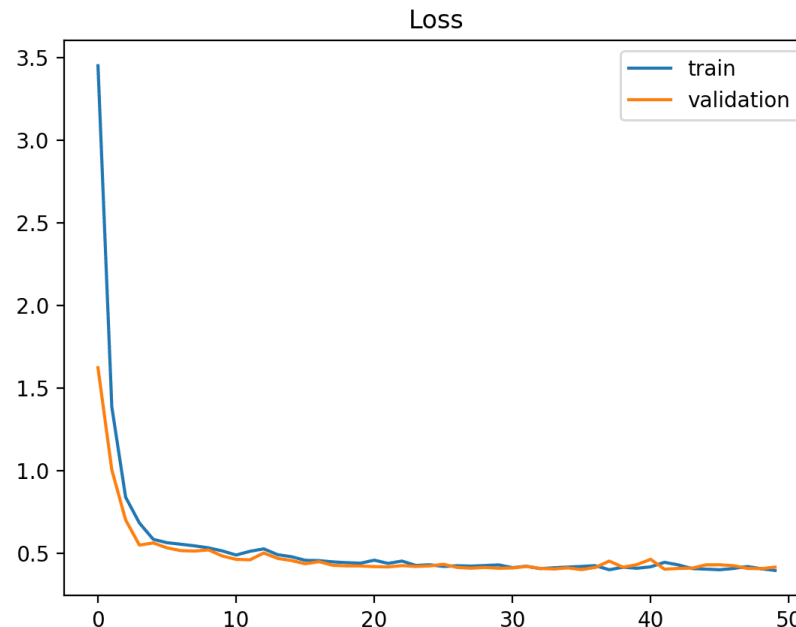
Backpropagation is adapting the weights and biases based on the output of the loss function. This is done by an optimizer algorithm called **gradient descent**. Today a variant of this algorithm called **adam** is mostly used.



Backpropagation is done after every **batch**. So in our very simple example, the feedforward/backpropagation is executed 3 times when going over the training dataset (1 **epoch**).

Training a NN – epochs

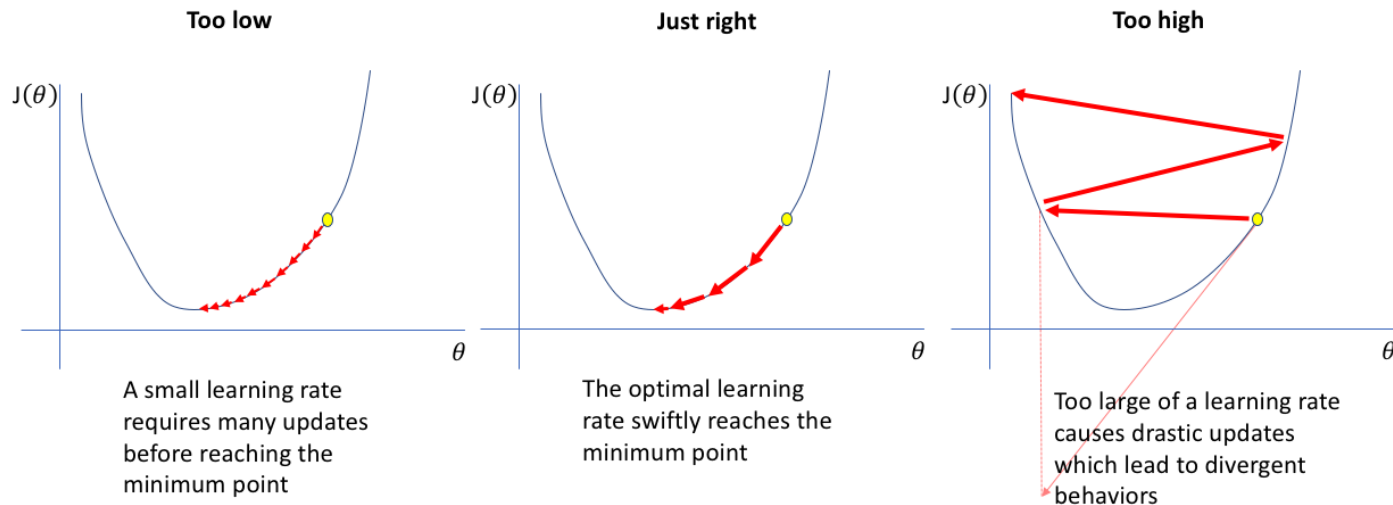
Training is done for a number of **epochs** (1 epoch is 1 pass over the training dataset). After every epoch, the loss function is also calculated for the validation data set. The number of epochs used for the training is a hyperparameter you have to define. It is also possible to automatically stop the training if the loss function is not going down anymore.



Training a NN – learning rate

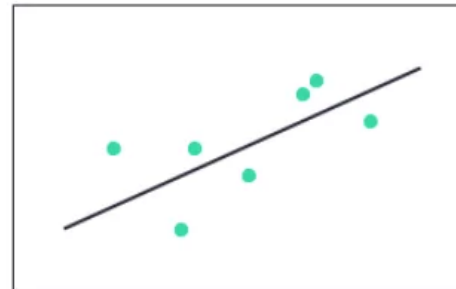
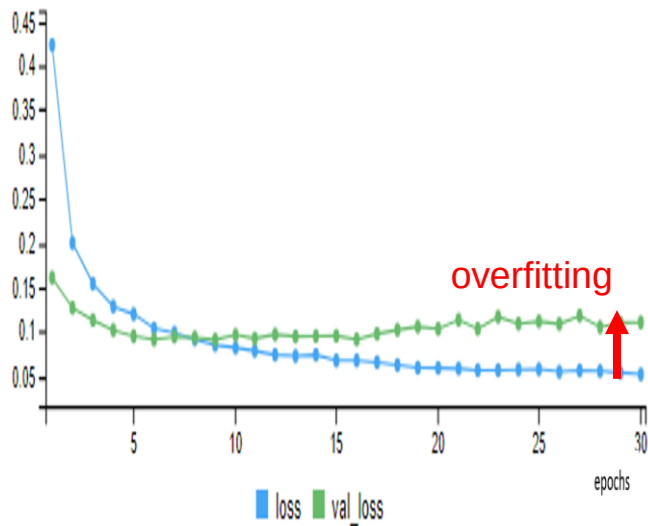
The **learning rate** is the step size in the gradient descent or adam optimizer

- A small learning rate will be slow and requires a lot of epochs
- A too large learning rate will not find the minimum of the loss function
- Typical learning rate are between 0.01 and 0.0001
- Sometimes a learning rate schedule is defined, starting with a bigger value and decreasing the step after a number of epochs

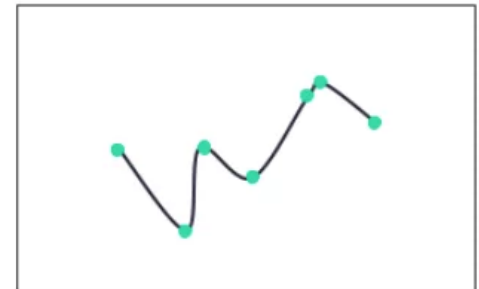


Training a NN – overfitting

Overfitting is happening when the model is performing well on the training dataset, but performing worse on the validation dataset (validation loss is higher than training loss). Most of the time, this is coming from the fact that the dataset is too small to train the model. Sometime **data augmentation** techniques are used to create more samples (e.g. create more pictures by rotating the existing ones a bit)



well generalized



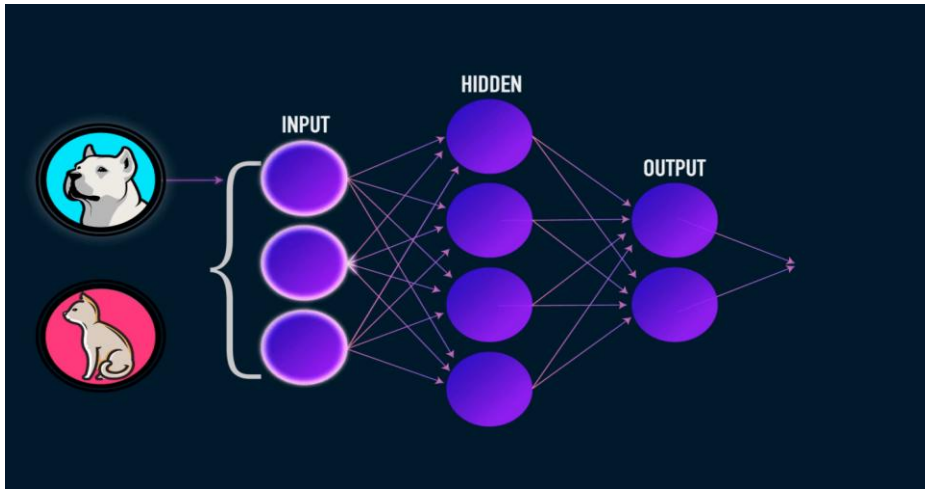
overfitting

Training a NN – hyperparameter tuning

A lot of **hyperparameters** need to be defined for the NN model

- Number of layers
- Number of nodes in the layers
- Number of epochs
- Batch size
- Learning rate
- Type of activation function
- Type of loss function
- ...

In reality several model with different hyperparameters are trained and the model giving the best results for the validation dataset is chosen. This process is called **hyperparameter tuning**.



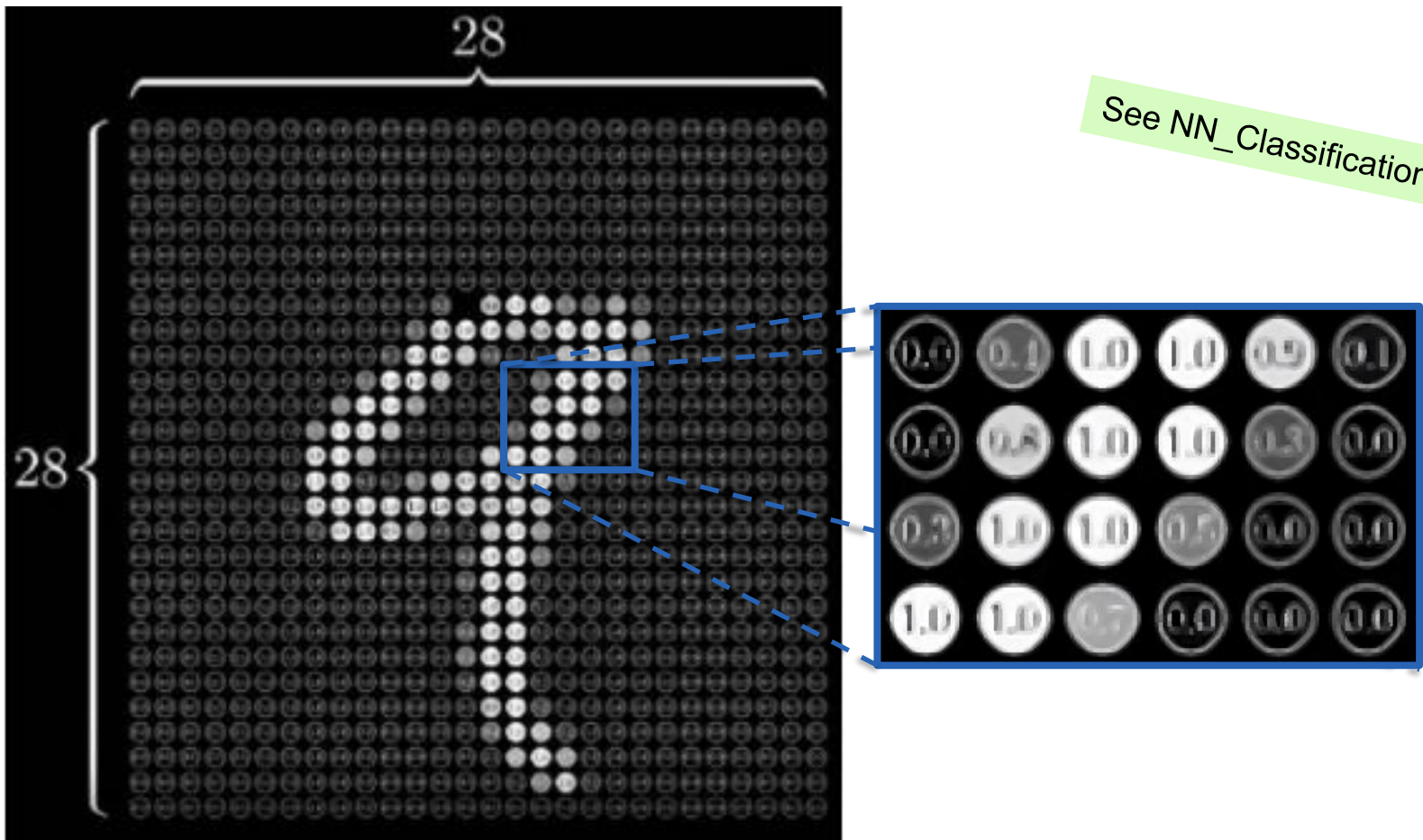
Neural Network – Classification

Classification

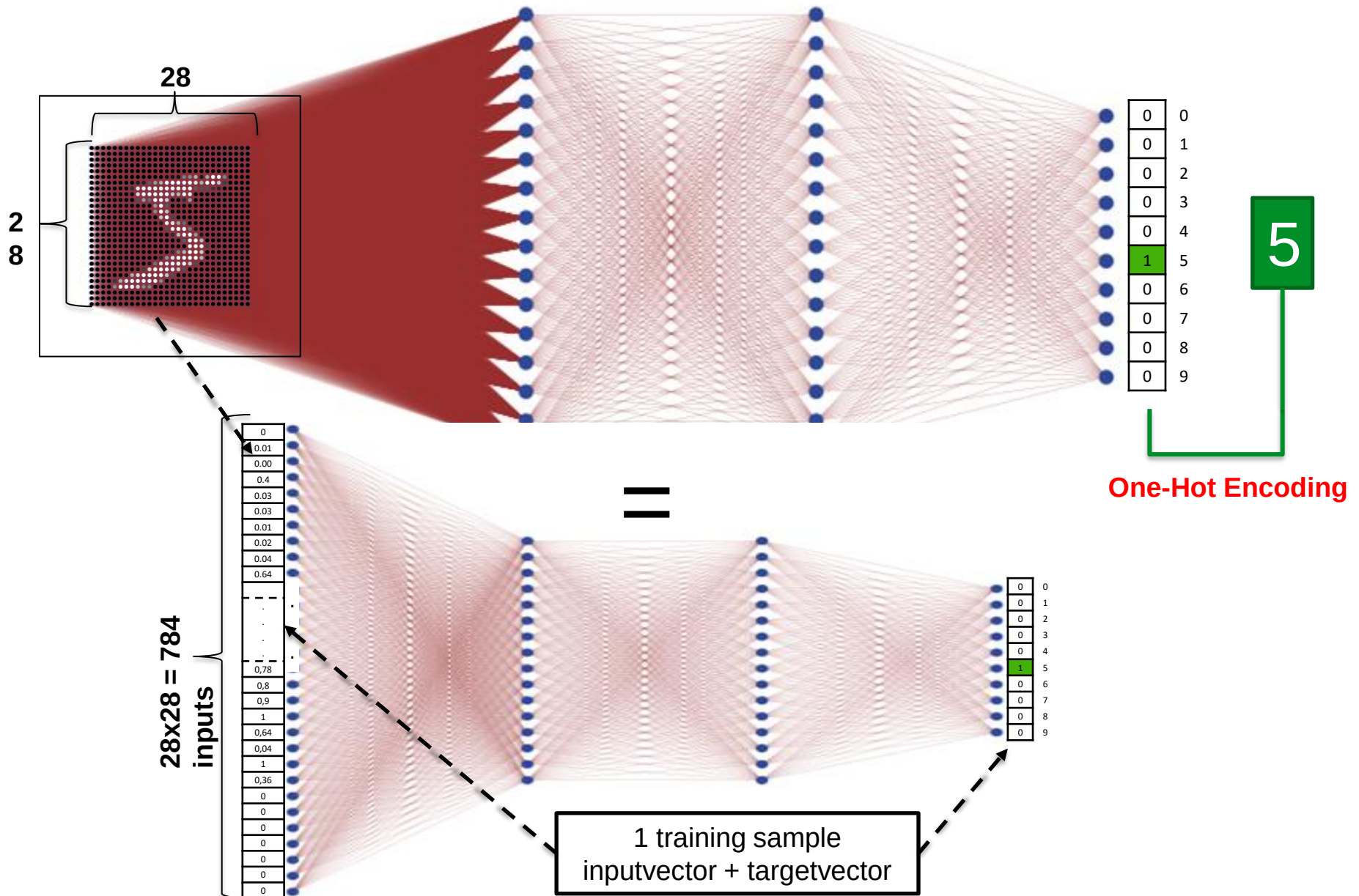
Classification models are used to predict discrete categories or classes.

Example: Make a model that can recognize handwritten digits.

MNIST dataset: The training dataset has 60000 images, the test dataset 10000. Images are grayscale and have a size of 28x28 pixels.

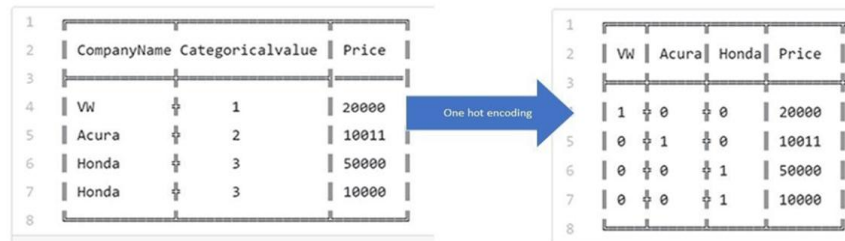


Classification

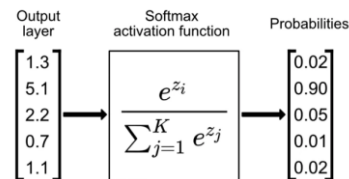


Classification

One-Hot Encoding: transforms categorical variable into binary columns



When the classification model predict a class, each output node will give the probability for the corresponding class. The sum of all the output nodes (sum of all the probabilities) is equal to 1. To enforce this, the **softmax** activation function is used for the output layer together with a **categorical_crossentropy** loss.



For one-hot encoding the `to_categorical` function from `tensorflow.keras.utils` can be used. In case the dataset is a pandas dataframe, the `pd.get_dummies` function can be used.

Remark: in case of a binary classification (“positive”/“negative”), it is also possible to have 1 output node (so not using one-hot encoding), giving the probability of “positive”. In this case a sigmoid activation is used (giving a value between 0 and 1) together with a `binary_crossentropy` loss function.

Define the model

input layer: 784 nodes

```
inputs = Input(shape=(784,))
```

2 hidden layers: 128 and 64 nodes and relu activation

```
x = Dense(128, activation='relu')(inputs)
```

```
x = Dense(64, activation='relu')(x)
```

output layer: 10 nodes and softmax activation

```
outputs = Dense(10, activation='softmax')(x)
```

construct the model

```
model = Model(inputs, outputs, name='MNIST')
```

print a summary

```
model.summary()
```

compile the model

categorical_crossentropy loss

```
model.compile(  
    optimizer=Adam(learning_rate=0.001),  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Model: "MNIST"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 784)]	0
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 64)	8256
dense_2 (Dense)	(None, 10)	650
=====		
Total params: 109386 (427.29 KB)		
Trainable params: 109386 (427.29 KB)		
Non-trainable params: 0 (0.00 Byte)		

Train, evaluate and predict

train the model

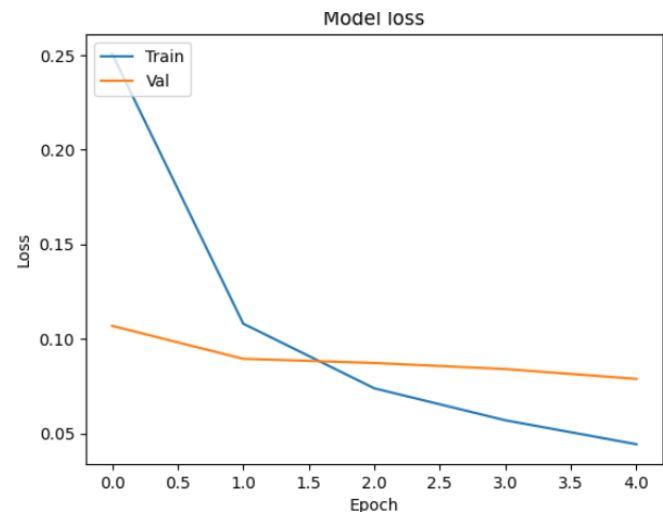
```
history = model.fit(  
    x_train_norm,          # training input: normalized, e.g. with MinMaxScaler()  
    y_train_onehot,        # training targets: one-hot encoding  
    epochs=5,  
    batch_size=32,  
    validation_split=0.1,  # 10% of training data used for validation  
)
```

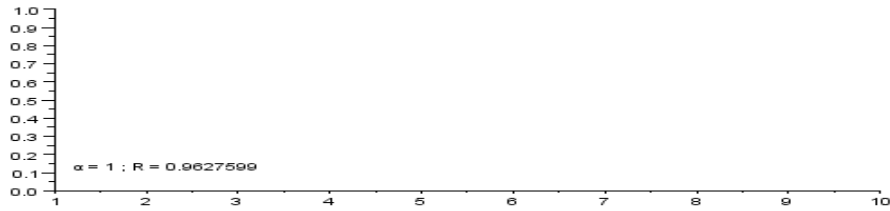
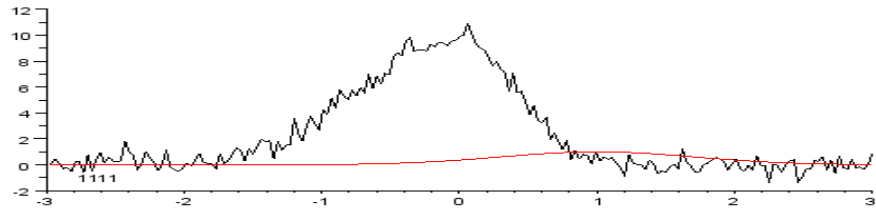
evaluate

```
model.evaluate(x_test_norm, y_test_onehot)
```

predict: also normalize first!

```
predicted = model.predict(x_test_norm)
```





Neural Network – Regression

Regression

See [NN_Regression.ipynb](#)

Regression models are used to predict continuous values

- For regression usually a **linear** activation is used for the output node.
- The loss function can be an **mean squared error** (MSE) or **mean absolute error** (MAE) loss
- As metric, mean absolute percentage error (MAPE) can be used

	output activation	loss	metrics
regression	linear	mean_squared_error	mean_absolute_percentage_error
		mean_absolute_error	mean_absolute_percentage_error
classificatie one-hot encoding	softmax	categorical_crossentropy	accuracy
binary classification	sigmoid	binary_crossentropy	accuracy

Exercise time!

See `NN_Exercises.ipynb`

Data Science 2

Meta-heuristieken (deel 1)

Agenda

1. Inleiding – Optimalisatieproblemen
2. Algoritme versus heuristiek
 - ↳ Computationale complexiteit
3. Soorten heuristieken
4. Simulated annealing



Inleiding - Optimalisatieproblemen



Inleiding - Optimalisatieproblemen

Problemen –in bedrijfscontext-

- steken dagelijks de kop op
- toenemende complexiteit

Probleem oplossen

= Beslissing nemen

= Keuze maken uit verschillende alternatieven

Doel: beste resultaat, rekeninghoudend met **bestaande beperkingen** (geld, tijd, beschikbare mensen, aanwezige kennis, grondstoffen, wetgeving,...)

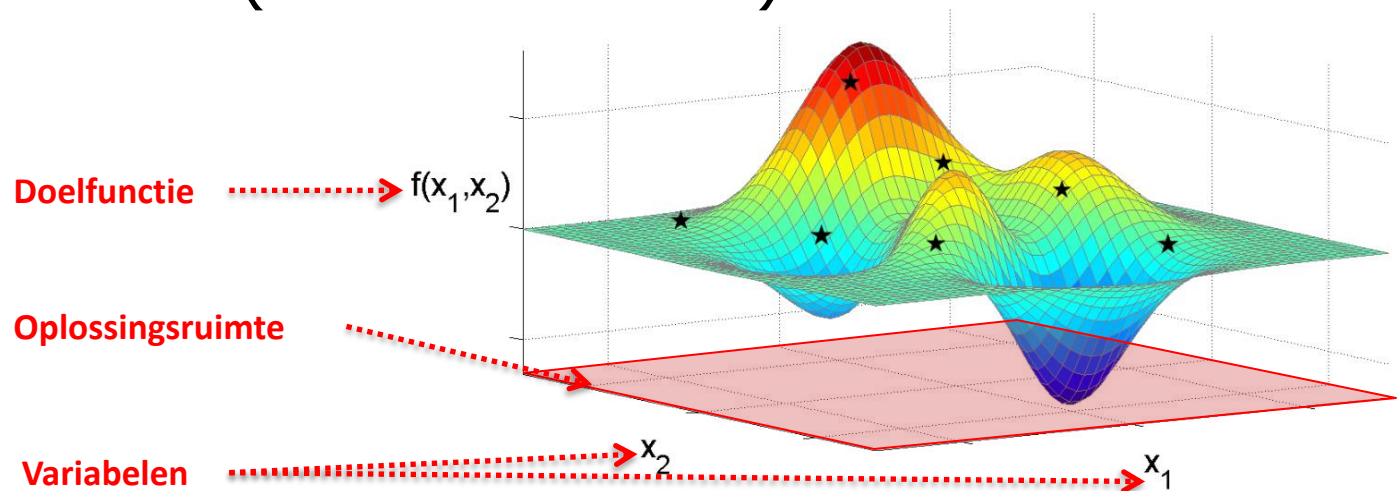


Inleiding - Optimalisatieproblemen

Optimalisatieprobleem omvat:

- **Variabelen**
- Omschrijving van de verzameling van alle mogelijke **oplossingen**(*)
- Lijst van beperkingen: **constraints**
- Een te maximaliseren of te minimaliseren **doelfunctie** (of **kostfunctie**)

**oplossings-
ruimte**

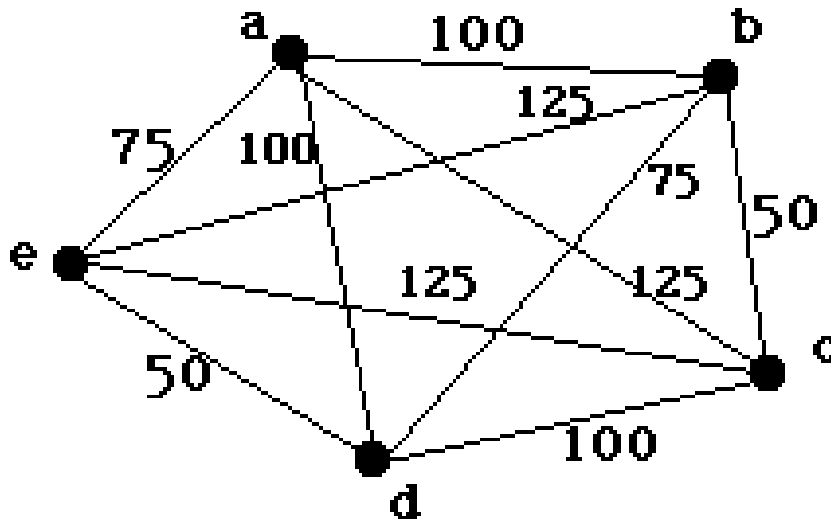


(*) een oplossing = toekennen van concrete waarden aan de variabelen

Inleiding - Optimalisatieproblemen

Voorbeeld: Handelsreizigersprobleem (Traveling Salesman Problem of TSP)

Als er n steden gegeven zijn die een **handelsreiziger** moet bezoeken, samen met de afstand tussen ieder paar van deze steden, **zoek** dan **de kortste weg** die kan worden gebruikt, waarbij iedere stad eenmaal wordt bezocht.



	a	b	c	d	e
a	0	100	125	100	75
b	100	0	50	75	100
c	125	50	0	100	125
d	100	75	100	0	50
e	75	100	125	50	0

Hoe ziet een oplossing eruit?

Opeenvolging van de steden : (a,e,d,b,c)

Waarde doelfunctie?

$$75 + 50 + 75 + 50 + 125 = 375$$

Aantal mogelijke oplossingen?

$$5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

(alle mogelijke permutaties, algemeen: $n!$)

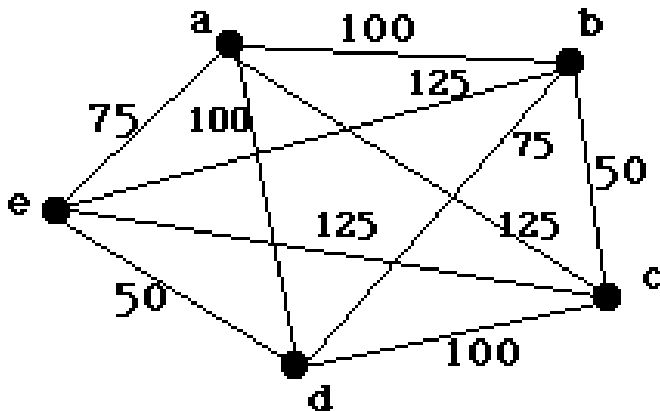
Inleiding - Optimalisatieproblemen

Hoe de 'beste' oplossing bepalen?

Voor de hand liggende benadering:

- alle oplossingen bepalen
- voor elke oplossing de doelfunctie bepalen
- de oplossing nemen met de laagste doelfunctie

⇒ **Enumeration method**



Oplossingsruimte
kan onhandel-
baar groot zijn



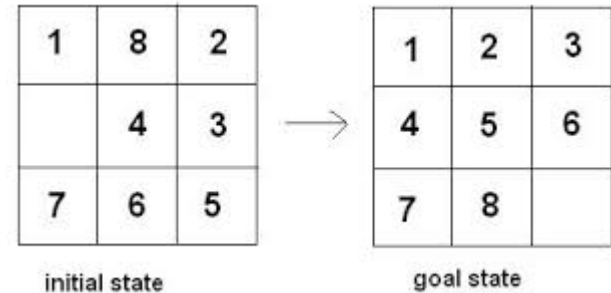
n	n!
5	120
10	3.628.800
15	1.307.674.368.000
20	2.432.902.008.176.640.000

Inleiding - Optimalisatieproblemen

De 8 – puzzel

Vakjes in juiste volgorde zetten

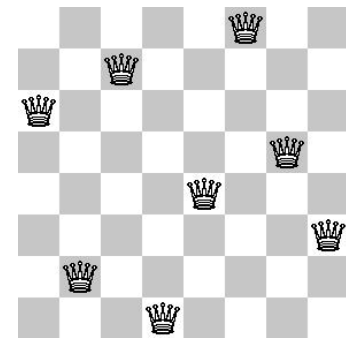
Hoe ziet *een* oplossing eruit?

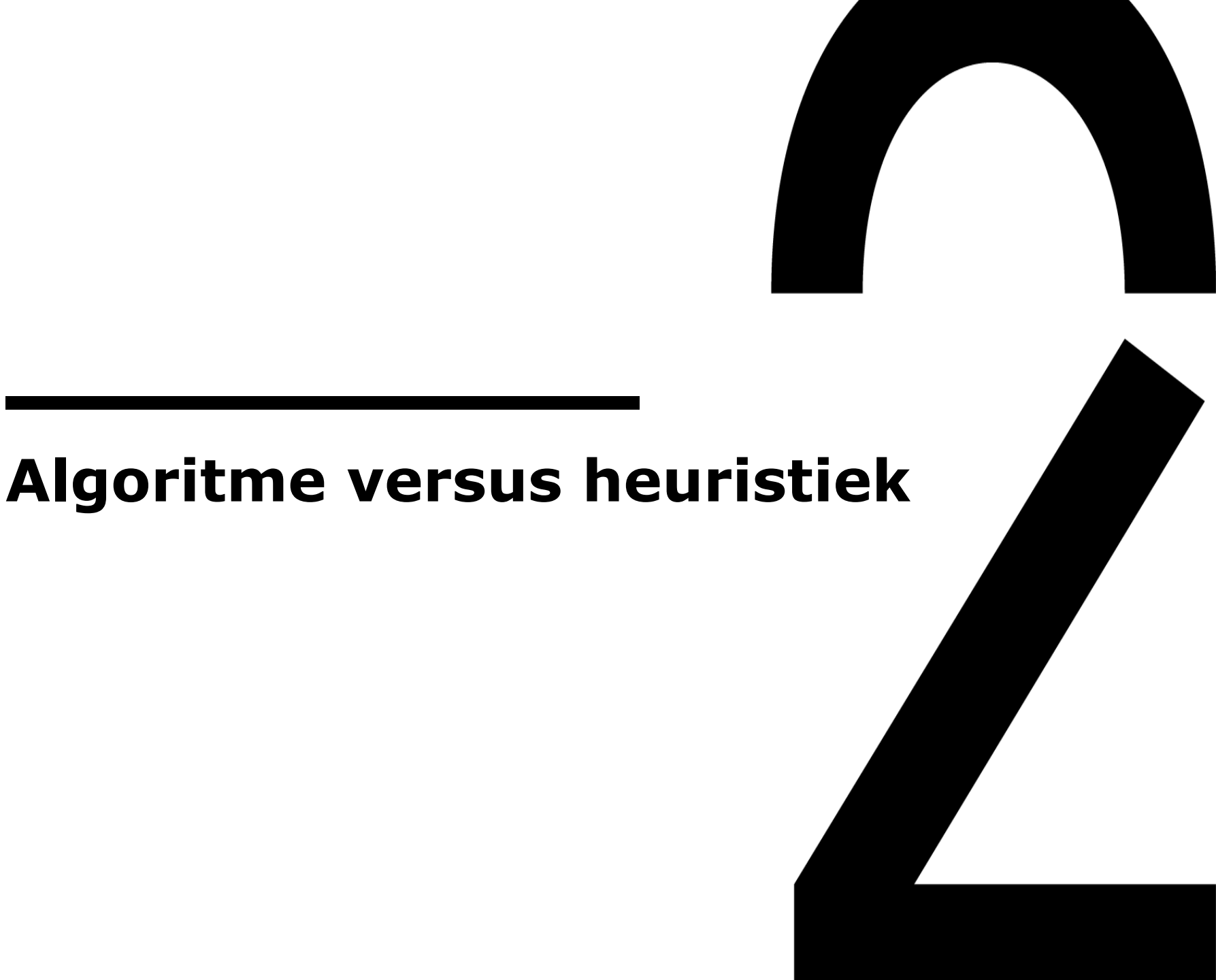


N – Queens

Acht koninginnen op schaakbord plaatsen
zonder dat ze elkaar kunnen aanvallen

Hoe ziet *een* oplossing eruit?



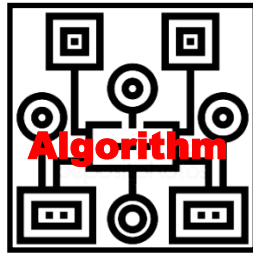
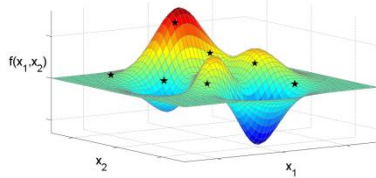


Algoritme versus heuristiek

Wat is een algoritme en wat is een heuristiek?

Doel optimalisatieprobleem: 'Beste' oplossing vinden tussen alle mogelijke oplossingen.

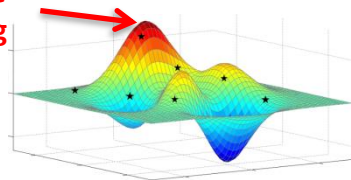
Optimalisatieprobleem



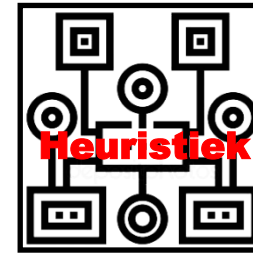
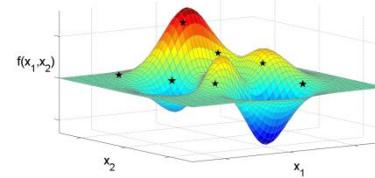
Algorithm



Optimale oplossing



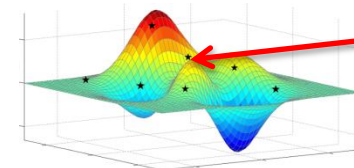
Optimalisatieprobleem



Heuristiek

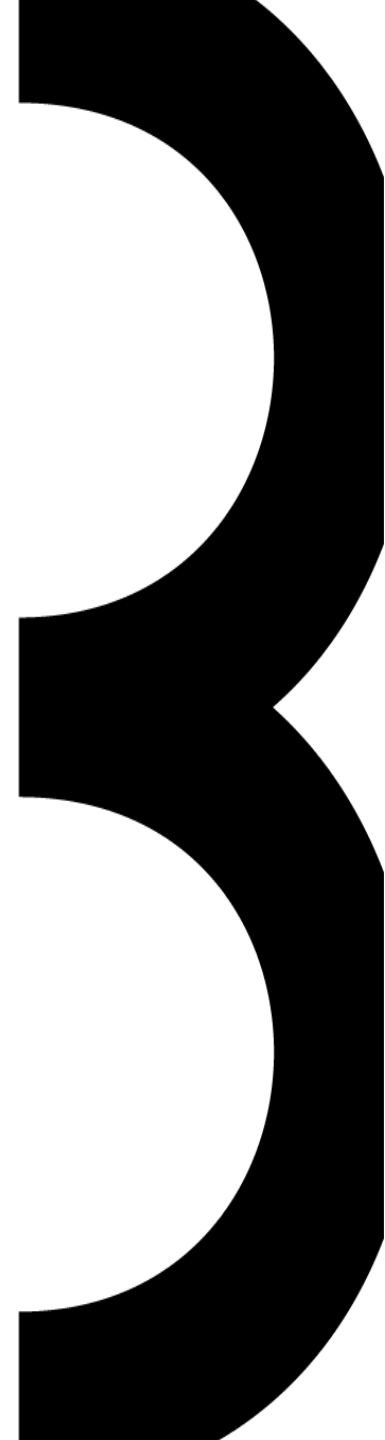


Goede oplossing



Waarom een heuristiek gebruiken?

Soorten heuristieken



'Custum made'-heuristieken

- ontwikkeld voor een specifiek optimalisatieprobleem
- niet herbruikbaar voor andere optimalisatieproblemen.
- gebruikt/exploiteert specifieke aspecten en eigenschappen niet noodzakelijk aanwezig in andere optimalisatieproblemen.

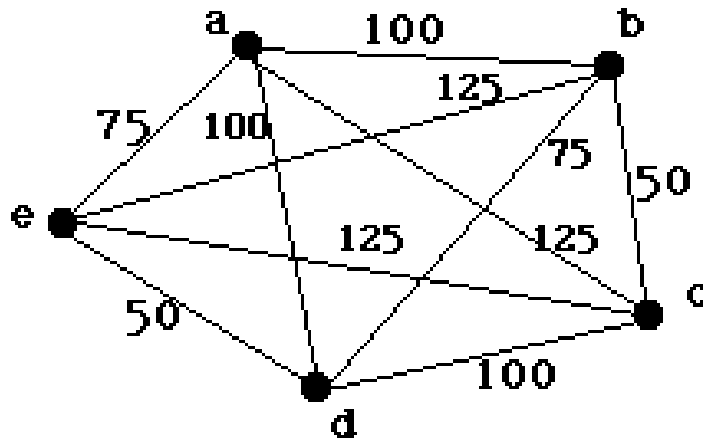


Soorten heuristieken

'*Custum made*'-heuristieken

D.m.v. een eenvoudig toe te passen criterium snel een goede oplossing bepalen

vb *nearest neighbour* benadering van het handelsreizigersprobleem:



Startpunt : a

a - dichtste buur: e

e - dichtste buur (nog niet bezocht): d

d - dichtste buur (nog niet bezocht): b

b - dichtste buur (nog niet bezocht): c

c – terug naar startpunt a

Oplossing : (a, e, d, b, c) met afstand 375

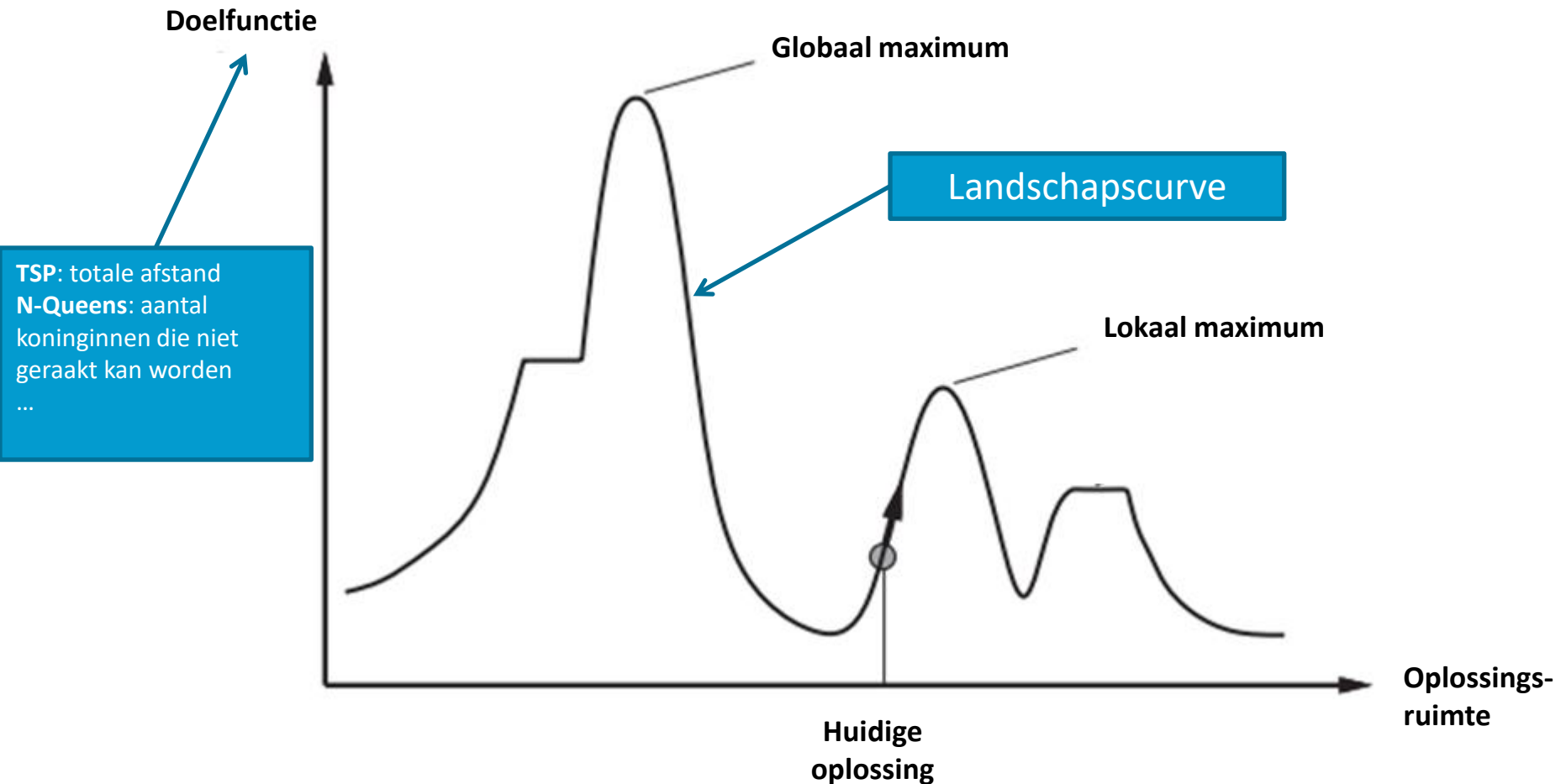
- Oplossing kan 'ver' van optimale oplossing liggen.
- 'Quick & dirty'-oplossing –beter dan random-oplossing-
- Worden vaak als startoplossing voor andere heuristieken (zie verder) gebruikt

'Lokale zoek'-heuristieken

Zoekmethoden die steeds in de 'buurt' zoeken van de vorige oplossing naar een betere oplossing. Een of meerdere stopcriteria worden gehanteerd.

- Slechts één of enkele oplossingen bijhouden en die "verbeteren"
- Oplossingen in de buurt zijn oplossingen met 'kleine' aanpassingen
- Resultaat is de 'beste' oplossing die tijdens de zoektocht werd gevonden (niet noodzakelijk de laatste oplossing)
- Risico 'vast' te raken in een lokaal minimum

Soorten heuristieken – Vizualiseren 'lokaal zoek'-heuristiek

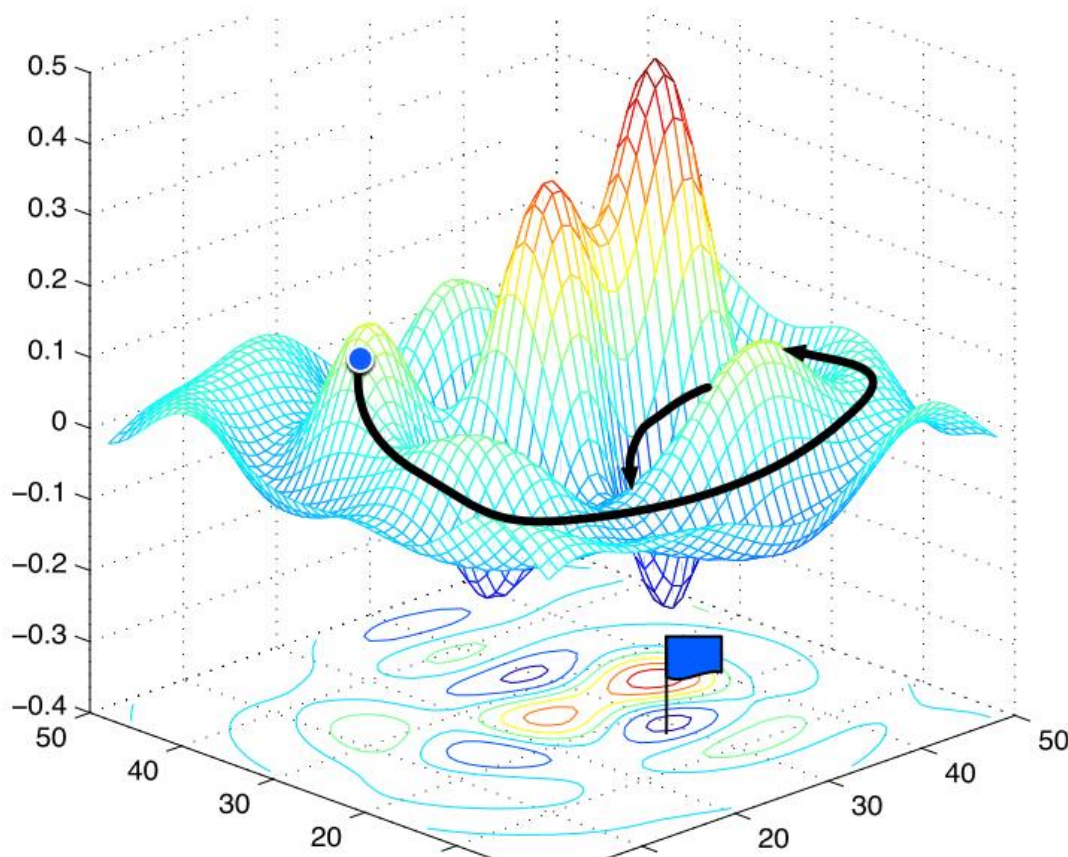


- Hoe voorkomen dat we vast komen te zitten?
 - Meerdere keren na elkaar uitgevoerd met andere initiële oplossingen
 - Zie meta-heuristieken

Soorten heuristieken – Visualiseren 'lokaal zoek'-heuristiek

Oplossingruimte als n-dimensionaal "landschap"

- Oplossingen zijn punten
- Hoogte van landschap is kwaliteit van de oplossing



Per stap evolueren de oplossingen richting een (lokaal) optimum

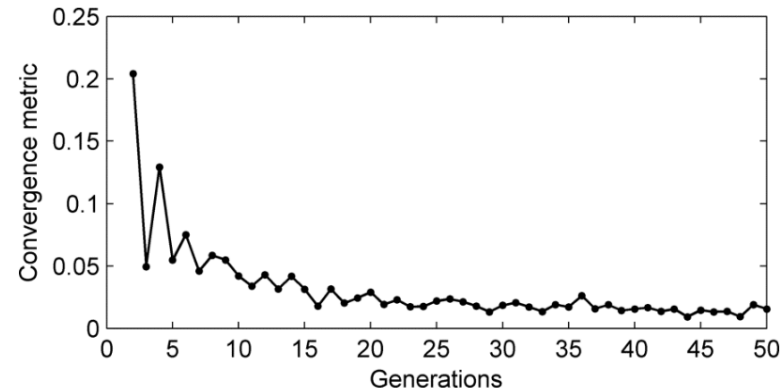
Soorten heuristieken

Meta-heuristieken

High level procedure toepasbaar op gelijk welk optimalisatie-probleem.

Het bevat de volgende elementen:

- Initiële oplossing bepalen
(at random of d.m.v. een eenvoudige heuristiek)
- Toepassen 'lokaal zoek'-principe: huidige oplossing vervangen door 'betere' oplossing in de buurt
- Toelaten om af en toe toch naar een 'slechtere' buur te gaan
(een manier om aan een lokaal optimum te ontsnappen)
- Gebaseerd op een analogie uit de fysica, de biologie of de ethologie
- Parameters sturen de duur van de heuristiek en de kwaliteit van de gevonden oplossing

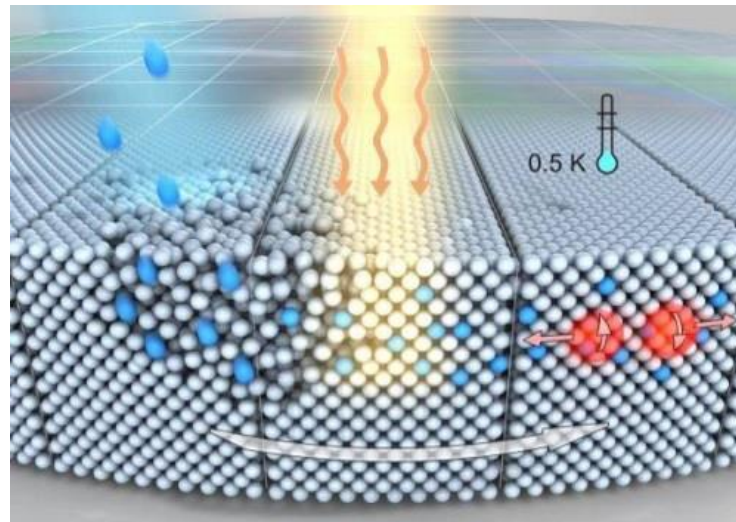


Soorten heuristieken

Voorbeeld van meta-heuristiek

Simulated Annealing(Kirkpatrick, e.a. ,1970)

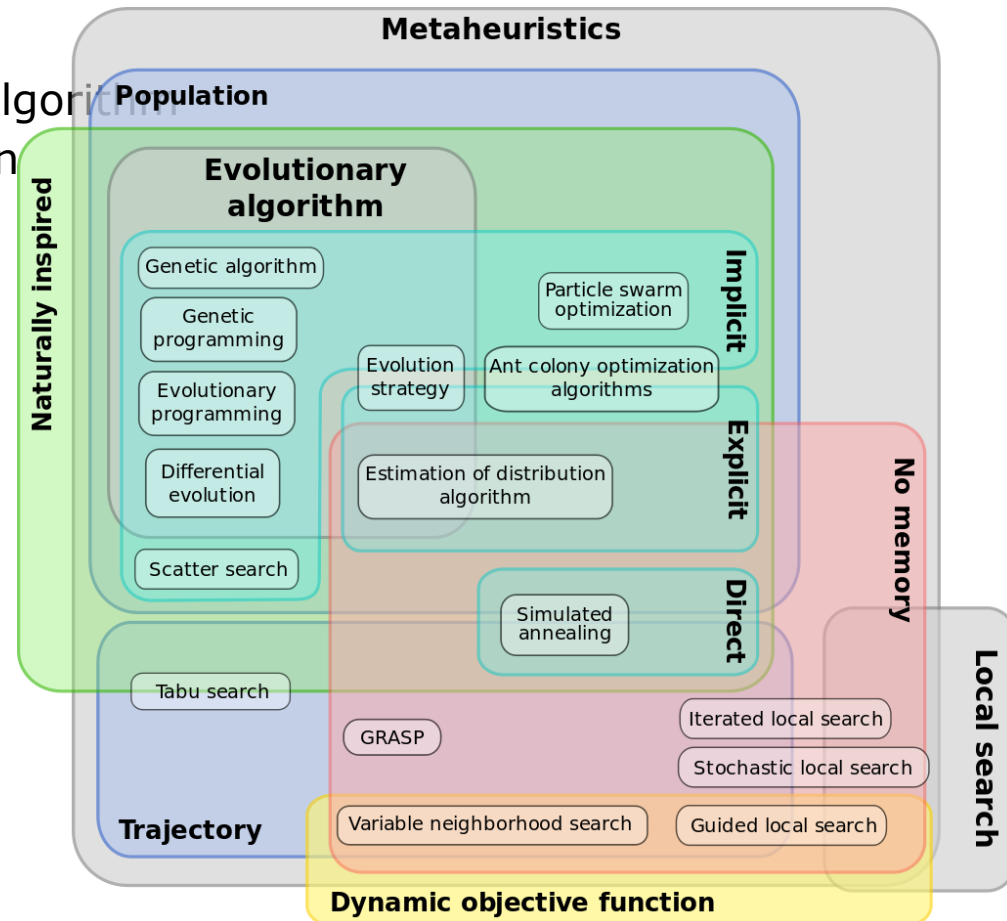
- gebaseerd op het afkoelen van een metaal
- potentiële oplossingen kristalliseren uit



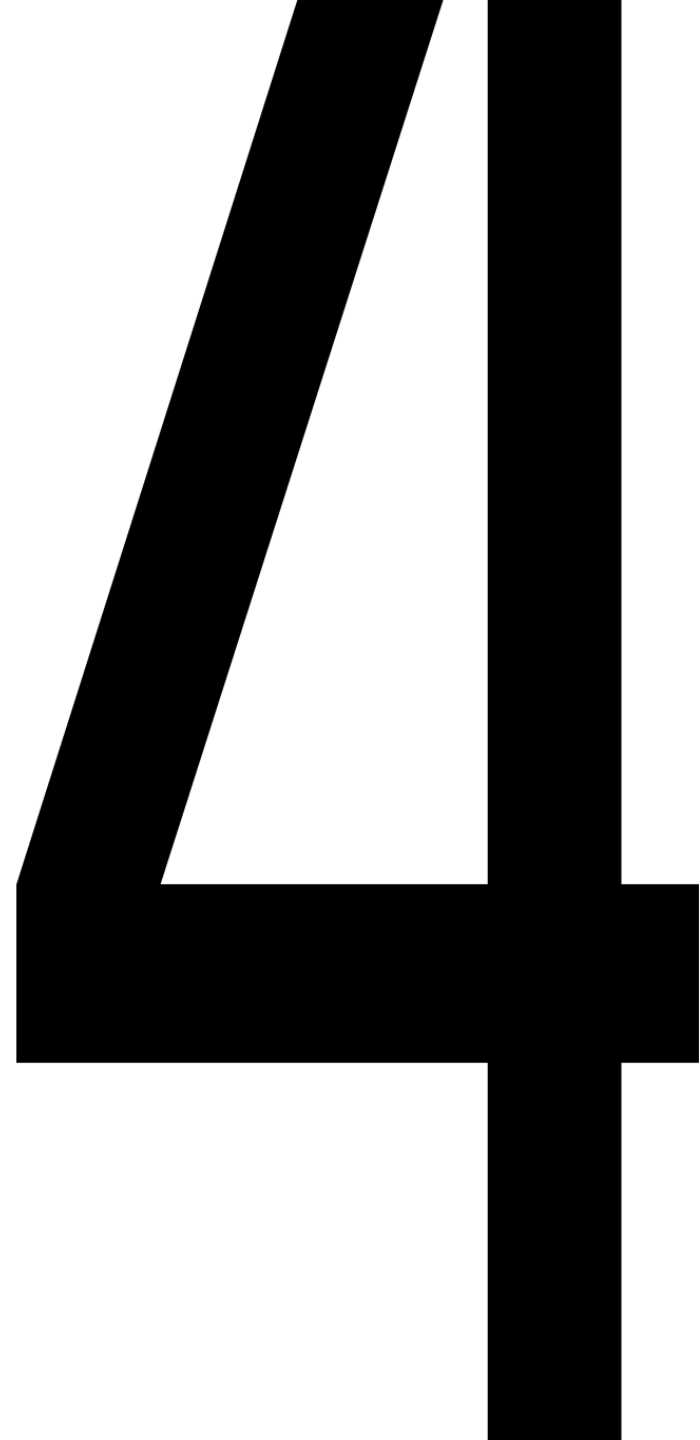
Soorten heuristieken

Andere voorbeelden van meta-heuristieken:

- ant colony optimisation algorithm
- genetic algorithm
- particle swarm optimization
- artificial immune systems
- bacterial foraging optimization algorithm
- biogeography-based optimization
- coevolutionary algorithms
- cultural algorithms
- differential evolution algorithm
- greedy randomized adaptive search procedure
- scatter search
- ...



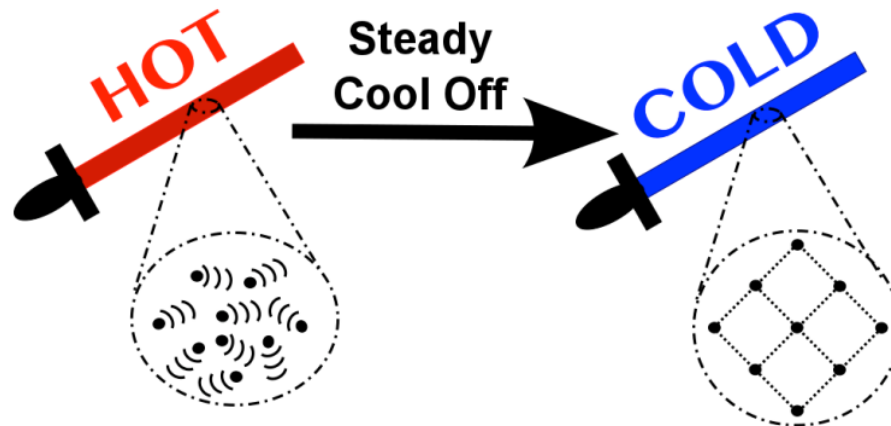
Simulated Annealing



Simulated Annealing

Gebaseerd op afkoelingsproces in materialen: annealing

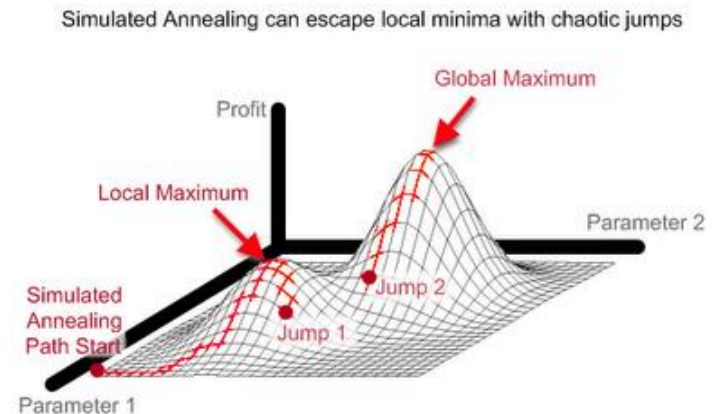
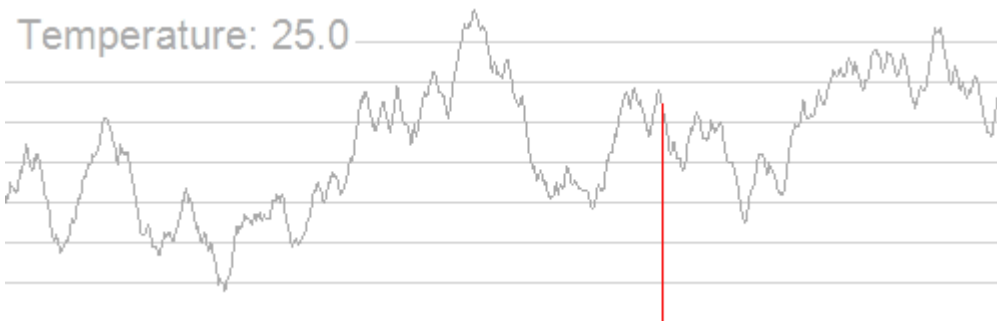
- atomen in materiaal bewegen, maar hoe lager de temperatuur, hoe minder
- Materiaal opwarmen om in juiste vorm te krijgen, nadien terug afkoelen
- afkoeling te snel $\Rightarrow$ onzuiverheden (amorfe structuren)
- afkoeling geleidelijk $\Rightarrow$ sterkere kristalstructuren
- Eindtoestand wordt dus geleidelijk aan bereikt.



Simulated Annealing

Simulated annealing

- Willekeurige zoektocht naar oplossing d.m.v. “**sprongen**” (random walk)
- In principe enkel “sprong” naar een betere oplossing
→ soms ook minder goede (*Waarom?*).
- Na elke “sprong” **beste oplossing** onthouden.
- Kans op “sprongen” naar minder goede oplossingen worden kleiner met **dalende temperatuur**
→ oplossing kristalliseert uit



Simulated Annealing – Pseudo-code

```
InitializeParameters (Temperature  $t$ , TemperatureReduction  $\alpha$ )
initialSolution (Solution  $s$ )
 $s^* = s$     //best found solution
while  $t > TMIN$ 
    temperatureIteration = 0
    While temperatureIteration < maxIterations
         $s' = \text{SelectNeighbour}(s)$ 
         $\Delta = \text{objectiveFunction}(s') - \text{objectiveFunction}(s)$ 
        // objectiveFunction must be minimized
        if ( $\Delta < 0$ )
            then  $s = s'$ 
                if  $\text{objectiveFunction}(s') < \text{objectiveFunction}(s^*)$ 
                    then  $s^* = s'$ 
            else if  $\text{atRandom}[0,1] < \exp(-\Delta/T)$ 
                then  $s = s'$ 
        end while
     $t = \alpha * t$ 
end while
return  $s^*$ 
```

Simulated Annealing - Temperatuurdaling

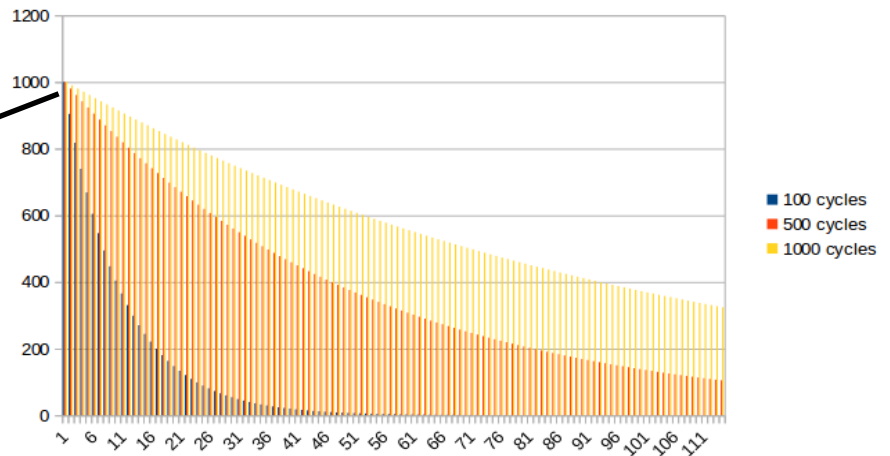
Two methods

1. Constante temperatuurdaling: per tijdseenheid temperatuur vaste stapgrootte laten dalen

2. Exponentiële temperatuurdaling: kies een begin- (T_s) en eindtemperatuur (T_e) kiezen en verlaag temperaturen telkens

$e^{\left(\frac{\log \frac{T_e}{T_s}}{c-1}\right)}$ met c = aantal cycles (iteraties) per temperatuurwaarde

Daling vanaf 1000 °C



Simulated annealing

In Python: Meerdere Python libraries ondersteunen Simulated Annealing. Een daarvan is de *simanneal*.

Zie [GitHub - perrygeo/simanneal: Python module for Simulated Annealing optimization](#) voor meer informatie omtrent deze package

```
>>> pip install simanneal    # alleen de eerste keer
```

Het doel van de simanneal library is om de probleem-specifieke berekeningen te scheiden van de Metaheuristiek-specifieke berekeningen:

Probleem-specifieke berekeningen

- **move**: hoe van een oplossing naar een buur-oplossing te gaan
- **energy**: berekent de waarde van de objectieve functie voor een oplossing

Metaheuristiek-specifieke berekeningen

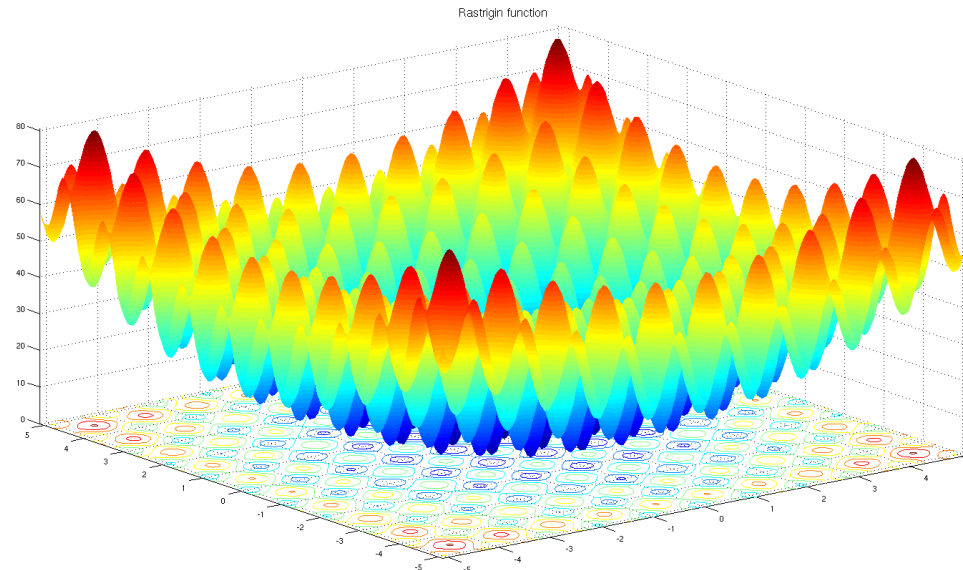
- Opgeven van de annealing parameters, zo niet, worden er default waarden gebruikt
- Uitvoeren van de simulated annealing heuristiek

Simulated annealing

Laten we aan de hand van een voorbeeld *simanneal* illustreren:

Rastrigin functie - klassieke case om optimalisatie algoritmen en heuristieken te testen

- Oplossingsruimte: (x, y) met x en $y \in [-5.12, 5.12]$
- Doelfunctie: $f(x) = 20 + x^2 - 10 \cdot \cos(2 \cdot \pi \cdot x) + y^2 - 10 \cdot \cos(2 \cdot \pi \cdot y)$
- Minimaliseer doelfunctie



Simulated annealing

Voorbeeld: Rastrigin functie

```
>>> from simanneal import Annealer

>>> class RastriginProblem(Annealer):
    def move(self):
        # x: self.state[0] en y: self.state[1]
        i = np.random.randint(0, 2)
        self.state[i] += np.random.normal(0, 0.1)
        self.state[i] = np.clip(self.state[i], -5.12, 5.12)

    def energy(self):
        sum = 20 + self.state[0]**2 - 10*math.cos(2*math.pi*self.state[0])
            + self.state[1]**2 - 10*math.cos(2*math.pi*self.state[1])
        return sum

>>> init_sol = np.random.uniform(-5.12, 5.12, size=2) # initiele [x, y]

>>> rastrigin = RastriginProblem(init_sol) # initiele [x, y] -> self.state

>>> # opgeven van de annealing parameters. Zo niet: default waarden
>>> rastrigin.anneal()
```


Simulated annealing

We hebben in ons voorbeeld verschillende default waarden gebruikt, maar deze kunnen worden gewijzigd - voordat het annealing proces wordt uitgevoerd -:

- Tmax = 25000.0 # Max (start) temperatuur
- Tmin = 2.5 # Min (eind temperatuur)
- temperature steps = 50000 # Aantal iteraties
- updates = 100 # Aantal updates (per default wordt een update geprint op stdout)

Dit zijn attributen van het object rasterin. Vb.:

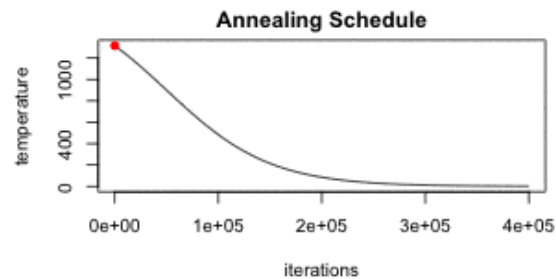
```
>>> rastrigin.Tmin = 25.0
```

Opmerkingen:

- Per default, wordt de *energy* functie **geminimaliseerd**, dus dient het resultaat van de objectieve functie met -1 te worden vermenigvuldigd indien de objectieve functie gemaximaliseerd dient te worden
- **Boundaries** op de waarden van de oplossing kunnen in de *move* functie worden opgenomen
- **Andere constraints** naast boundaries op de waarden van een oplossing dienen in de *energy* functie (= objectieve functie) te worden opgenomen.

Simulated annealing - TSP

Distance: 43,499 miles
Temperature: 1,316
Iterations: 0



Simulated annealing

Voorbeeld: TSP

```
distance_matrix = np.array([[0, 100, 125, 100, 75],
                             [100, 0, 50, 75, 100],
                             [125, 50, 0, 100, 125],
                             [100, 75, 100, 0, 50],
                             [75, 100, 125, 50, 0]])
```

```
>>> class TSPProblem(Annealer):
    def move(self):
        # Swaps two cities in the route
        a = np.random.randint(0, len(self.state))
        b = np.random.randint(0, len(self.state))
        self.state[a], self.state[b] = self.state[b], self.state[a]

    def energy(self):
        # Calculates the length of the route
        dist = 0
        for i in range(len(self.state)):
            dist += distance_matrix[self.state[i - 1], self.state[i]]
        return dist

>>> initial_state = [0, 4, 1, 3, 2]
>>> tsp = TravellingSalesmanProblem(initial_state)
>>> route, distance = tsp.anneal()
```

Simulated annealing

Voorbeeld: TSP

```
distance_matrix = np.array([[0, 100, 125, 100, 75],
                             [100, 0, 50, 75, 100],
                             [125, 50, 0, 100, 125],
                             [100, 75, 100, 0, 50],
                             [75, 100, 125, 50, 0]])
```

```
>>> class TSPProblem(Annealer):
    def move(self):
        # Swaps two cities in the route
        a = np.random.randint(0, len(self.state))
        b = np.random.randint(0, len(self.state))
        self.state[a], self.state[b] = self.state[b], self.state[a]

    def energy(self):
        # Length of the route without for loop
        from_city = self.state
        # shift the array one position to the right
        to_city = np.roll(from_city, -1)
        return distance_matrix[from_city, to_city].sum()

>>> initial_state = [0, 4, 1, 3, 2]
>>> tsp = TravellingSalesmanProblem(initial_state)
>>> route, distance = tsp.anneal()
```